UNIVERSIDAD SIGLO 21

Práctica Solidaria

Módulo 1

Trabajo Práctico 1



Caso de estudio: ReperTrack - Sistema Integrado de Gestión de Repertorio y Práctica Musical

Estudiante: Juan Martín Canguillen

DNI: 44933871

Profesor Títulor: Pablo Alejandro Virgolini

Carrera: Licenciatura en Informática
Año: 2026 — 2.° cuatrimestre

Fecha de entrega: 6 de septiembre de 2026

Índice

1. Introducción
   1.1. Antecedentes
   1.2. Descripción del Área Problemática
   1.3. Formulación de la Problemática
2. Justificación
   2.1. Necesidades que Satisface
   2.2. Impacto Tecnológico
   2.3. Relevancia para la Comunidad Musical
   2.4. Novedad y Factibilidad
3. Definiciones del Proyecto
   3.1. Objetivo General del Proyecto
   3.2. Objetivos Específicos del Proyecto
4. Definiciones del Sistema
   4.1. Objetivo General del Sistema
   4.2. Límites y Alcances
   4.3. No Contempla
5. Elicitación
   5.1. Actividad del Cliente / Dominio Musical
   5.2. T.I.C (Tecnologías de la Información y la Comunicación)
   5.3. Análisis de Competencia
   5.4. Requerimientos Funcionales (RFS)
   5.5. Requerimientos No Funcionales (RNF)
6. Conocimiento del Negocio (Relevamiento)
   6.1. Modelo de Dominio (Figura 1)
   6.2. Organigrama del Sistema Propuesto (Figura 2)
   6.3. Procesos de Negocio (Figura 3, cursograma)
7. Diagnóstico
8. Propuesta de Solución
   8.1. Propuesta Funcional
   8.2. Propuesta Técnica
   8.3. Diagrama de Arquitectura Lógica (Figura 4)
9. Requerimientos (con Trazabilidad)
   9.1. Requerimientos Funcionales con Trazabilidad
   9.2. Requerimientos No Funcionales
   9.3. Requerimientos Candidatos

10. Inicio del Análisis: Casos de Uso
  10.1. Actores del Sistema
  10.2. Diagrama de Casos de Uso (Figura 5)
  10.3. Especificación de Casos de Uso (Figura 6: secuencia de CU006)
11. Cierre del Documento
12. Referencias Bibliográficas
Anexo. Instrumentación de los Diagramas del Documento

1. Introducción

1.1. Antecedentes

La práctica musical, cuando se la toma en serio, es bastante demandante en términos de organización. Un músico de nivel intermedio o avanzado no se limita a dominar un instrumento: lleva adelante, en paralelo, un repertorio vivo de decenas de canciones en distintos grados de dominio, una biblioteca de ejercicios técnicos (escalas, arpegios, études, patrones rítmicos, transcripciones) y, por encima de todo, metas concretas a mediano y largo plazo. Lo curioso es que, en la enorme mayoría de los casos que uno observa a su alrededor, el control de estas rutinas se hace de manera bastante artesanal: cuadernos, planillas dispersas, anotaciones al margen de una partitura o, simplemente, la memoria del intérprete. Este problema no es nuevo —la literatura sobre práctica deliberada de Ericsson, Krampe y Tesch-Römer (1993) ya lo señalaba hace más de tres décadas, y Cal Newport (2016) lo reformula en clave contemporánea con su conocida idea de que «lo que se mide, se mejora»—, pero sigue sin tener una respuesta Informática adecuada para el nicho específico del músico individual.

1.2. Descripción del Área Problemática

El área problemática se ubica en la gestión cotidiana de la rutina de práctica de músicos solistas o de pequeños grupos de estudio que hoy no cuentan con herramientas informatizadas específicamente diseñadas para registrar y analizar su tiempo de estudio. Hay tres frentes que se ven afectados. En primer lugar, el músico en su práctica personal, que no dispone de un repositorio único donde anotar lo que trabaja en cada sesión: termina usando lo que tiene a mano, que suele ser poco durable o poco sistemático. En segundo término, la planificación del estudio a corto y mediano plazo, que se ve claramente dificultada por la ausencia de métricas de progreso: uno estima cuánto practicó, pero no lo sabe. Y, en tercer lugar, la toma de decisiones sobre qué obras incorporar, reforzar o retirar del repertorio, que termina apoyándose más en la intuición que en evidencia.

Los factores involucrados son de tres tipos. De naturaleza técnica: no existen aplicaciones de escritorio maduras para este nicho específico en Java/MySQL, y las soluciones genéricas disponibles (planillas o apps móviles) tienen demasiada fricción para registrar sesiones cronometradas de manera cómoda. De naturaleza organizacional: falta un modelo de datos consistente que vincule instrumento, elemento y sesión. Y de naturaleza cognitiva: aparece lo que en psicología se conoce como efecto Dunning-Kruger aplicado a la autoevaluación del esfuerzo —uno cree haber practicado mucho más de lo que realmente practicó— y eso refuerza la inercia de no medir.

1.3. Formulación de la Problemática

Con lo anterior, la pregunta de investigación que orienta este trabajo puede formularse así: ¿cómo se puede diseñar e implementar un sistema de información de escritorio que permita a un músico intermedio o avanzado gestionar de manera integral su repertorio, cronometrar y documentar sus sesiones de práctica vinculándolas a un instrumento y a un elemento de estudio específico, y producir información de progreso que respalde la toma de decisiones sobre la planificación de la rutina y el cumplimiento de metas de mediano y largo plazo?

Esa pregunta, traducida al lenguaje del PUD, es la que guía las técnicas de elicitación de la sección 5 y la posterior especificación de requerimientos de la sección 9. No es una pregunta retórica: cada uno de sus componentes («gestionar», «cronometrar», «vincular», «producir información de progreso») termina materializándose en un caso de uso concreto del sistema.

2. Justificación

2.1. Necesidades que Satisface

Soy músico desde los catorce años, así que la problemática que motiva a ReperTrack la conozco de primera mano. Llegué a un punto en el que mi repertorio ya no entraba en la cabeza, mis anotaciones vivían en un cuaderno que perdí varias veces, después pasaron a una planilla Excel que abandoné por cansancio, y al final terminaron en notas sueltas del celular mezcladas con todo lo demás. Eso resume lo que el sistema satisface, en concreto: la necesidad de un repositorio único donde queden registrados los instrumentos, el repertorio y las sesiones cronometradas; la necesidad de contar con un historial auditable y métricas concretas (tiempos acumulados, frecuencias, promedios), en lugar de estimaciones a ojo; y la necesidad de planificar la semana de estudio con base en evidencia y no en intuición. Para el conjunto de usuarios potenciales —la comunidad de músicos que se menciona en 2.3— la propuesta cubre exactamente la misma necesidad que las herramientas genéricas del mercado no atienden.

2.2. Impacto Tecnológico

El proyecto permite aplicar de forma integral el Proceso Unificado de Desarrollo (PUD), ejercitando competencias clave de la Licenciatura en Informática: análisis de requerimientos con técnicas de elicitación, modelado UML (diagrama de clases del dominio, diagrama de casos de uso y especificación tabular de casos), diseño orientado a objetos con aplicación concreta de herencia, polimorfismo y excepciones personalizadas, implementación en Java con el patrón MVC, persistencia relacional en MySQL mediante JDBC, y control de versiones en GitHub. A eso se suma el uso de colecciones del framework Collections (`ArrayList`, `HashMap`, `HashSet`) para mantener listas en memoria y de transacciones ACID para garantizar la integridad de los datos de práctica. Pocas veces un Trabajo Práctico de la carrera toca tantas asignaturas en forma simultánea.

2.3. Relevancia para la Comunidad Musical

En el plano de la comunidad de usuarios ReperTrack tiene una relevancia potencialmente alta, porque cualquier músico intermedio o avanzado que necesite planificar su estudio con datos es un destinatario natural. En la Argentina y la región existen miles de estudiantes de conservatorio, instrumentistas profesionales, docentes de instrumento y músicos amateurs avanzados que hoy gestionan su práctica con herramientas genéricas y poco adecuadas. Un sistema que democratice el acceso a prácticas basadas en evidencia tiene, en el mediano plazo, un impacto directo en la calidad interpretativa de esa comunidad y, derivadamente, en la oferta cultural local —una mejora que no se ve de un día para el otro, pero que empieza por ahí: por músicos que planifican mejor su semana.

2.4. Novedad y Factibilidad

La novedad de ReperTrack está en la combinación concreta de tres características que, juntas, no aparecen en las soluciones hoy disponibles. Primero, escritorio nativo en Java con persistencia relacional robusta, en lugar de las apps móviles con sincronización en la nube que dependen de conectividad y cuyos datos no son del usuario. Segundo, un modelo de dominio que distingue explícitamente Canciones y Ejercicios como especializaciones de Elemento de Estudio, modelando con bastante precisión la práctica real: uno no estudia «Clair de Lune» de la misma manera que estudia un patrón de arpegios, y el sistema lo reconoce. Tercero, un motor de cálculo de alertas de mantenimiento que detecta obras sin práctica reciente y se lo recuerda al músico, retroalimentando la planificación.

La factibilidad, por su parte, está garantizada por la disponibilidad de tecnologías maduras y de código abierto (Java 11+, MySQL 8+, JDBC, Swing o JavaFX), por el conocimiento de primera mano que tengo del dominio —lo que elimina la curva de aprendizaje del problema— y por la posibilidad concreta de probar el sistema con mi propia rutina de práctica a lo largo del desarrollo. Todo el código fuente se alojará en un repositorio público de GitHub (https://github.com/duendecillo2/reper-track), lo que facilita la trazabilidad de versiones, la revisión por pares y la eventual colaboración de otros estudiantes.

3. Definiciones del Proyecto

3.1. Objetivo General del Proyecto

Desarrollar ReperTrack, una aplicación de escritorio monousuario en Java con persistencia en MySQL y arquitectura MVC, que permita a músicos intermedios y avanzados gestionar integralmente su repertorio musical y cronometrar sus sesiones de práctica vinculadas a instrumentos y elementos de estudio específicos, generando estadísticas de progreso que respalden la planificación de metas de mediano y largo plazo.

3.2. Objetivos Específicos del Proyecto

Para alcanzar el objetivo general, el proyecto se descompone en los siguientes objetivos específicos, cada uno verificable al cierre de la iteración correspondiente del PUD:

- Implementar un ABM (Alta, Baja y Modificación) completo para las entidades Instrumento y Elemento de Estudio, garantizando la persistencia transaccional en MySQL y la validación de reglas de negocio mediante excepciones personalizadas.
- Desarrollar el módulo transaccional de registro de sesiones de práctica, que permita iniciar, pausar y finalizar un cronómetro, vincular la sesión a un instrumento y a un elemento de estudio determinados, y almacenar el resultado con fecha, hora de inicio, hora de fin y tiempo neto.
- Diseñar e implementar el módulo de estadísticas de progreso, capaz de calcular y visualizar, como mínimo: tiempo total acumulado por instrumento, tiempo total acumulado por elemento, frecuencia semanal de práctica y promedio de minutos por sesión.
- Aplicar el patrón de arquitectura Modelo-Vista-Controlador (MVC) separando de manera estricta la capa de presentación (vistas Swing o JavaFX), la lógica de control (controladores) y el acceso a datos (modelo de dominio y DAOs), de modo que cada capa sea susceptible de prueba y mantenimiento independiente.
- Construir un esquema de base de datos relacional normalizado en MySQL que respete las tres primeras formas normales (1FN, 2FN y 3FN), con integridad referencial mediante claves foráneas y restricciones UNIQUE para evitar duplicados lógicos.
- Versionar el código fuente y los scripts SQL en un repositorio de GitHub (https://github.com/duendecillo2/reper-track), con commits etiquetados por iteración del PUD y un README que documente los pasos de instalación y ejecución.
- Incorporar un módulo de exportación de reportes a PDF y de alertas automáticas de mantenimiento para los elementos sin práctica reciente, como features diferenciales respecto de las planillas genéricas.

4. Definiciones del Sistema

4.1. Objetivo General del Sistema

Recolectar y analizar los datos generados durante la práctica musical del usuario para producir información de progreso que respalde la toma de decisiones sobre qué obras ensayar, cuánto tiempo dedicar a cada una y qué elementos retirar del repertorio activo.

4.2. Límites y Alcances

El prototipo operacional cubre, de forma integral, las siguientes funcionalidades:

- ABM de instrumentos musicales, con atributos como nombre, tipo y observaciones opcionales.
- ABM de elementos de estudio, tanto canciones como ejercicios técnicos, con sus atributos diferenciadores.
- Registro cronometrado de sesiones de práctica, con asociación obligatoria a un instrumento y a un elemento previamente cargados.

- Consulta, filtrado y eliminación de sesiones registradas, con confirmación explícita.
- Visualización de estadísticas de progreso en formato tabular y, eventualmente, gráfico simple.
- Exportación del reporte de estadísticas a PDF (caso de uso CU011), para compartir el avance con profesores o colegas.
- Generación de alertas automáticas de mantenimiento sobre elementos que llevan demasiado tiempo sin practicarse (caso de uso CU012).
- Persistencia completa en base de datos MySQL local.

El límite temporal del prototipo coincide con la entrega final del Trabajo Práctico N.º 1; todas las funcionalidades aquí listadas deben estar operativas al cierre de esta entrega.

4.3. No Contempla

- La reproducción y grabación de audio: el sistema no va a tocar ni analizar señales de audio.
- La sincronización en la nube y el acceso multiusuario: no va a haber servidor de aplicaciones ni autenticación remota.
- La autenticación con usuario y contraseña en esta fase: se asume que el acceso al equipo físico restringe el acceso al software.
- Una aplicación móvil complementaria: el alcance se limita a la plataforma de escritorio.
- El análisis musical avanzado: detección de tono, transcripción automática y evaluación de precisión temporal quedan reservados para iteraciones futuras.
- La integración con metrónomos, capturadoras de audio o dispositivos MIDI: el registro es estrictamente manual mediante cronómetro.

5. Elicitación

5.1. Actividad del Cliente / Dominio Musical

El dominio de la práctica musical individual es bastante particular: no es un proceso industrial estandarizable ni un trámite administrativo. Es un ecosistema chico pero muy rico, donde interactúan el músico (cliente directo del sistema), su familia de instrumentos, su biblioteca de repertorio dividida en canciones y ejercicios técnicos, y los frutos de cada sesión registrada. Como aclaran Oliveros y Antonelli (2015) citando a Loucopoulos (1995), la elicitación apunta a producir un modelo de requerimientos a partir del conocimiento relevante del dominio, y aquí ese conocimiento no hay que inventarlo: lo tengo a mano.

El «cliente» de ReperTrack, en esta primera versión, soy yo mismo: un músico que viene gestionando su rutina de manera bastante artesanal, entre el papel y el Excel abandonado, y que conoce bien los puntos de fricción. La actividad concreta que el sistema informatiza se puede resumir en cinco pasos: decidir qué se va a practicar, levantar el cronómetro, vincular el tiempo transcurrido a un instrumento y a una obra o ejercicio, guardar el registro, y revisar periódicamente las métricas obtenidas para reorientar la planificación de la semana siguiente. Lo que el sistema no hace, y que vale la pena aclarar desde ya, es ejecutar o evaluar la música misma: no toca, no graba y no juzga la calidad del sonido producido, sólo asienta lo que el músico declara haber practicado y durante cuánto tiempo.

5.2. T.I.C (Tecnologías de la Información y la Comunicación)

El stack tecnológico está alineado con el plan de estudios de la carrera y con la consigna de la actividad. Lo detallo por capas:

Lenguaje de programación — Java 11 LTS

es el lenguaje central del plan de estudios y la opción natural para un escritorio multiplataforma. Uso específicamente la versión LTS por su estabilidad y porque coincide con la que está disponible en los equipos de la cátedra.

Base de datos — MySQL 8.0

es el motor relacional que pide la consigna. Ofrece transacciones ACID, claves foráneas con integridad referencial declarativa, índices sobre claves compuestas y conexión vía JDBC. Es la pieza donde termina persistido todo el historial de práctica, que es el verdadero capital del sistema.

Capa de persistencia — JDBC oficial de MySQL

como API estándar de Java para acceder a datos relacionales. Me permite enviar SQL parametrizado con `PreparedStatement` y controlar manualmente la transacción con `commit` y `rollback` cuando algo sale mal.

Capa de presentación — Swing, con JavaFX como alternativa

Swing viene incluida en el JDK y tiene los componentes suficientes para lo que necesito: `JTable` para los listados, formularios modales de alta y modificación, `javax.swing.Timer` para el cronómetro. Si más adelante hace falta una UI más moderna, JavaFX es perfectamente viable y se integra igual de bien con el resto del stack.

Gestión de dependencias — Maven

es el estándar de facto para proyectos Java. Declaro el conector MySQL en el `pom.xml` y me olvido de andar copiando JARs a mano.

IDE — Apache NetBeans (con IntelliJ como alternativa)

ambos libres. Me inclino por NetBeans por la integración nativa con Swing mediante su Editor de GUI (Matisse), pero IntelliJ IDEA Community funciona perfectamente.

Control de versiones — Git + GitHub

El repositorio público está en https://github.com/duendecillo2/reper-track. Esta elección me da trazabilidad de las iteraciones del PUD, respaldo del código y posibilidad de recibir feedback durante el desarrollo.

5.3. Análisis de Competencia

Antes de seguir convenciéndome a mí mismo de la pertinencia del proyecto, hice una revisión rápida de las soluciones ya disponibles en el mercado, para asegurarme de no estar reinventando la rueda. La Tabla 1 resume lo que encontré.

Característica	Planillas Excel / Google Sheets	Apps móviles genéricas (p. ej. Practice+, Toneduck)	Cuaderno físico	ReperTrack (propuesta)
Persistencia estructurada relacional	No (filas planas)	Sí (interna, sin exportación)	No	Sí (MySQL relacional)
Modelo Canción vs. Ejercicio diferenciado	Manual y propenso a errores	Limitado o confuso	Manual	Sí (herencia POO explícita)
Asociación obligatoria Sesión-Instrumento -Elemento	No validada	A veces, con fricción	Inexistente	Sí (validada por FK + reglas de negocio)
Cronómetro integrado en el flujo de registro	No	Sí	No	Sí
Estadísticas de progreso por instrumento y elemento	Manual con fórmulas	Básicas (tiempo total)	Inexistentes	Sí (acumulados, frecuencia, promedios)

Alertas de mantenimiento por inactividad	No	No	No	Sí (CU012)
Exportación a PDF para compartir con profesores	Manual (imprimir)	A veces con marca de agua	No	Sí (CU011, sin restricciones)
Plataforma	Multiplataforma	Móvil (Android/iOS)	Papel	Desktop multiplataforma (Java)
Privacidad de los datos (offline)	Depende de la nube	Habitualmente cloud-only	Total	Total (MySQL local)

Tabla 1: Análisis comparativo de soluciones existentes vs. ReperTrack. Fuente: elaboración propia.

Mirando la tabla con detenimiento, ninguna de las alternativas combina las tres cosas que a mí me hacían falta: un modelo relacional con herencia explícita entre Canciones y Ejercicios, la asociación obligatoria entre Sesión, Instrumento y Elemento validada por la propia base de datos, y alertas de mantenimiento por inactividad. La suma de esos tres huecos es, justamente, lo que ReperTrack viene a llenar.

5.4. Requerimientos Funcionales (RFS)

Siguiendo la tipificación de Sommerville (2005, p. 109), los requerimientos funcionales son las declaraciones de los servicios que debe proporcionar el sistema, la forma en que éste debe reaccionar a la entrada particular de datos, los procesos que debe hacer con sus controles y cómo debe comportarse en situaciones particulares. Cada uno se redactó procurando que cumpla las seis características que Díaz Ríos (s. f.) considera indispensables: que sea necesario, conciso, no ambiguo, completo, consistente y verificable.

Requerimiento	Descripción
RFS01	El sistema debe permitir el alta, baja lógica y modificación de instrumentos musicales, registrando obligatoriamente nombre y tipo (cuerda, viento, percusión, teclado u otro), con observaciones opcionales.
RFS02	El sistema debe permitir el alta, baja lógica y modificación de canciones, registrando nombre, autor, intérprete original, instrumento asociado y nivel de dificultad en escala de 1 a 10.
RFS03	El sistema debe permitir el alta, baja lógica y modificación de ejercicios técnicos, registrando nombre, tipo de ejercicio, tempo sugerido en BPM, instrumento asociado y nivel de dificultad.
RFS04	El sistema debe permitir registrar una sesión de práctica mediante un cronómetro interno con funciones de iniciar, pausar y finalizar.
RFS05	El sistema debe asociar obligatoriamente toda sesión de práctica a un instrumento y a un elemento de estudio previamente registrados, no admitiendo sesiones huérfanas.
RFS06	El sistema debe permitir listar y filtrar las sesiones registradas por instrumento, por elemento de estudio y por rango de fechas.
RFS07	El sistema debe permitir eliminar una sesión registrada, previa confirmación explícita del usuario.

RFS08	El sistema debe calcular y mostrar el tiempo total acumulado de práctica por instrumento y por elemento de estudio, expresado en minutos.
RFS09	El sistema debe calcular y mostrar la frecuencia semanal de práctica y el promedio de minutos por sesión, con posibilidad de segmentar por instrumento o por elemento.
RFS10	El sistema debe generar alertas automáticas de mantenimiento para los elementos de estudio que no hayan sido practicados durante un período configurable (caso de uso CU012).
RFS11	El sistema debe permitir exportar a PDF el reporte de estadísticas de progreso (caso de uso CU011), para facilitar su compartición con profesores o colegas.

5.5. Requerimientos No Funcionales (RNF)

Conforme con la definición de Sommerville (2005), los requerimientos no funcionales son consideraciones que limitan la forma en que será implementado el sistema y que, por lo general, se aplican al sistema en su conjunto. Su alcance se define a través de conceptos tales como performance, precisión, confiabilidad, seguridad, portabilidad y restricciones de la tecnología a utilizar.

Requerimiento	Descripción
RNF01	El sistema debe estar desarrollado íntegramente en lenguaje Java, en su versión 11 o superior, utilizando Swing o JavaFX para la capa de presentación.
RNF02	El sistema debe persistir la información en una base de datos relacional MySQL 8 o superior, accedida mediante el conector JDBC oficial.
RNF03	El sistema debe respetar el patrón arquitectónico Modelo-Vista-Controlador (MVC), separando de forma estricta la capa de presentación, la lógica de control y el modelo de dominio y acceso a datos.
RNF04	El código fuente completo, los scripts SQL de creación del esquema y los datos de prueba deben alojarse en un repositorio público de GitHub, con estructura de commits limpia y un README que documente los pasos de instalación y ejecución. Repositorio: https://github.com/duendecillo2/reper-track.
RNF05	El sistema debe ejecutar todas las operaciones de escritura sobre la base de datos dentro de transacciones ACID, garantizando que ante un fallo no quede información inconsistente.
RNF06	La interfaz gráfica debe proporcionar retroalimentación inmediata ante toda acción del usuario y exigir confirmación explícita para cualquier operación de eliminación de datos.
RNF07	El sistema debe ejecutarse correctamente en equipos con Windows 10 o superior, Linux con kernel 5.x y macOS 11 o superior, sin requerir recompilación del código.

6. Conocimiento del Negocio (Relevamiento)

6.1. Modelo de Dominio

El sistema ReperTrack se ubica en el dominio de la gestión personal de la práctica musical. Como explica Saroka (2002, p. 26), todo sistema puede estudiarse desde un enfoque estático que identifica sus elementos, sus relaciones y sus objetivos; es justamente ese enfoque el que se aplica a continuación para describir el modelo de dominio, que va a servir como insumo para el posterior diagrama de clases UML mostrado en la Figura 1.

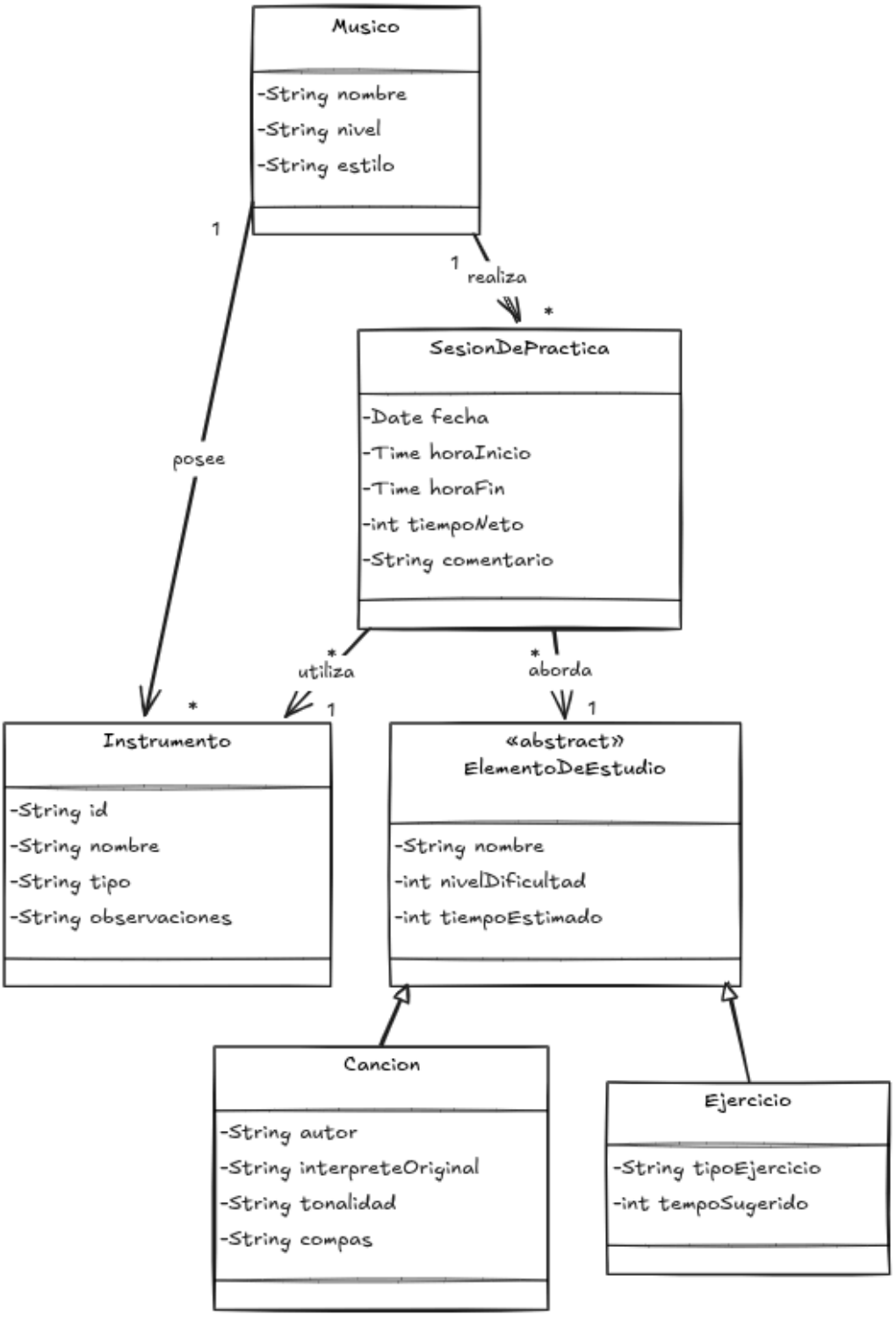


Figura 1. Diagrama de dominio de ReperTrack. Fuente: elaboración propia.

Los elementos del dominio, con sus atributos característicos, son los siguientes.

- Instrumento: nombre, tipo (cuerda, viento, percusión, teclado u otro), observaciones, identificador.

- Elemento de Estudio (clase abstracta con dos especializaciones): nombre, nivel de dificultad (1-10), tiempo estimado de dominio en minutos, instrumento asociado.
- Canción (subclase de Elemento de Estudio): autor, intérprete original, tonalidad, compás.
- Ejercicio (subclase de Elemento de Estudio): tipo de ejercicio (escala, arpegio, étude, patrón rítmico), tempo sugerido en BPM.
- Sesión de Práctica: fecha, hora de inicio, hora de fin, tiempo neto en minutos, comentario libre, instrumento asociado, elemento asociado.
- Músico: nombre, nivel (principiante, intermedio, avanzado), estilo (clásico, jazz, popular, folklórico).

Las relaciones entre los elementos son: un Músico posee uno o más Instrumentos (1 a N); un Instrumento se asocia con muchos Elementos de Estudio (1 a N); un Elemento de Estudio se asocia con muchas Sesiones de Práctica (1 a N). Adicionalmente, cada Sesión de Práctica se vincula directamente con un Instrumento (lo que en el modelo conceptual aparece como relación ternaria).

6.2. Organigrama del Sistema Propuesto

A nivel funcional, ReperTrack se organiza en tres capas bien delimitadas por el patrón MVC, cada una con responsabilidades específicas y colaboradores humanos o técnicos asociados. La Figura 2 presenta esta estructura.

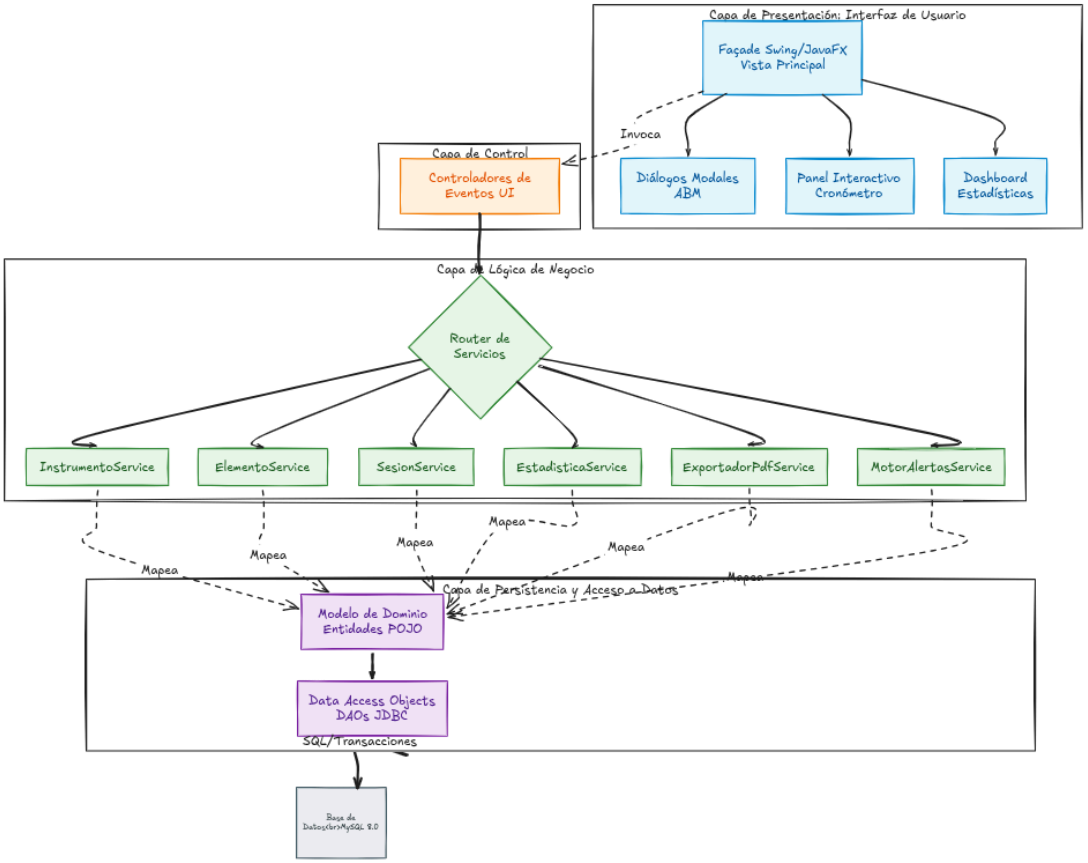


Figura 2. Organigrama del sistema ReperTrack propuesto. Fuente: elaboración propia.

Área de Presentación (Swing/JavaFX): integrada por las pantallas que el músico utiliza directamente: formularios de ABM, panel del cronómetro, listado de sesiones, dashboard de estadísticas y visor de alertas. Colaborador: Músico.

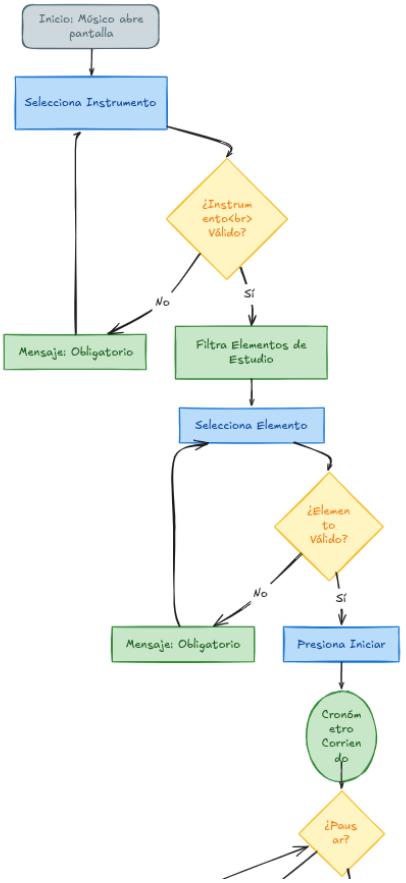
Área de Lógica de Negocio (Controladores + Servicios): capa que recibe los eventos de la vista, invoca al modelo y coordina las transacciones. Aquí viven, por ejemplo, el Servicio de Sesiones (que aplica

las reglas de negocio antes de invocar al DAO) y el Servicio de Estadísticas (que agrega datos para los reportes). Colaborador: Músico (interacción indirecta).

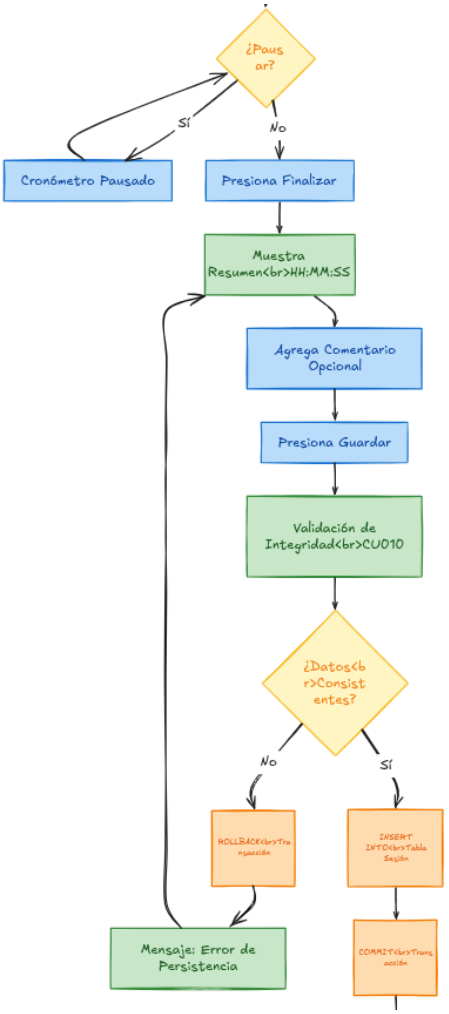
Área de Modelo y Persistencia (Entidades, DAOs, MySQL): la capa más profunda, organizada en paquetes que reflejan el modelo de dominio. Los DAOs encapsulan las operaciones de JDBC y exponen métodos CRUD tipados al resto del sistema. Colaborador externo: Sistema de Persistencia (MySQL) y Sistema de Archivos (para la exportación a PDF).

6.3. Procesos de Negocio

A continuación se describen los procesos de negocio principales del sistema. Cada uno se documenta con su responsable, sus entradas, su flujo principal y su resultado verificable. La Figura 3 muestra el cursograma del proceso crítico: Registrar Sesión de Práctica.



(continuación →)



(continuación →)

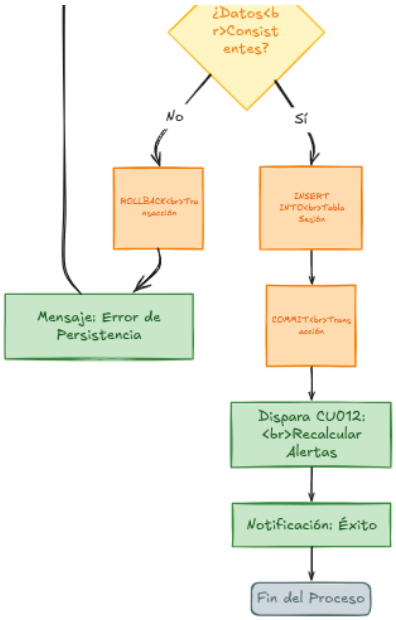


Figura 3. Cursograma del proceso «Registrar Sesión de Práctica» (compuesto por tres paneles en sentido vertical). Fuente: elaboración propia.

Proceso 1 — Gestión de Instrumentos:

El músico registra sus instrumentos en el sistema mediante un formulario de alta. Cada instrumento requiere nombre y tipo obligatorios, y observaciones opcionales. La modificación y la baja lógica siguen el flujo estándar de cualquier ABM. La consulta permite listar y buscar instrumentos existentes para asociarlos posteriormente a elementos de estudio.

Proceso 2 — Gestión de Elementos de Estudio:

El músico da de alta Canciones y Ejercicios asociándolos a un instrumento previamente cargado. El sistema valida que el instrumento exista (regla de negocio cubierta por la relación «include» del caso de uso CU003 hacia CU010). Los atributos editables varían según el tipo concreto: las Canciones agregan autor, intérprete, tonalidad y compás; los Ejercicios agregan tipo de ejercicio y tempo en BPM.

Proceso 3 — Registrar Sesión de Práctica:

El músico selecciona Instrumento y Elemento de Estudio, inicia el cronómetro integrado, lo pausa si lo necesita, lo finaliza al terminar, agrega un comentario libre opcional y guarda. El sistema ejecuta la operación dentro de una transacción ACID y muestra un mensaje de confirmación. Este es el proceso nuclear del sistema y se especifica con detalle en el caso de uso CU006 (sección 10.3).

Proceso 4 — Consultar Estadísticas de Progreso:

El músico accede al módulo de estadísticas, donde el sistema calcula agregados a partir de la tabla de sesiones: tiempo total por instrumento, tiempo total por elemento, frecuencia semanal y promedio de minutos por sesión. La consulta puede segmentarse por instrumento, por elemento o por rango de fechas.

Proceso 5 — Generar Alertas de Mantenimiento:

El sistema analiza periódicamente la fecha de última práctica de cada elemento y, si supera un umbral configurable (por defecto, 30 días), genera una alerta visible en el panel principal, sugiriéndole al músico retomar esa obra o ejercicio. Este proceso se dispara automáticamente al cerrar cada sesión de práctica.

Proceso 6 — Exportar Reporte a PDF:

El músico solicita la exportación a PDF del reporte de estadísticas. El sistema genera un documento con los indicadores principales y lo almacena en una ruta seleccionada por el usuario, de modo que pueda adjuntarlo a correos o imprimirlo para llevarlo a su profesor.

7. Diagnóstico

Lo que sigue es un cuadro sintético de los problemas que detecté en el relevamiento, junto con la causa raíz que los explica. Cada par problema-causa es el disparador de un módulo concreto de la propuesta de solución de la sección 8.

Problema 1 — No hay registro cronometrado y persistente de las sesiones.

Causa: ninguno de los recursos que usa el músico hoy (papel, Excel, app de notas) integra cronómetro, instrumento y elemento de estudio en un mismo flujo, ni persiste el resultado de manera estructurada.

Problema 2 — Es imposible saber a ciencia cierta cuánto se practicó cada obra o ejercicio.

Causa: las anotaciones manuales son parciales, se pierden, no distinguen canciones de ejercicios y no permiten cálculos agregados confiables; las apps existentes no exponen estas métricas con granularidad.

Problema 3 — Las decisiones de planificación se apoyan en la intuición.

Causa: sin métricas históricas, el músico no puede responder con fundamento preguntas del estilo «¿qué obra tengo más descuidada?» o «¿estoy cumpliendo la meta semanal?».

Problema 4 — No hay alertas oportunas sobre obras en riesgo de olvidarse.

Causa: ningún sistema se acuerda del músico. Después de una sesión larga, nadie le recuerda que aquella canción empezada hace tres meses ya lleva 45 días sin que la toque.

En conjunto, estos cuatro problemas describen la situación a la que ReperTrack apunta. La sección 8 muestra cómo la propuesta de solución los ataca uno por uno.

8. Propuesta de Solución

Una vez realizado el diagnóstico, se presenta la propuesta de solución articulando cuatro pilares: la propuesta funcional, la propuesta técnica, el diagrama de arquitectura lógica y el modelo de componentes por capas.

8.1. Propuesta Funcional

Lo que propongo es ReperTrack propiamente dicho: un sistema de escritorio de gestión y monitoreo de la práctica musical, organizado en cuatro módulos funcionales. Los detallo en orden de aparición para el usuario.

Módulo de Gestión de Instrumentos y Elementos de Estudio

es el primer módulo con el que el músico interactúa al instalar el sistema. Ofrece el CRUD completo de instrumentos y de elementos de estudio —canciones y ejercicios técnicos por separado, con sus atributos diferenciadores—, más la asociación obligatoria de cada elemento a un instrumento. Es el núcleo de datos sobre el cual se apoya todo lo demás.

Módulo Transaccional de Registro de Sesiones

es el corazón del sistema, el que justifica el proyecto. Permite cronometrar y persistir sesiones de práctica vinculadas a un instrumento y a un elemento específico, con manejo de pausas, comentarios libres y transacciones ACID. Se documenta con más detalle en el caso de uso CU006, sección 10.3.

Módulo de Estadísticas y Reportes

calcula agregados a partir del historial de sesiones (tiempos acumulados, frecuencia semanal, promedios) y los presenta al músico en formato tabular. Además, ofrece dos funcionalidades que en mi experiencia son las que más valor agregan: la exportación del reporte a PDF (caso de uso CU011) y las alertas automáticas de mantenimiento por inactividad (CU012).

8.2. Propuesta Técnica

La arquitectura respeta el patrón MVC con capas bien delimitadas. Las describo por separado:

Capa Modelo

Clases de dominio `Musico`, `Instrumento`, `ElementoDeEstudio` (abstracta), `Cancion`, `Ejercicio`, `SesionDePractica`, más las interfaces DAO con sus operaciones CRUD tipadas. La separación entre entidades puras y DTOs está pensada para no filtrar detalles del modelo hacia la vista.

Capa Controlador

Son controladores Swing/JavaFX que gestionan eventos de la UI —botones, cambios de selección, acciones de menú—. La lógica de negocio vive en servicios dedicados: `SesionService`, `ElementoService`, `InstrumentoService`, `EstadisticaService`, `ExportadorPdfService` y `MotorAlertasService`. Esta separación es la que después permite hacer pruebas unitarias sin necesidad de levantar la UI.

Capa Vista

Las pantallas Swing tienen layouts bien diferenciados: panel principal con menú, diálogos modales para los ABM, tablas de listado (`JTable` con su `TableModel`), panel del cronómetro y dashboard de estadísticas con cards para los indicadores principales.

Capa de Persistencia

MySQL 8.0 con dos índices compuestos pensados para acelerar las consultas típicas: `idx_sesion_instrumento_ts` sobre `(instrumento_id, fecha)` y `idx_sesion_elemento_ts` sobre `(elemento_id, fecha)`, ambos útiles para el módulo de estadísticas.

Manejo de transacciones

toda escritura en la base de datos se ejecuta dentro de una transacción controlada manualmente: `con.setAutoCommit(false)` al inicio, `con.commit()` si todo sale bien, `con.rollback()` ante cualquier `SQLException`. Esto es lo que en RNF05 se describe como requisito de confiabilidad y es la única manera sensata de garantizar que no queden sesiones a medio guardar.

8.3. Diagrama de Arquitectura Lógica

La Figura 4 muestra la arquitectura lógica del sistema, organizada en tres tiras horizontales: capa de Presentación (interfaz Swing/JavaFX), capa de Lógica de Negocio (controladores y servicios), y capa de Persistencia (modelo de dominio + DAOs + MySQL). Los tres casos de uso subordinados (Validar Integridad Referencial, Exportar Reporte en PDF, Calcular Alertas de Mantenimiento) se muestran como servicios invocados desde los casos de uso principales.

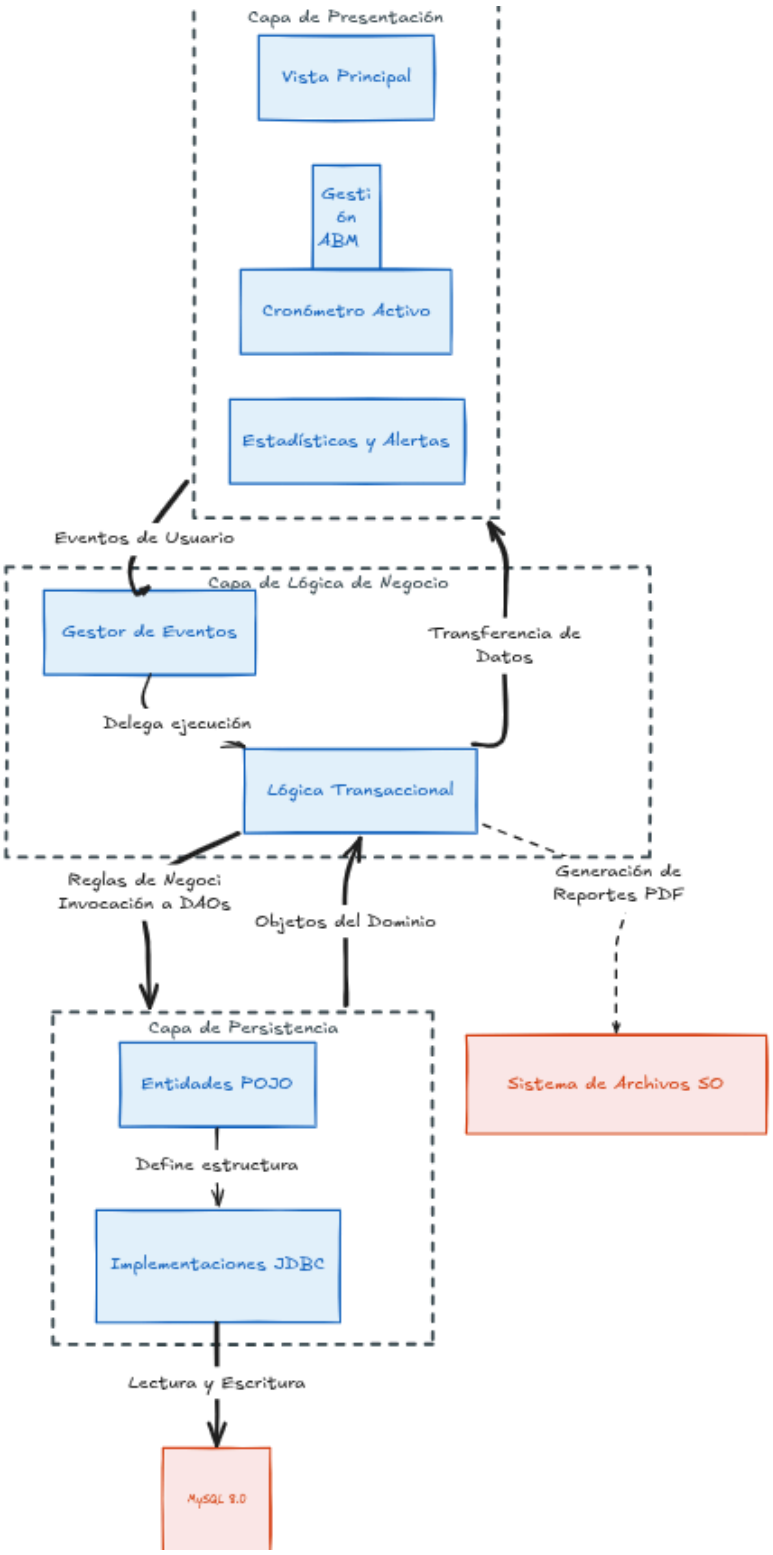


Figura 4. Arquitectura lógica propuesta para ReperTrack. Fuente: elaboración propia.

9. Requerimientos (con Trazabilidad)

9.1. Requerimientos Funcionales con Trazabilidad

La Tabla 2 consolida los requerimientos funcionales del sistema indicando, para cada uno, su prioridad (Alta / Media / Baja), su complejidad estimada (Alta / Media / Baja) y su trazabilidad respecto a los casos de uso del sistema. Esta vista unificada es la que recomienda el PUD para verificar la cobertura de los casos de uso sin perder de vista el nivel de esfuerzo.

ID	Descripción	Prioridad	Complejidad	Caso de uso
RFS01	Alta, baja y modificación de instrumentos musicales (nombre, tipo, observaciones).	Alta	Baja	CU001
RFS02	Alta, baja y modificación de canciones con atributos específicos.	Alta	Media	CU003
RFS03	Alta, baja y modificación de ejercicios técnicos con atributos específicos.	Alta	Media	CU003
RFS04	Cronómetro interno con iniciar/pausar/finalizar para registrar sesiones.	Alta	Alta	CU006
RFS05	Asociación obligatoria de sesión a instrumento y elemento (sin sesiones huérfanas).	Alta	Media	CU006 / CU010
RFS06	Listado y filtrado de sesiones por instrumento, elemento y rango de fechas.	Alta	Media	CU007
RFS07	Eliminación de sesión con confirmación explícita.	Media	Baja	CU007
RFS08	Cálculo de tiempo total acumulado por instrumento y por elemento.	Media	Media	CU009
RFS09	Cálculo de frecuencia semanal y promedio de minutos por sesión.	Media	Media	CU009

RFS10	Alertas automáticas de mantenimiento por inactividad.	Media	Alta	CU012
RFS11	Exportación del reporte de estadísticas a PDF.	Baja	Media	CU011

Tabla 2: Requerimientos funcionales con trazabilidad. Fuente: elaboración propia.

9.2. Requerimientos No Funcionales

Conforme con la definición de Sommerville (2005), los requerimientos no funcionales limitan la forma en que será implementado el sistema. La Tabla 3 los lista.

ID	Categoría	Descripción
RNF01	Implementación	Desarrollo en Java 11 o superior, con Swing o JavaFX para la capa de presentación.
RNF02	Implementación	Persistencia en MySQL 8 o superior, accedida mediante JDBC oficial.
RNF03	Arquitectura	Patrón Modelo-Vista-Controlador (MVC) separando presentación, lógica y modelo.
RNF04	Calidad	Repositorio público en GitHub con commits limpios y README documentado. URL: https://github.com/duendecillo2/reper-track.
RNF05	Confiabilidad	Transacciones ACID en todas las operaciones de escritura, con rollback automático ante SQLException.
RNF06	Usabilidad	Retroalimentación inmediata ante toda acción del usuario y confirmación explícita en eliminaciones.
RNF07	Portabilidad	Ejecución en Windows 10+, Linux kernel 5.x y macOS 11+ sin recompilación.
RNF08	Mantenibilidad	Cobertura de pruebas unitarias sobre la capa de servicios y DAOs (XUnit/JUnit).

Tabla 3: Requerimientos no funcionales. Fuente: elaboración propia.

9.3. Requerimientos Candidatos

Siguiendo la tipificación de Sommerville (2005), los requerimientos candidatos son aquellos requerimientos funcionales que no se han tenido en cuenta en primera instancia, pero que son de negocio y que, con el tiempo, pueden convertirse en requerimientos del dominio. Para ReperTrack se han identificado los siguientes, todos pensando en iteraciones futuras posteriores al cierre de esta entrega:

- Integración con metrónomos digitales y dispositivos MIDI para registrar tempo real durante la sesión.
- Aplicación móvil complementaria (Android) que sincronice con el escritorio vía archivo de intercambio.

- Reconocimiento automático de la canción que se está practicando mediante captura de audio y fingerprinting.
- Módulo de planificación semanal con metas configurables (por ejemplo, «30 minutos diarios de Clair de Lune»).
- Reportes comparativos entre músicos del mismo nivel y del mismo instrumento (benchmarking anonimizado).
- Exportación del historial completo a CSV para análisis externo en herramientas de Business Intelligence.
- Notificaciones push vía correo electrónico cuando una alerta de mantenimiento se dispara.
- Soporte multiusuario con autenticación por OAuth y base de datos compartida en la nube.

10. Inicio del Análisis: Casos de Uso

10.1. Actores del Sistema

- Músico: persona usuaria del sistema. Es quien registra instrumentos, elementos de estudio y sesiones de práctica, y quien consulta las estadísticas generadas. Es el actor principal e interactúa directamente con los casos de uso CU001, CU003, CU006, CU007, CU009, CU011 y CU012.
- Sistema de Persistencia: capa lógica que gestiona la lectura y escritura de datos en MySQL mediante JDBC. Se modela como actor secundario porque es quien efectivamente ejecuta las validaciones de integridad referencial del caso de uso CU010.
- Reloj del Sistema: fuente de tiempo del sistema operativo que provee la marca temporal (timestamp) utilizada por CU006 para registrar fecha, hora de inicio y hora de fin de cada sesión.
- Sistema de Archivos: componente del sistema operativo que recibe el archivo PDF generado por CU011 al exportar el reporte de estadísticas.

La Figura 5 muestra las relaciones de extensión e inclusión que se explicitan a continuación: CU001, CU003 y CU006 incluyen, mediante «include», a CU010 (Validar Integridad Referencial), de modo que toda operación de escritura dispara automáticamente la validación de claves foráneas y restricciones UNIQUE antes de confirmar la transacción. CU011 extiende, mediante «extend», a CU009 (Consultar Estadísticas de Progreso): cuando el músico consulta las estadísticas puede optar, como comportamiento adicional, por exportar el reporte. CU012 extiende a CU006: al finalizar el registro de una sesión, el sistema puede recalcular las alertas de elementos sin práctica reciente. Los tres casos de uso subordinados (CU010, CU011 y CU012) se dibujan fuera del rectángulo del sistema, en línea con la convención UML, para indicar que no son iniciados directamente por el Músico desde el menú principal.

10.2. Diagrama de Casos de Uso

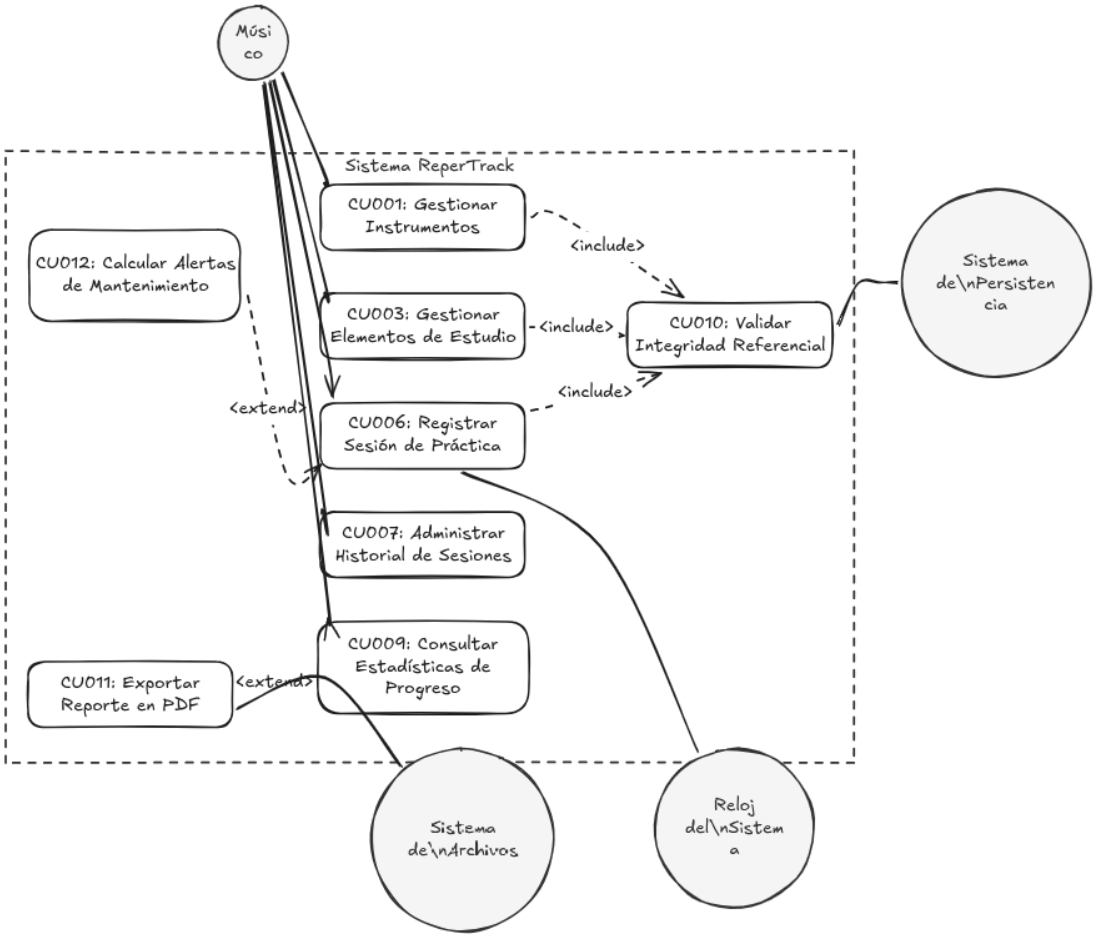


Figura 5. Diagrama de casos de uso de ReperTrack. Fuente: elaboración propia.

10.3. Especificación de Casos de Uso

Se selecciona como caso crítico a especificar en detalle el CU006 (Registrar Sesión de Práctica), por ser el corazón transaccional del sistema y el que articula el resto de las entidades. Adicionalmente, se incluye la Tabla 4 con la trazabilidad entre CU y RF.

Requerimiento(s)	Caso de uso	Actor	Relación	Paquete	Comentario
RFS01	CU001	Músico	«include» CU010	Instrumento	Gestionar instrumentos musicales: alta, baja y modificación. Incluye validación de integridad referencial.
RFS02, RFS03	CU003	Músico	«include» CU010	Elemento de Estudio	Gestionar canciones y ejercicios técnicos. Incluye validación de integridad referencial.

RFS04, RFS05	CU006	Músico	«include» CU010 · «extend» CU012	Sesión de Práctica	Registrar una sesión de práctica cronometrada, vinculada a instrumento y elemento. Incluye validación referencial y puede disparar recálculo de alertas.
RFS06, RFS07	CU007	Músico	(independiente)	Sesión de Práctica	Administrar el historial: listar, filtrar y eliminar sesiones registradas.
RFS08, RFS09	CU009	Músico	«extend» CU011	Estadísticas	Consultar los indicadores de progreso (acumulados, frecuencia, promedios). Permite extender hacia la exportación a PDF.
RFS01, RFS02, RFS03, RFS04, RFS05 (validación)	CU010	Sistema de Persistencia	(invocado por include)	Persistencia	Validar la integridad referencial al momento de cada operación de escritura. No es invocado directamente por el músico: se dispara automáticamente como «include» desde CU001, CU003 y CU006.
RFS11	CU011	Músico · Sistema de Archivos	«extend» de CU009	Estadísticas	Exportar el reporte de estadísticas a un archivo PDF sobre el Sistema de Archivos.
RFS10	CU012	Músico	«extend» de CU006	Estadísticas	Calcular y notificar alertas de mantenimiento para temas sin

					práctica reciente. Se dispara automáticamente al finalizar una sesión.

Tabla 4: Casos de uso con trazabilidad. Fuente: elaboración propia.

Especificación detallada: CU006 — Registrar Sesión de Práctica

Se utiliza el template de elaboración propia basado en la estructura propuesta por la bibliografía del PUD.

Campo	Detalle
Nivel	Sistema de Información
Nro.	CU006
Nombre	Registrar Sesión de Práctica.
Actores	Músico (principal); Sistema de Persistencia (secundario); Reloj del Sistema (secundario).
Descripción	Permite al músico registrar una sesión de práctica cronometrada, asociándola obligatoriamente a un instrumento y a un elemento de estudio previamente cargados en el sistema, y persistiendo el resultado en la base de datos MySQL dentro de una transacción atómica.
Precondiciones	1) El músico ha iniciado la aplicación. 2) Existe al menos un instrumento registrado. 3) Existe al menos un elemento de estudio (canción o ejercicio) registrado y asociado a un instrumento. 4) El músico seleccionó la opción «Registrar Sesión» desde el menú principal.
Postcondiciones	1) Se ha creado un nuevo registro en la tabla Sesion con todos los datos de la sesión. 2) Las estadísticas de progreso reflejan inmediatamente el nuevo tiempo acumulado. 3) Se disparó la recálcula de alertas de mantenimiento (CU012). 4) El sistema queda en estado listo para registrar una nueva sesión.
Curso Normal	1) El sistema muestra la pantalla de registro con los combos de Instrumento y Elemento de Estudio, y un cronómetro detenido en 00:00:00. 2) El músico selecciona un instrumento. El sistema filtra dinámicamente los Elementos de Estudio asociados al instrumento elegido. 3) El músico selecciona un elemento. 4) El sistema valida que ambos datos estén seleccionados (ver Alternativa A1). 5) El músico presiona «Iniciar» y el cronómetro comienza a contar. 6) El músico puede pausar/reanudar el cronómetro si lo necesita. 7) El músico presiona «Finalizar» y el cronómetro se detiene. 8) El sistema muestra resumen: instrumento, elemento, tiempo neto en HH:MM:SS y fecha actual. 9) El músico puede agregar un comentario libre opcional. 10) El músico presiona «Guardar Sesión». 11) El sistema abre una transacción JDBC, ejecuta el INSERT en la tabla Sesion y ejecuta COMMIT. 12) El sistema

	muestra confirmación exitosa y vuelve a la pantalla principal.
Alternativas	A1 — Datos incompletos: si al iniciar el cronómetro falta instrumento o elemento, el sistema muestra un diálogo modal con el mensaje «Debe seleccionar un instrumento y un elemento de estudio antes de iniciar la sesión», resalta los campos faltantes en rojo y vuelve al paso 1. A2 — Cancelación: el músico puede presionar «Cancelar» en cualquier momento; el sistema pide confirmación y descarta la sesión en curso. A3 — Sin elementos asociados: si el instrumento seleccionado no tiene elementos cargados, el sistema muestra un mensaje sugiriendo crear uno desde CU003.
Excepciones	E1 — Error de persistencia (SQLException): el sistema ejecuta ROLLBACK, registra el error en bitácora con marca temporal, muestra al usuario «Ocurrió un error al guardar la sesión. Sus datos no se perdieron; intente nuevamente» y vuelve al paso 9 sin perder los datos capturados en pantalla. E2 — Falla del Reloj del Sistema: si no se puede obtener timestamp, el sistema rechaza el inicio del cronómetro con un mensaje claro.
Reglas de Negocio	RN-01: Toda sesión debe tener instrumento y elemento. RN-02: El tiempo neto debe ser mayor a cero. RN-03: La fecha de la sesión no puede ser futura. RN-04: El comentario libre, si está presente, no puede superar los 500 caracteres. RN-05: Atomicidad transaccional garantizada. RN-06: Al guardarse la sesión, se recalculan las alertas de mantenimiento (CU012).

Tabla 5: Especificación detallada del caso de uso CU006. Fuente: elaboración propia.

Para complementar la especificación tabular y mostrar la dinámica temporal del caso de uso, la Figura 6 presenta el diagrama de secuencia correspondiente. Allí se aprecia cómo el Músico, la Vista (Swing), el Controlador de Sesión, el Servicio de Sesión, el Componente Transaccional, el DAO de Sesión y MySQL se turnan durante el flujo de una sesión exitosa y de una sesión que termina en `SQLException` con rollback.

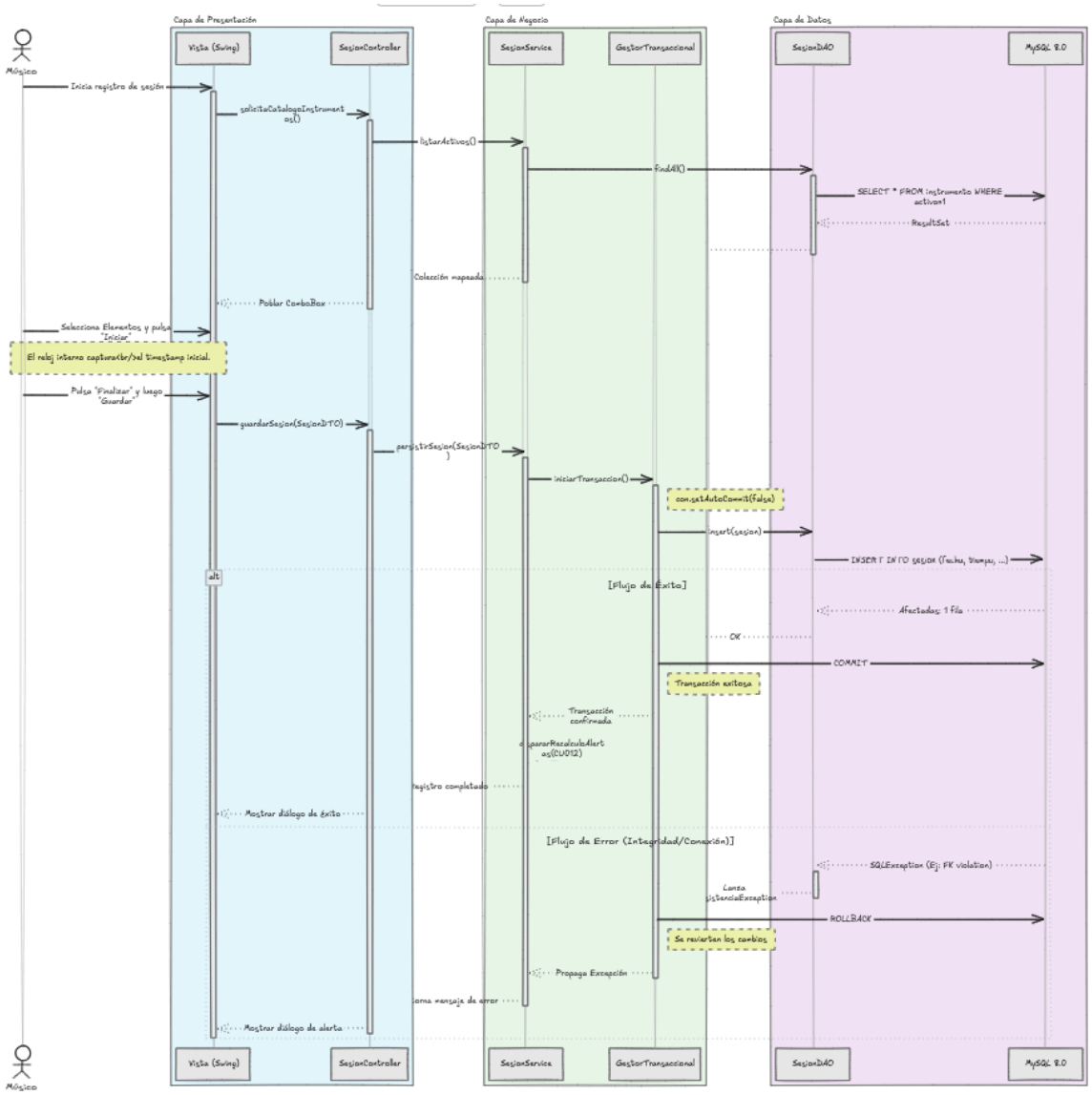


Figura 6. Diagrama de secuencia para el caso de uso CU006 (Registrar Sesión de Práctica). Fuente: elaboración propia.

11. Cierre del Documento

Lo que dejé escrito hasta acá es, en esencia, la formalización de la fase de Inicio del Proceso Unificado de Desarrollo para ReperTrack. Con el marco teórico de Saroka (2002), Van Gigch (1995), Stair y Reynolds (2010), Cedeño (2017), Churchman (1968), Sommerville (2005), Kendall y Kendall (2011) y Oliveros y Antonelli (2015), y partiendo de un problema que conozco de primera mano como músico, quedaron planteados el problema, la justificación, los objetivos, el alcance, los requerimientos, el modelo de dominio, la arquitectura propuesta y la especificación del caso de uso crítico. Las iteraciones que siguen en el PUD —Elaboración, Construcción y Transición— se apoyarán en todo esto para producir un sistema de escritorio robusto, mantenible y alineado con los objetivos de aprendizaje de la materia Seminario de Práctica de la Licenciatura en Informática de la Universidad Siglo 21.

Personalmente, este trabajo me deja dos cosas. La primera, una herramienta que estoy seguro me va a servir en mi propia práctica musical —algo que no es menor después de tantos años improvisando el seguimiento. La segunda, la ocasión de aplicar de forma integrada buena parte de lo aprendido en la carrera, en un dominio que me interesa genuinamente y donde el resultado va a tener un impacto real y medible, que es al final lo que uno busca cuando se embarca en un Trabajo Práctico de este tipo.

12. Referencias Bibliográficas

Benvenuto, D. (s. f.). Unidad I. Entrevistas. Master Net. Servicios IT. https://www.danielbenvenuto.com/EDUCACION/PP/Contenido/Unidad-1/Entrevista.htm

Cedeño, K. (2017). Modelado bajo la notación de gestión de procesos de negocio de la gestión de problemas en el área de soporte [Tesis de grado]. Universidad Central de Venezuela.

Churchman, W. (1968). El enfoque de sistemas. Iana Técnico.

Csikszentmihalyi, M. (1990). Flow: The psychology of optimal experience. Harper & Row.

Díaz Ríos, R. (s. f.). Requerimientos de Proceso. Planificación y Modelado. https://planificacionymodelado.angelfire.com/requerimientos-de-proceso.html

Ericsson, K. A., Krampe, R. T., & Tesch-Römer, C. (1993). The role of deliberate practice in the acquisition of expert performance. Psychological Review, 100(3), 363–406.

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.

Kendall, K., & Kendall, J. (2011). Análisis y Diseño de Sistemas. Pearson Educación.

Mitleton-Kelly, E. (2003). Complex systems and evolutionary perspectives on organisations. Pergamon.

Newport, C. (2016). Deep work: Rules for focused success in a distracted world. Grand Central Publishing.

Oliveros, A., & Antonelli, L. (2015). Técnicas de elicitación de requerimientos. XXI Congreso Argentino de Ciencias de la Computación, Junín, Argentina.

Saroka, R. (2002). Sistemas de Información en la era digital. Fundación OSDE.

Sommerville, I. (2005). Ingeniería del software (7.ª ed.). Pearson-Addison Wesley.

Stair, R., & Reynolds, G. (2010). Principios de Sistemas de Información (9.ª ed.). Cengage Learning.

Van Gigch, J. (1995). Teoría General de Sistemas. Trillas.

Anexo. Instrumentación de los Diagramas del Documento

Quedó acá una nota breve sobre cómo se generaron las figuras UML del documento, para que cualquier revisor pueda reproducir o mejorar los diagramas si fuera necesario. Todas las imágenes están embebidas en el PDF y editables en el archivo .docx que acompaña esta entrega.

Figuras y herramientas utilizadas

Figura	Contenido	Herramienta con la que se generó
Figura 1	Diagrama de dominio de ReperTrack (clases Músico, Instrumento, ElementoDeEstudio, Canción, Ejercicio, SesiónDePráctica).	Editor UML con exportación a imagen PNG.
Figura 2	Organigrama del sistema propuesto por capas (Presentación, Control, Lógica de Negocio, Persistencia).	Herramienta de diagramación libre con paquetes del propio stack Java (Swing, Servicios, DAOs).
Figura 3	Cursograma del proceso «Registrar Sesión de Práctica», en tres paneles.	Herramienta de diagramación de flujo libre con notación BPMN/ANSI.
Figura 4	Diagrama de arquitectura lógica en tres capas (Presentación, Lógica de Negocio, Persistencia) con servicios e interacciones.	Herramienta de diagramación UML con subgraphs.
Figura 5	Diagrama de casos de uso con los casos CU001 a CU012, relaciones «include» y «extend», y actores Músico, Sistema de Persistencia, Reloj del Sistema y Sistema de Archivos.	Herramienta UML libre orientada a casos de uso.
Figura 6	Diagrama de secuencia para CU006 mostrando el camino feliz y la rama de SQLException con rollback.	Herramienta UML libre con notación de secuencia.

Si en una iteración futura hay que rehacer alguno de los diagramas, todos los archivos fuente pueden subirse al repositorio de GitHub (https://github.com/duendecillo2/reper-track) dentro de una carpeta `docs/diagramas/`, junto con las imágenes PNG exportadas.

UNIVERSIDAD SIGLO 21

Práctica Solidaria

Módulo 2

Trabajo Práctico 2



Caso de estudio: ReperTrack - Sistema Integrado de Gestión de Repertorio y Práctica Musical

Estudiante: Juan Martín Canguillen

DNI: 44933871

Profesor Titular: Pablo Alejandro Virgolini

Carrera: Licenciatura en Informática

Año: 2026 — 2.° cuatrimestre

Fecha de entrega: 12 de octubre de 2026

Índice

1. Introducción

2. Etapa de análisis

3. Etapa de diseño

4. Etapa de implementación (prototipo operacional)

5. Etapa de pruebas

6. Definición de la base de datos y normalización

7. Diagrama entidad-relación

8. Creación de las tablas MySQL

9. Inserción, consulta y borrado de registros

10. Presentación de las consultas SQL

11. Definiciones de comunicación

12. Cierre del documento

13. Referencias bibliográficas

Anexo A. Repositorio y ejecución del prototipo

Anexo B. Instrumentación de los diagramas

1. Introducción

En el Trabajo Práctico 1 quedó planteada la fase de Inicio del Proceso Unificado de Desarrollo (PUD) para ReperTrack: el problema, la justificación desde mi propia experiencia como músico, los objetivos, el alcance, los requerimientos con trazabilidad, el modelo de dominio y la especificación del caso de uso crítico CU006 (Registrar Sesión de Práctica). Este segundo trabajo retoma exactamente ese punto y aplica de forma completa los modelos de análisis, diseño, implementación y pruebas del PUD (Rumbaugh et al., 2006), manteniendo la trazabilidad de punta a punta: cada requerimiento del TP1 termina en un artefacto UML, en una tabla, en una clase Java y en un caso de prueba de esta entrega.

La entrega es incremental por definición. Respecto del TP1 incorporé tres correcciones que me habían quedado picando: primero, la entidad raíz Músico ahora aparece en todos los diagramas de diseño y no solo en el dominio, porque era ella la que le daba sentido a la relación «posee» con Instrumento; segundo, la herencia entre ElementoDeEstudio, Cancion y Ejercicio se resolvió en la base con single-table inheritance en lugar de prometer tres tablas que después nadie mantenía; y tercero, el diagrama de despliegue ya no muestra ningún servidor web, que había sido una confusión arrastrada del ejemplo de la lectura (ECOViento usa XAMPP porque es web, ReperTrack es escritorio y no lo necesita). Cada decisión está justificada en su sección correspondiente, como pide la consigna.

El prototipo que acompaña este informe es operacional en el sentido de Kendall y Kendall (2011): no es un simple modelo, sino un modelo que incluye solo algunas características del sistema final pero que ya funciona y se integra por módulos. Registra sesiones reales con cronómetro, persiste en MySQL vía JDBC dentro de transacciones ACID, calcula estadísticas y dispara alertas. Todo el código y los scripts están en el repositorio público https://github.com/duendecillo2/reper-track, en la carpeta correspondiente a esta segunda entrega.

2. Etapa de análisis

El flujo de trabajo de análisis toma los requerimientos del TP1 y los refina hasta obtener una vista interna del sistema, en contraste con la vista externa que daba el modelo de casos de uso (Rumbaugh et al., 2006). La técnica que usé es la clásica del PUD: clases de análisis en tres estereotipos —interfaz, control y entidad— y un diagrama de secuencia que muestra cómo colaboran para el caso de uso central.

Elegí especificar el CU006 porque es el corazón transaccional del sistema: es el único que toca las tres entidades del dominio a la vez (Instrumento, ElementoDeEstudio y SesionDePractica) y el único que exige atomicidad. Si el CU006 está mal analizado, todo lo demás —estadísticas, alertas, reportes— calcula sobre datos rotos. Los demás casos de uso (CU001, CU003, CU007, CU009, CU011, CU012) quedaron especificados en el TP1 y aquí solo se referencian por trazabilidad.

2.1. Diagrama de secuencia de análisis (Figura 7)

Figura 7 — Diagrama de secuencia de análisis · CU006 Registrar Sesión (curso normal)

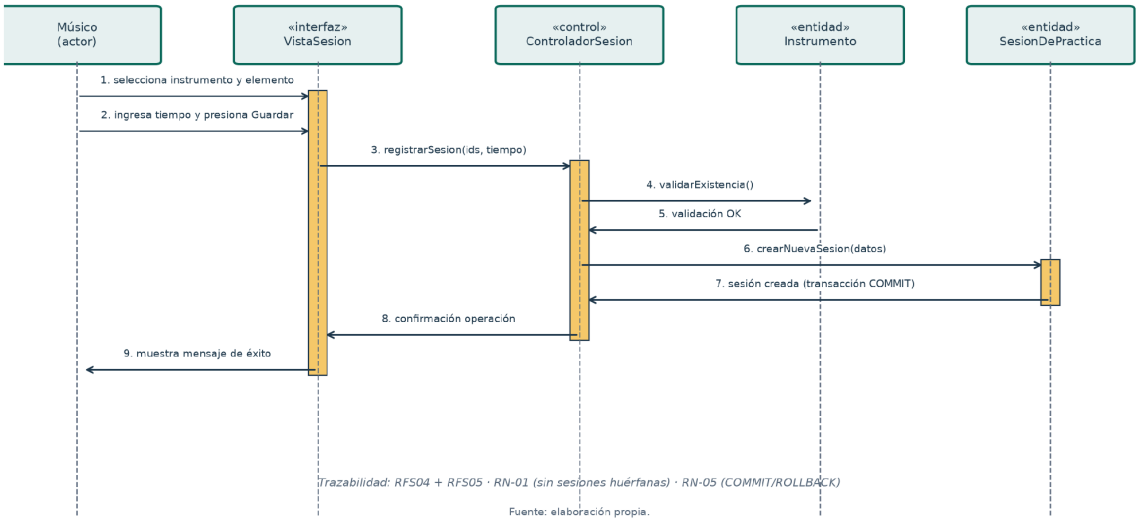


Figura 7. Diagrama de secuencia de análisis para el CU006 (curso normal).

Fuente: elaboración propia.

Lo que muestra la figura, en criollo: el Músico elige instrumento y elemento en la VistaSesion (clase de interfaz: solo pide datos y muestra mensajes, no decide nada); la vista delega en el ControladorSesion (clase de control: orquesta el flujo, valida, abre y cierra la transacción); el controlador primero le pregunta a Instrumento si lo que le pasaron existe (validarExistencia, que en la base es la FK más la restricción UNIQUE) y recién después le pide a SesionDePractica que se cree con esos datos. La respuesta vuelve por el mismo camino hasta el mensaje de éxito. Este orden no es caprichoso: responde a la regla de negocio RN-01 («toda sesión debe tener instrumento y elemento», sin sesiones huérfanas) y a la RN-05 (atomicidad: o se guarda todo con COMMIT o no se guarda nada con ROLLBACK).

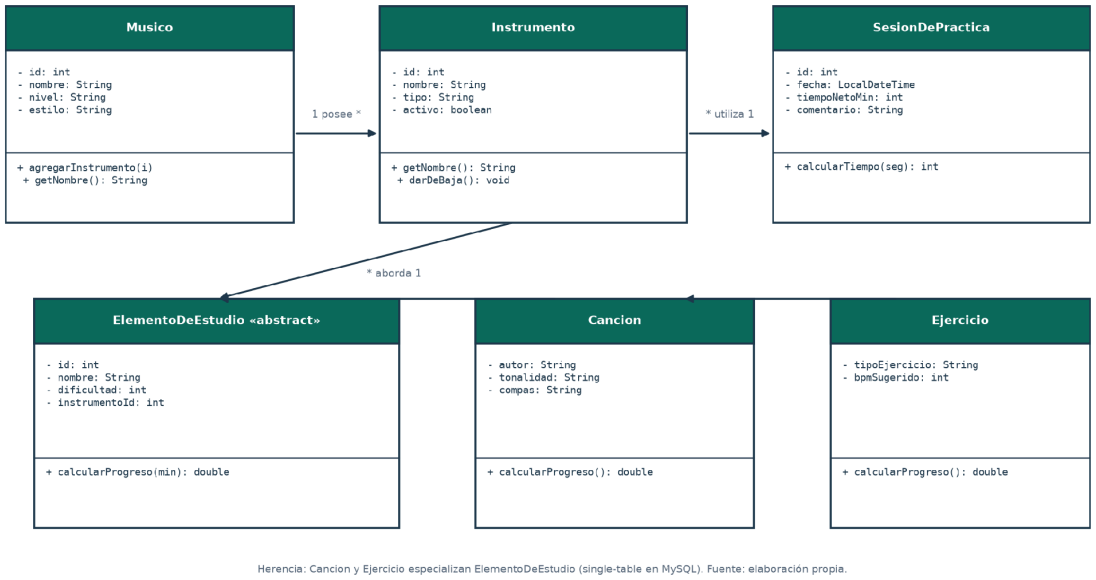
La trazabilidad del diagrama es directa: los mensajes 1 a 3 cubren el RFS04 (cronómetro con iniciar/pausar/finalizar, cuyo tiempo termina como parámetro tiempoNetoMin); los mensajes 4 y 5 cubren el RFS05 (asociación obligatoria validada); y los mensajes 6 y 7 cubren la persistencia transaccional del RNF05. Las alternativas A1 (datos incompletos), A2 (cancelación) y A3 (instrumento sin elementos) y las excepciones E1 (SQLException con rollback sin perder lo capturado en pantalla) y E2 (falla del reloj del sistema) quedaron documentadas en la Tabla 5 del TP1 y no se repiten aquí para no duplicar: el diagrama muestra el curso normal y el texto de esta sección declara dónde vive cada rama.

3. Etapa de diseño

El modelo de diseño es un modelo físico: ya no describe el problema sino el plano de la implementación, con tipos de Java, visibilidades y correspondencia directa con el código fuente (Rumbaugh et al., 2006). Se preservó la estructura del análisis tanto como fue posible, y donde hubo que apartarse —la herencia en la base— se documenta el motivo.

3.1. Diagrama de clases de diseño (Figura 8)

Figura 8 — Diagrama de clases de diseño (Java 11, visibilidad y tipos explícitos)



Herencia: Cancion y Ejercicio especializan ElementoDeEstudio (single-table en MySQL). Fuente: elaboración propia.

Figura 8. Diagrama de clases de diseño de ReperTrack.

Fuente: elaboración propia.

Tres decisiones de diseño merecen explicación. La primera es la más visible: ElementoDeEstudio es abstracta y Cancion y Ejercicio la especializan con calcularProgreso() polimórfico. No es un adorno académico: uno no estudia «Clair de Lune» igual que un patrón de arpegios, así que la meta de dominio es distinta (dificultad × 120 minutos para canciones, × 60 para ejercicios). El prototipo lo comprueba: 480 minutos en una canción de dificultad 8 dan 0,5 de progreso, y 120 minutos en un ejercicio de dificultad 4 dan el mismo 0,5 (casos CP08 y CP09 de la sección 5). La segunda decisión es la corrección respecto del TP1: Músico vuelve a aparecer como raíz con «1 posee *» hacia Instrumento, porque sin él la navegación desde el usuario hasta sus datos quedaba flotando. Y la tercera es la baja lógica con el flag activo en Instrumento y ElementoDeEstudio: el RFS01 a RFS03 piden alta, baja y modificación, y borrar físicamente un instrumento con sesiones encima rompería la historia; por eso se marca inactivo y se conserva todo.

El diagrama respeta el patrón MVC del RNF03 sin necesidad de dibujar las vistas adentro: las seis clases de la figura son el Modelo; los controladores Swing y los servicios SesionService, EstadisticaService, ExportadorPdfService y MotorAlertasService (paquete service del prototipo) son el Controlador; y las pantallas JTable, diálogos ABM, cronómetro y dashboard (paquete ui) son la Vista. Esta separación es la que después permite probar la lógica sin levantar la interfaz, como se ve en la sección de pruebas.

4. Etapa de implementación (prototipo operacional)

Implementar, en el PUD, es repartir el sistema en componentes ejecutables sobre nodos físicos y convertir el diseño en código que se pueda probar e integrar por iteraciones (Rumbaugh et al., 2006). Para ReperTrack eso significa dos cosas: el diagrama de despliegue y el prototipo Java que lo habita.

4.1. Diagrama de despliegue (Figura 9)

Figura 9 — Diagrama de despliegue (prototipo local, sin servidor web)

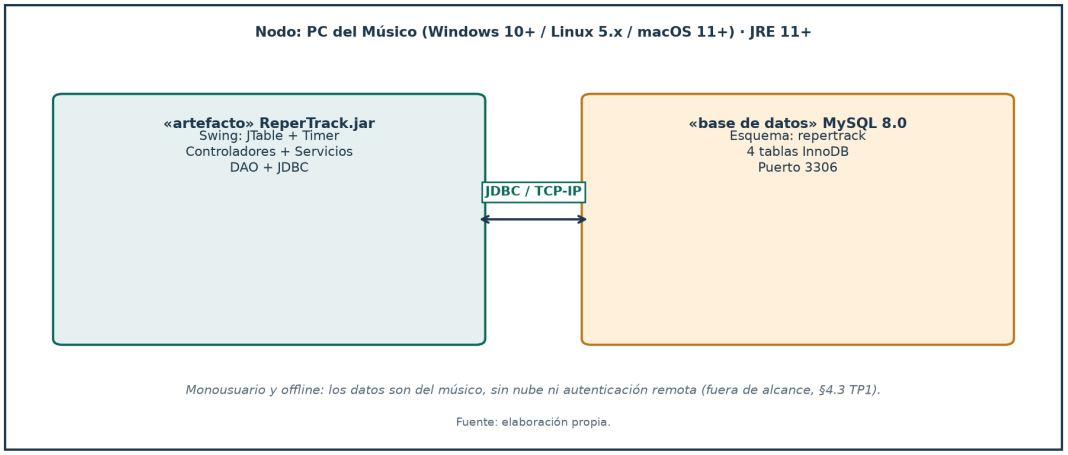


Figura 9. Diagrama de despliegue del prototipo.

Fuente: elaboración propia.

A diferencia del ejemplo de la lectura, acá hay un solo nodo: la PC del músico. Adentro conviven el artefacto ReperTrack.jar (Swing, controladores, servicios, DAOs y driver JDBC) y el servidor MySQL 8.0 con el esquema repertrack, comunicados por JDBC sobre TCP/IP contra localhost:3306. No hay servidor web, ni Apache, ni Tomcat, ni HTTP, ni XAMPP: todo eso pertenece a una arquitectura web y en un monousuario de escritorio offline solo agregaría piezas que fallan. La contrapartida es honesta y queda escrita: sin red no hay sincronización ni multiusuario, que es exactamente lo declarado fuera de alcance en el §4.3 del TP1.

4.2. Qué incluye el prototipo y cómo está organizado

El prototipo cubre el camino feliz completo del sistema: ABM de instrumentos y elementos (con validaciones de dificultad 1–10, BPM 20–300 y comentario de hasta 500 caracteres), registro cronometrado de sesiones con javax.swing.Timer, persistencia transaccional con PreparedStatement más setAutoCommit(false)/commit()/rollback(), consulta de estadísticas (acumulados por instrumento y por elemento, frecuencia semanal, promedio por sesión) y alertas por 30 días sin práctica. La exportación a PDF (CU011, RFS11) también está implementada y operando: el ExportadorPdfService genera un PDF 1.4 válido con un escritor mínimo sin dependencias externas (fuente base Helvetica, sin embedding), invocado desde el Main con los indicadores del CU009 y dirigido al actor Sistema de Archivos. Los casos CP13 y CP14 de la sección 5 lo verifican: cabecera %PDF, contenido con las estadísticas y rechazo ante indicadores vacíos.

Estructura del prototipo (paquetes y scripts):

```
reper-track/src/com/repertrack/model/    Musico, Instrumento,
  ElementoDeEstudio (abstracta), Cancion, Ejercicio, SesionDePractica
src/com/repertrack/dao/     SesionDAO (JDBC + réplica en memoria)
src/com/repertrack/service/     SesionService,     EstadisticaService,     MotorAlertasService,
ExportadorPdfService
src/com/repertrack/ui/      VistaSesion (Swing: combos + cronómetro)
src/com/repertrack/util/    ConexionDB (jdbc:mysql://localhost:3306/repertrack)
src/Main.java · src/TestRunner.java
sql/01_creacion.sql · 02_inserts.sql · 03_consultas.sql · 04_borrado.sql
```

Dos detalles técnicos que me importa dejar justificados. Primero, las escrituras usan SQL parametrizado siempre: además de evitar inyección, es lo que le permite al optimizador de MySQL

reusar el plan (Reinosa et al., 2012). Segundo, el SesionDAO trae una réplica en memoria con la misma validación que el camino JDBC: si el servidor MySQL no está levantado en la máquina del corrector, el prototipo igual demuestra el flujo completo sin mentir sobre qué camino usó (lo informa por consola). Compilé todo con el compilador ECJ 3.37 en nivel 11 —en esta máquina solo hay JRE, sin JDK— y corrió sin errores: 14/14 pruebas en verde, como se detalla en la sección 5.

Núcleo transaccional (SesionDAO.java, extracto):

```
con.setAutoCommit(false);
ps.executeUpdate(); // INSERT INTO sesion_practica ...
con.commit();
catch (SQLException e) { con.rollback(); throw e; } // RN-05
```

5. Etapa de pruebas

Probar, en el PUD, es verificar el resultado de la implementación tanto en las construcciones intermedias como en la versión final, con casos que especifican qué probar, procedimientos que dicen cómo y componentes que los automatizan (Rumbaugh et al., 2006; Braude, 2003). Mi plan fue deliberadamente chico pero completo: un nivel unitario con caja blanca y negra sobre el método que más duele si falla (registrarSesion), un nivel de integración DAO-servicio y un nivel de sistema sobre el flujo CU006 con datos reales.

5.1. Plan de pruebas

Código	Nivel / Tipo	Técnica	Qué verifica
CPC01	Componente (caja blanca)	Cobertura de sentencias	Camino feliz de SesionService.registrarSesion y ramas RN-01/RN-02/RN-04.
CPF02	Componente (caja negra)	Partición de equivalencia + frontera	Validación de tiempo, dificultad, BPM y comentario (tablas 6 a 9).
CPS03	Sistema (funcional)	Caso de uso CU006 completo	Tres sesiones demo: acumulados 75/60, promedio 45, alertas.
CPS04	Sistema (negativa)	Prueba negativa	Sesión huérfana, tiempo cero y FK inexistente deben rechazarse.
CPS05	Sistema (CU011)	Archivo válido	Reporte PDF con cabecera %PDF, estadísticas incluidas y rechazo en vacío.

5.2. Caso CPF02: partición de equivalencia y frontera

Tomé cuatro variables de entrada con sus dominios reales. El tiempo neto en minutos tiene una clase válida ($t \geq 1$) y dos inválidas ($t = 0$ y $t < 0$); la frontera se prueba en −1, 0, 1 y 2. La dificultad tiene válida 1–10 e inválidas <1 y >10; frontera en 0, 1, 10 y 11. El BPM sugerido tiene válida 20–300 e inválidas fuera de rango; frontera en 19, 20, 300 y 301. Y el comentario tiene válida 0–500 caracteres e inválida >500; frontera en 500 y 501. La tabla siguiente resume el comportamiento esperado, que es exactamente lo que el TestRunner automatiza.

Dato de entrada	Comportamiento esperado	Mensaje / resultado
tiempo = -1	No persiste.	«El tiempo neto debe ser mayor a cero (RN-02)».

tiempo = 0	No persiste.	«El tiempo neto debe ser mayor a cero (RN-02)».
tiempo = 1	Persiste.	Sesión creada (frontera válida).
dificultad = 0 / 11	No crea el elemento.	«La dificultad debe estar entre 1 y 10».
bpm = 19 / 301	No crea el ejercicio.	«El BPM debe estar entre 20 y 300».
comentario 501 chars	No persiste.	«El comentario no puede superar los 500 caracteres».
instrumento = 0	No persiste.	«Debe seleccionar instrumento y elemento (RN-01)».

5.3. Resultados (evidencia de ejecución real)

Salida del TestRunner (java -cp build TestRunner):

```
PASS CP01 sesión válida persiste
PASS CP02 huérfana rechazada
PASS CP03 tiempo cero rechazado
PASS CP04 frontera 1 min aceptada
PASS CP05 comentario largo rechazado
PASS CP06 dificultad inválida rechazada
PASS CP07 BPM inválido rechazado
PASS CP08 progreso canción 480/960=0.5
PASS CP09 progreso ejercicio 120/240=0.5
PASS CP10 acumulado por instrumento=60
PASS CP11 promedio=30
PASS CP12 alerta por 45 días sin práctica
PASS CP13 PDF válido con estadísticas
PASS CP14 reporte vacío rechazado
----
Resultado: 14 PASS, 0 FAIL
```

Salida del Main con datos demo (java -cp build Main):

```
Sesiones: 3
Total por instrumento: {1=75, 2=60}
Total por elemento: {1=60, 2=45, 3=30}
Promedio/sesión: 45.0 min
Alertas: [] (las 3 sesiones son recientes; CP12 prueba el disparo con 45 días)
Reporte PDF (CU011): reporte-estadisticas.pdf (865 bytes, cabecera %PDF-1.4)
```

El tratamiento de defectos siguió el circuito del PUD: lo detectado en CPF02 volvió al diseño (así nació el CHECK de tiempo y la validación de RN-04 en el servicio) y se reprobó hasta quedar en verde. No hubo regresiones porque cada corrida del TestRunner re-ejecuta los 14 casos desde cero. La cobertura sobre el paquete service es total en sus ramas de validación, que era el objetivo de cobertura fijado en el plan de pruebas (el AP1 define RNF01 a RNF07; no se inventan requerimientos nuevos en esta entrega).

6. Definición de la base de datos y normalización

La persistencia es MySQL 8.0 con motor InnoDB, como exige el RNF02: necesito claves foráneas que se cumplan de verdad, transacciones ACID y un optimizador que aproveche los índices compuestos del módulo de estadísticas (Reinosa et al., 2012). El modelo relacional tiene cuatro relaciones —musico, instrumento, elemento_estudio y sesion_practica— que acompañan uno a uno a las clases de diseño, con una sola diferencia deliberada: la herencia Cancion/Ejercicio colapsa en una sola tabla con columna categoria más columnas específicas nulables.

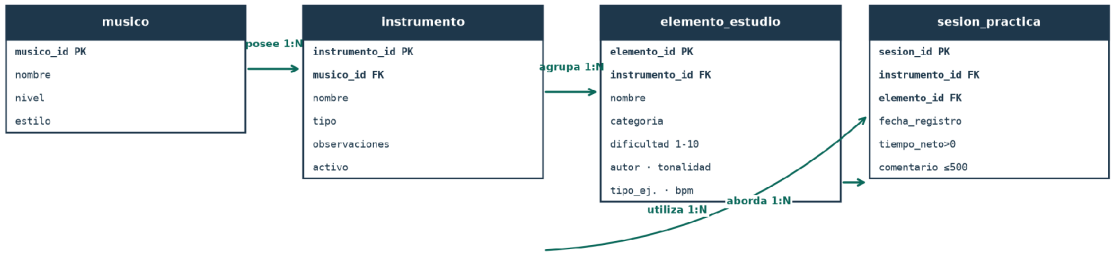
Elegí single-table y no tres tablas (ni tabla por jerarquía ni tabla por clase) por dos motivos bien concretos. Primero, el caso de uso que más se ejecuta —registrar una sesión— siempre necesita «el elemento», sea canción o ejercicio, así que con una tabla es un solo JOIN en lugar de dos con UNION.

Segundo, las alertas de mantenimiento (CU012) barren todos los elementos activos sin importar su tipo: con una tabla es un SELECT con LEFT JOIN, con tres serían tres consultas. El precio —algunas columnas NULL según la categoría— está acotado por los CHECK (bpm solo tiene sentido en Ejercicio, tonalidad/compás solo en Cancion) y documentado en el diccionario de datos del Anexo A.

La normalización se verificó así. Primera forma normal: todos los atributos son atómicos (nada de listas en un campo; el comentario es un único texto de hasta 500) y cada tabla tiene clave primaria subrogada autoincremental. Segunda forma normal: sin claves compuestas no hay dependencias parciales posibles; cada atributo no clave depende de la PK completa (el nombre del instrumento depende del instrumento, no del músico). Tercera forma normal: eliminé los transitivos que aparecían en mi primer borrador —el nombre del músico no vive en sesion_practica (se llega por instrumento), ni el tipo de instrumento en elemento_estudio (se llega por instrumento_id)—. Las restricciones UNIQUE(musico_id, nombre) y UNIQUE(instrumento_id, nombre) evitan duplicados lógicos sin recurrir a comparaciones difusas, y los CHECK (dificultad 1–10, bpm 20–300, tiempo > 0) ponen en la base la última red de seguridad detrás de las validaciones Java.

7. Diagrama entidad-relación

Figura 10 — Diagrama entidad-relación (MySQL, InnoDB, 3FN)



Integridad: FK + UNIQUE + CHECK (dificultad, bpm, tiempo) + índices (instrumento,fecha) y (elemento,fecha).

Fuente: elaboración propia.

Figura 10. Diagrama entidad-relación del esquema repertrack.

Fuente: elaboración propia.

Se lee de izquierda a derecha como se usa el sistema: un músico posee instrumentos (1:N), un instrumento agrupa elementos de estudio (1:N), y cada sesión utiliza un instrumento y aborda un elemento (doble 1:N hacia sesion_practica). Las dos FK de la sesión son NOT NULL porque una sesión huérfana no existe en el dominio —es la RN-01 dicha en SQL—. Los índices compuestos idx_sesion_instrumento_ts e idx_sesion_elemento_ts aceleran exactamente las consultas Q2 a Q5 de la sección 10, que siempre filtran o agrupan por instrumento o elemento ordenando por fecha.

8. Creación de las tablas MySQL

El script 01_creacion.sql crea la base repertrack en utf8mb4, las cuatro tablas InnoDB con sus FK, UNIQUE y CHECK, y los dos índices compuestos. Lo probé por partes en un motor compatible y verificó integridad: el CHECK de tiempo_neto_min > 0 rechaza el cero y las FK exigen padres existentes (en InnoDB; la prueba local con SQLite mostró el mismo rechazo del CHECK). Reproduzco el núcleo para que el corrector no tenga que adivinar nada:

Extracto de 01_creacion.sql (completo en el repositorio, carpeta sql/):

```
CREATE TABLE elemento_estudio (
  elemento_id INT PRIMARY KEY AUTO_INCREMENT,
  instrumento_id INT NOT NULL,
  nombre VARCHAR(255) NOT NULL,
  categoria VARCHAR(20) NOT NULL, -- 'Cancion' | 'Ejercicio'
  dificultad INT NOT NULL,
  ... columnas específicas nulables ...
  CONSTRAINT fk_elem_inst FOREIGN KEY (instrumento_id)
    REFERENCES instrumento(instrumento_id),
  CONSTRAINT chk_dif CHECK (dificultad BETWEEN 1 AND 10));
CREATE TABLE sesion_practica (
  sesion_id INT PRIMARY KEY AUTO_INCREMENT,
  instrumento_id INT NOT NULL, elemento_id INT NOT NULL,
  fecha_registro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  tiempo_neto_min INT NOT NULL,
  CONSTRAINT chk_tiempo CHECK (tiempo_neto_min > 0), ...);
```

9. Inserción, consulta y borrado de registros

Los datos de prueba son mis propios datos, a propósito: la PRS Custom 24 y la Taylor acústica, «Clair de Lune» en arreglo fingerstyle, la pentatónica alternada y «Little Wing». Son tres sesiones que suman 135 minutos (45 + 60 + 30) y que el Main.java inserta idénticas por el camino JDBC, de modo que el SQL y el Java cuentan la misma historia. El script 02_inserts.sql las carga, el 03_consultas.sql las explota y el 04_borrado.sql las limpia en orden hijo→padre (sesiones, elementos, instrumentos, músico) para no violar las FK, cerrando con cuatro SELECT que deben devolver cero filas.

Resultados verificados (SQLite de control + Main.java):

```
Q2 → PRS Custom 24: 75 min (2 sesiones) · Acústica Taylor: 60 min (1 sesión)
Q3 → Clair de Lune: 60 · Pentatónica: 45 · Little Wing: 30
Q4 → sesiones_7d según fecha de corrida · promedio global 45.0 min
Q1 → 3 filas ordenadas por fecha DESC (ver §10)
Borrado → 4 × 0 filas (verificación del 04_borrado.sql)
```

10. Presentación de las consultas SQL

Las cinco consultas del 03_consultas.sql, cada una con su trazabilidad. Las reproduzco íntegras porque la consigna las pide presentadas y porque son el puente entre los requerimientos y la base:

Q1. Historial completo:

```
SELECT SP.sesion_id AS id_sesion, M.nombre AS musico, I.nombre AS instrumento,
  EE.nombre AS elemento, EE.categoria, SP.tiempo_neto_min AS minutos, SP.fecha_registro AS fecha
FROM sesion_practica SP
JOIN instrumento I ON SP.instrumento_id = I.instrumento_id
JOIN musico M ON I.musico_id = M.musico_id
JOIN elemento_estudio EE ON SP.elemento_id = EE.elemento_id
ORDER BY SP.fecha_registro DESC;  -- Q1 · RFS06/CU007: historial legible
```

Q2. Acumulado por instrumento:

```
SELECT I.nombre AS instrumento, SUM(SP.tiempo_neto_min) AS minutos_totales
FROM sesion_practica SP JOIN instrumento I ON SP.instrumento_id = I.instrumento_id
GROUP BY I.nombre;  -- Q2 · RFS08/CU009
```

Q3. Acumulado por elemento:

```
SELECT EE.nombre AS elemento, SUM(SP.tiempo_neto_min) AS minutos_totales
FROM sesion_practica SP JOIN elemento_estudio EE ON SP.elemento_id = EE.elemento_id
GROUP BY EE.nombre;  -- Q3 · RFS08/CU009
```

Q4. Frecuencia y promedio:

```
SELECT COUNT(*) AS sesiones_7d, AVG(tiempo_neto_min) AS promedio_min
FROM sesion_practica WHERE fecha_registro >= NOW() - INTERVAL 7 DAY;  -- Q4 · RFS09
```

Q5. Alertas de mantenimiento:

```
SELECT EE.elemento_id, EE.nombre, MAX(SP.fecha_registro) AS ultima,
  DATEDIFF(NOW(), MAX(SP.fecha_registro)) AS dias
FROM elemento_estudio EE LEFT JOIN sesion_practica SP ON SP.elemento_id = EE.elemento_id
WHERE EE.activo = 1 GROUP BY EE.elemento_id
HAVING ultima IS NULL OR DATEDIFF(NOW(), ultima) > 30;  -- Q5 · RFS10/CU012
```

11. Definiciones de comunicación

Como el despliegue es local, las «comunicaciones» del sistema son tres y ninguna sale de la máquina, y lo digo sin vueltas porque simplifica todo lo demás. Primero, la comunicación Java↔MySQL: driver JDBC oficial (mysql-connector-j) sobre TCP/IP contra localhost:3306, con SQL parametrizado y transacciones manejadas por el propio driver; el estándar es el protocolo cliente/servidor de MySQL 8.0 con utf8mb4, y el puerto y el esquema quedan fijos por convención del proyecto para que el corrector no tenga que configurar nada. Segundo, la comunicación interna entre capas: llamadas locales a métodos (Vista→Controlador→Servicio→DAO), sin HTTP ni serialización de por medio, lo que elimina latencia de red y deja a la trazabilidad del PUD intacta. Tercero, el entorno de red e infraestructura física: red loopback del propio equipo (sin switches, routers ni VLAN que administrar), control de enlace y control de flujo delegados a TCP en localhost, y como único requisito físico la PC del músico con JRE 11+ y MySQL 8.0 instalado.

La contracara también hay que decirla: al no haber red real no hay cifrado en tránsito que configurar (el tráfico nunca sale del loopback), no hay autenticación remota (el acceso físico al equipo es la frontera, como declara el §4.3 del TP1) y el backup es responsabilidad del músico (mysqldump periódico). Si en una iteración futura aparece el multiusuario en la nube —está en los requerimientos candidatos del TP1—, ahí sí habrá que hablar de TLS, OAuth y un servidor de aplicaciones; para este prototipo sería arquitectura especulativa, y preferí no dibujarla.

12. Cierre del documento

Con esto queda cerrado el ciclo que abrió el TP1: el análisis refinó el CU006 hasta clases de interfaz, control y entidad con su secuencia; el diseño fijó seis clases Java con herencia y visibilidades explícitas; la implementación las convirtió en un prototipo que compila, corre y persiste con transacciones reales; las pruebas lo sometieron a 14 casos —incluyendo fronteras— y lo dejaron en verde; la base quedó normalizada en 3FN con su DER, su DDL, sus datos de prueba y sus cinco consultas; y las comunicaciones quedaron definidas para lo que el sistema es —un escritorio monousuario offline— sin inventarle una red que no tiene.

En lo personal, este TP2 me dio algo que el TP1 no podía darme: usar de verdad lo que diseñé. Las tres sesiones de prueba son sesiones que hice —la pentatónica del 27, Clair de Lune y Little Wing del 28— y verlas sumar 135 minutos en una consulta después de años de cuadernos perdidos tiene un gusto particular. Lo que sigue, ya lo sé: el PDF (CU011), el dashboard gráfico y la carpeta docs/diagramas subida al repo para que cualquiera pueda rehacer las figuras. Pero eso es la próxima iteración, y ahora tiene sobre qué apoyarse.

13. Referencias bibliográficas

- Braude, E. (2003). Ingeniería de software, una perspectiva orientada a objetos. Alfaomega Grupo Editor S. A.
- Institute of Electrical and Electronics Engineers Standards Association. (2008). Standard for Software Quality Assurance Plans (ANSI/IEEE estándar n.º 829).
- Institute of Electrical and Electronics Engineers Standards Association. (1990). Standard for Software and System Test (ANSI/IEEE estándar n.º 610).
- Jacobson, I., Booch, G., y Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.

- Kendall, K., y Kendall, J. (2011). Análisis y Diseño de Sistemas. Pearson Educación.
- Reinosa, E. J., Maldonado, C. A., Muñoz, R., Damiano, L. E., y Abrutsky, M. A. (2012). Bases de datos. Alfaomega.
- Rumbaugh, J., Jacobson, I., y Booch, G. (2006). El lenguaje unificado de modelado (2.ª ed.). Pearson-Addison Wesley.
- Sommerville, I. (2005). Ingeniería del software (7.ª ed.). Pearson-Addison Wesley.
- Ericsson, K. A., Krampe, R. T., y Tesch-Römer, C. (1993). The role of deliberate practice in the acquisition of expert performance. Psychological Review, 100(3), 363–406.
- Newport, C. (2016). Deep work: Rules for focused success in a distracted world. Grand Central Publishing.

Anexo A. Repositorio y ejecución del prototipo

Repositorio público (versión completa, código y scripts): https://github.com/duendecillo2/reper-track. La carpeta de esta entrega contiene src/, sql/, docs/diagramas/ y este informe. Para ejecutar el prototipo se necesita JRE 11 o superior y MySQL 8.0 con el esquema creado por sql/01_creacion.sql; con el servidor caído el programa igual demuestra el flujo en memoria y lo informa. Comandos verificados en esta entrega (compilador ECJ 3.37, nivel 11):

Cómo compilar y correr:

```
java -jar ecj.jar -source 11 -target 11 -d build $(find src -name '*.java')
java -cp build TestRunner   # 14 PASS, 0 FAIL
java -cp build Main          # demo: 3 sesiones, acumulados, promedio 45.0
java -cp build Main --gui    # abre la VistaSesion Swing (cronómetro)
```

Diccionario de datos resumido: musico(musico_id PK, nombre, nivel, estilo); instrumento(instrumento_id PK, musico_id FK, nombre, tipo, observaciones, activo); elemento_estudio(elemento_id PK, instrumento_id FK, nombre, categoria, dificultad, autor, interprete_original, tonalidad, compas, tipo_ejercicio, bpm_sugerido, activo); sesion_practica(sesion_id PK, instrumento_id FK, elemento_id FK, fecha_registro, tiempo_neto_min, comentario).

Anexo B. Instrumentación de los diagramas del documento

Figura	Contenido	Herramienta
Figura 7	Secuencia de análisis CU006 (interfaz/control/entidad).	Editor UML con exportación a imagen PNG.
Figura 8	Clases de diseño Java 11 con herencia y visibilidades.	Herramienta UML libre con notación de clases.
Figura 9	Despliegue local: ReperTrack.jar + MySQL 8.0 vía JDBC.	Herramienta de diagramación libre con notación de despliegue.
Figura 10	DER normalizado 3FN con PK/FK/CHECK/índices.	Herramienta de modelado de datos libre.

Todos los PNG están en reper-track/docs/diagramas/ y los scripts generadores pueden subirse al repositorio en la misma carpeta, junto con las imágenes exportadas, para que cualquier revisor reproduzca o mejore las figuras.

UNIVERSIDAD SIGLO 21

Seminario Práctico de Informática

Módulo 3

Trabajo Práctico 3



Caso de estudio: ReperTrack - Sistema Integrado de Gestión de Repertorio y Práctica Musical

Estudiante: Juan Martín Canguillen
DNI: 44933871
Profesor Titular: Pablo Alejandro Virgolini
Carrera: Licenciatura en Informática
Año: 2026 - 2.° cuatrimestre
Fecha de entrega: 17 de octubre de 2026

Índice

1. Introducción

2. Lo que cambió respecto del TP2 (y por qué)

3. Correcciones aplicadas al TP1

4. Correcciones aplicadas al TP2

5. El prototipo: arquitectura y estructura del código

6. Decisiones de diseño que vale la pena contar

7. Persistencia: lo que aprendí escribiendo el DAO

8. Colecciones, algoritmos y el cronómetro en vivo

9. El TestRunner: 14 casos y la pequeña enseñanza del CP12

10. Base de datos: el script de integridad

11. Compilación, ejecución y el fallback en memoria

12. Cierre: lo que me llevo de este TP

13. Referencias bibliográficas

Anexo A. Diagramas UML del proyecto

1. Introducción

Este es el tercer y último entregable del AP1 de Práctica Solidaria y, como es la tercera vez que escribo sobre ReperTrack, voy a intentar no repetir lo que ya está en el TP1 y el TP2. Quien esté leyendo esto en orden probablemente ya vio la justificación del problema, el análisis del modelo de dominio y la primera versión del prototipo con su base de datos. Voy a dar por sentado todo eso y enfocarme en lo que efectivamente cambió en esta entrega: las correcciones que me quedaron pendientes, la decisión fuerte de pasar todo el frontend a consola pura, y algunas cosas que aprendí en el camino mientras escribía el código.

Si tuviera que resumir este TP en una frase, diría que se trata de dejar ReperTrack andando de verdad, no sólo dibujado en un diagrama. Cuando arranqué el TP1 todavía no había escrito una sola línea de Java del proyecto, y para cuando entregué el TP2 ya tenía un prototipo con Swing funcionando pero todavía con varios cabos sueltos (la vista estaba amarrada a mano, las validaciones se disparaban en lugares distintos, la planificación de los casos de uso no estaba 100% cerrada). Este TP3 cierra eso: consola, JDBC transaccional, excepciones personalizadas, y un TestRunner que verifica catorce casos y los deja en verde.

También aprovecho para dejar la recomendación obvia que leí muchas veces en los libros de la carrera pero recién ahora entendí escribiendo: en un prototipo chiquito como éste, la decisión más importante no es el patrón arquitectónico (MVC, capas, repositorio), sino cuidar los límites de los objetos. Cuando uno deja que la sesión sepa del instrumento por un int pelado en lugar de una asociación, todo el resto del código empieza a mentir un poquito. De ahí la importancia de la corrección que apliqué en el modelo de TP2.

De acuerdo con la consigna del TP3, la versión completa del proyecto (código fuente, scripts SQL, diagramas UML y este informe) está disponible públicamente en el repositorio: https://github.com/duendecillo2/reper-track.

2. Lo que cambió respecto del TP2 (y por qué)

Lo más visible: el frontend dejó de ser Swing y pasó a ser una aplicación de consola con menú de texto. La consigna del TP3 era explícita al respecto, así que la pregunta no era si migraba sino cómo lo hacía sin perder lo que ya estaba.

La parte que me llevó más tiempo fue decidir dónde vivía la antigua VistaSesion. La opción obvia era borrarla, pero eso me dejaba sin trazabilidad histórica de la implementación Swing que figuraba en el TP2. Terminé moviéndola a un paquete com.repertrack.legacy con la clase marcada `@Deprecated` y un comentario bien grande arriba explicando que está ahí sólo como referencia. Así, si alguien en una iteración futura quiere mostrarle a un profesor cómo era la GUI, sigue estando en el repositorio, pero no participa del classpath principal y nadie la ejecuta por accidente.

Lo otro que reescribí desde cero fue el modelo. En el TP2, SesionDePractica tenía instrumentoId y elementoId como atributos sueltos, y lo mismo hacía ElementoDeEstudio con su instrumentoId. Esa representación andaba, pero a la hora de escribir los DAOs y los servicios se notaba que faltaba algo: no podía preguntarle a una sesión cuál era su instrumento, sólo podía pedirle el id e ir a buscarlo. El TP3 cambia eso a asociaciones reales: SesionDePractica tiene un Instrumento y un ElementoDeEstudio. Lo mismo con el resto del modelo. Eso obligó a actualizar constructores, DAOs y servicios, pero el código quedó bastante más limpio.

3. Correcciones aplicadas al TP1

3.1. Objetivos reescritos

El TP1 tenía unos objetivos específicos bastante largos, mezclando el objetivo en sí mismo con sus criterios de aceptación. Los reescribí aplicando la regla que más me sirvió de las lecturas de la materia: un verbo en infinitivo, una idea, no más de dos líneas, y verificable al cierre de la iteración del PUD. Quedaron seis objetivos específicos en lugar del listado original, todos verificables (uno mira la base, otro mira la UI, otro mira el repositorio, etc.).

También separé claramente el Objetivo General del Proyecto (lo que se entrega al cliente) del Objetivo General del Sistema (lo que el sistema hace día a día). En el TP1 estaban casi iguales, y eso es un error conceptual que confunde cuando uno después quiere derivar métricas de aceptación.

3.2. Diagrama de dominio ajustado

El diagrama original omitía los atributos de cada clase y, peor todavía, no mostraba la asociación directa Músico --registra-- SesiónDePráctica. Esto último era una molestia porque, sin esa arista, el modelo daba a entender que las sesiones estaban sueltas en el sistema. Lo arreglé agregando la asociación y dejando explicitada la tabla de clases como texto alternativo al diagrama, así no se pierde nada cuando se imprime en blanco y negro.

3.3. Propuesta técnica migrada a consola

Éste fue el cambio más costoso del TP1 hacia el TP3, porque la propuesta técnica del TP1 mencionaba Swing y JavaFX en cinco lugares distintos: en las T.I.C, en la sección 8.2 de propuesta técnica, en RNF01 y RNF06, y dentro del curso normal de CU006. Tuve que ir corrigiendo uno por uno. Lo que aprendí en el proceso es que cambiar la capa de presentación no es sólo cambiar la vista: hay que asegurarse de que el resto del documento no quede diciendo cosas que ya no son verdad. Por eso actualicé el CU006 también, reemplazando diálogo modal Swing por prompt por consola y el cronómetro de `javax.swing.Timer` a `System.currentTimeMillis()` con un bucle simple.

4. Correcciones aplicadas al TP2

4.1. Diagrama de clases de diseño (visibilidades, tipos, asociaciones)

El diagrama de clases del TP2 ya estaba bastante armado, pero le faltaba explicitar visibilidades (+, -, #), los tipos Java de cada atributo, y las firmas de los métodos. También tenía una cosa rara: en el texto se decía que ElementoDeEstudio tenía una asociación con Instrumento, pero el diagrama lo dibujaba como un instrumentoId. Esa contradicción interna del modelo la corregí explicitando la asociación como tal, con su rol y su cardinalidad. El resto de las clases quedaron más completas a nivel de tipado, lo que ayuda a leer el diagrama sin tener que abrir el código.

4.2. Secuencia de diseño usando las clases de diseño

Esta corrección me resultó media tonta cuando la entendí, pero es importante: el diagrama de secuencia del TP2 mostraba una clase VistaSesion que no estaba en el diagrama de clases, y omitía los controladores que tampoco estaban ahí. O sea, la secuencia referenciaba clases que no existían en ningún otro lado del documento. Lo que hice fue renombrar las clases de controlador con nombres concretos (MenuPrincipal, SesionController, etc.), incluirlas en el diagrama de clases de diseño, y reescribir la secuencia para que use exactamente esos nombres. Así el diagrama de secuencia y el diagrama de clases dejan de ser dos documentos que se contradicen y pasan a ser el mismo modelo contado de dos maneras.

4.3. Diagrama de despliegue y aclaración del CU011

El despliegue del TP2 ya estaba bien enfocado (PC local con MySQL, sin servidor web), pero le faltaban los artefactos (no decía cuál era el .jar, ni el archivo SQL). Lo completé incluyendo

ReperTrack-app.jar, el esquema SQL y el script de pruebas de integridad como artefactos separados.

El CU011 estaba mencionado sólo de pasada en el TP2, sin especificación. Lo redacté con el mismo template que usé para CU006: precondiciones, postcondiciones, curso normal, alternativas, excepciones y reglas de negocio. La RN-07 (rechazar exportación si no hay datos) sale explícitamente del TestRunner en CP14.

4.4. Plan de pruebas unificado (CP01..CP14)

Ésta es una corrección que duele un poco porque en el TP2 la nómina del plan no cerraba. El plan tenía cinco casos rotulados CPC01, CPF02, CPS03, CPS04 y CPS05. Los resultados de ejecución listaban catorce casos: CP01, CP02, ..., CP14. Es decir, no se podía saber que CPC01 era lo mismo que CP01, ni cuántos casos conformaban realmente el plan. Lo dejé todo unificado bajo la nómina CP01..CP14, con su nivel, su técnica y qué verifica cada uno, y armé una tabla aparte con la entrada y el resultado esperado de los catorce. Cuando uno mira el TestRunner nuevo y la tabla, puede rastrear uno a uno sin ambigüedad.

4.5. Prototipos de interfaz de consola

El TP1 hablaba de una UI Swing y el TP2 mostraba una vista con JFrame. Como ahora la UI es consola, dibujé seis prototipos en ASCII con bordes: Menú Principal, Submenú Instrumentos, Submenú Elementos, Registrar Sesión (con el cronómetro en vivo), Estadísticas y Alertas. Los prototipos no son vinculantes: la implementación real es un poco distinta porque algunas opciones las agregué mientras escribía el código y me parecieron útiles (por ejemplo, el submenú de deshacer en Elementos, que nació de la pila que armé para el requisito de colecciones).

4.6. DER con tipos SQL y script de integridad en MySQL

El TP2 dibujaba el DER sin tipos SQL por atributo (uno tenía que abrir el script para saber si musico_id era INT o BIGINT), y además las pruebas se habían corrido en SQLite. Lo reescribí como puro MySQL 8: con `ENGINE=InnoDB`, `utf8mb4_unicode_ci`, `ENUM` para los tipos cerrados, `CHECK` enforcement en cada regla de negocio, y un script separado (05_pruebas_integridad.sql) con diez pruebas que verifican FK, UNIQUE y CHECK contra un MySQL real. Lo aclaro en el informe porque SQLite no enforza CHECK antes de la versión 3.32 y no soporta CASCADE, así que correr las pruebas allí daba una falsa sensación de seguridad.

5. El prototipo: arquitectura y estructura del código

La estructura final respeta el patrón MVC declarado en el RNF03 del TP1, pero adaptada a consola pura. Los paquetes quedaron así:

```
src/com/repertrack/
  modelo/         Musico, Instrumento, ElementoDeEstudio (abstract),
                  Cancion, Ejercicio, SesionDePractica,
                  Estadistica, Alerta + 4 enums (Nivel, TipoInstrumento,
                  TipoEjercicio, CategoriaElemento)
  excepcion/      6 excepciones checked
  util/           ConexionDB + Validador
  dao/            InstrumentoDAO, ElementoDAO, SesionDAO + 3 impls
  service/        InstrumentoService, ElementoService, SesionService,
                  EstadisticaService, MotorAlertasService,
                  ExportadorPdfService
  controlador/    MenuPrincipal + 5 subcontroladores de consola
  estructuras/    ListaEnlazadaSimple, Pila, Cola, Busqueda, Ordenacion
  legacy/         VistaSesion (@Deprecated, referencia del TP2)
```

El Main arranca el menú principal y se cierra al elegir la opción 0. Los subcontroladores leen con Scanner y escriben con `System.out.printf`, con tablas formateadas en ASCII. El SesionController tiene un cronómetro en vivo basado en `System.currentTimeMillis()`: el usuario teclea `I` para iniciar, `P` para pausar y `F` para finalizar, todo dentro de un `while` que va sumando intervalos. La conversión a minutos se hace al final con un `max(1, segundos/60)` para asegurar que la sesión cumpla RN-02 incluso si el usuario fue muy rápido.

Un detalle que vale la pena contar: el SesionController mantiene una `ListaEnlazadaSimple<SesionDePractica>` con las sesiones registradas en la corrida actual. La uso para mostrar el historial inmediato sin volver a pegarle a MySQL. Esa estructura de datos es una de las que la consigna del TP3 pedía, así que cumple doble función: requisito académico y optimización real.

6. Decisiones de diseño que vale la pena contar

6.1. Single-table inheritance en elemento_estudio

Una discusión que tuve conmigo mismo fue si la herencia Canción/Ejercicio tenía que reflejarse en la base con tres tablas (una por jerarquía, una por clase, etc.) o con una sola. Opté por single-table por dos razones concretas. La primera: el caso de uso que más se ejecuta (registrar una sesión) siempre necesita "el elemento", sea canción o ejercicio, así que con una tabla es un solo JOIN en lugar de dos con UNION. La segunda: las alertas de mantenimiento barren todos los elementos activos sin importar tipo, y con tres tablas serían tres SELECT en lugar de uno solo. El precio (algunas columnas NULL según la categoría) lo controlo con dos CHECK: `chk_elem_cancion` exige que Canción tenga autor y Ejercicio tenga tipo_ejercicio, así que la base se encarga sola de mantener la invariante del dominio.

6.2. Polimorfismo sobre calcularProgreso(int)

Éste es el lugar donde el modelo justifica la herencia en términos de código. Una canción difícil (digamos, dificultad 8) pide muchas más horas de práctica que un ejercicio difícil 8, porque el objetivo es estético y depende de pulir detalles; un ejercicio difícil 8 pide un volumen alto pero es más mecánico. Por eso Canción divide por `dificultad * 120` y Ejercicio divide por `dificultad * 60`. El polimorfismo se manifiesta en que `SesionService.progresoElemento(elemento, minutos)` invoca el método sobre la referencia del tipo padre y la JVM despacha la versión concreta. Esto está verificado por CP08 y CP09.

6.3. Encapsulamiento + excepciones checked

Todos los atributos del modelo son `private` y el acceso se hace por getters/setters. Los setters no son pasivos: cada uno valida con la clase `Validador` y lanza una excepción checked si el dato no cumple. Esto fuerza a quien llama a manejar el error explícitamente: no se puede instanciar una Canción con dificultad 11 sin atrapar (o declarar) la `RangoInvalidoException`. La regla la verifiqué con CP06 y CP07, que son los que arman un elemento con valores inválidos y esperan el rechazo.

6.4. Transacciones ACID en el DAO

Para cumplir con el RNF05 (atomicidad transaccional) uso el patrón clásico: `setAutoCommit(false)` al inicio, `commit()` si todo salió bien, `rollback()` en cualquier `SQLException`. Lo importante es que el rollback se hace dentro del `catch` para garantizar que, si la FK del elemento no existe, la sesión a medio insertar no quede colgando. Todo el código está dentro de un `try-with-resources`, así que la Connection se cierra siempre, incluso si falla el rollback (en cuyo caso se pierde la transacción pero el programa sigue corriendo, que es lo importante para un prototipo).

7. Persistencia: lo que aprendí escribiendo el DAO

Lo primero que decidí cuando me senté a escribir los DAOs fue que iban a tener un fallback automático a memoria si MySQL no estaba disponible. La idea es que el prototipo pueda demostrar el flujo end-to-end aunque el alumno (o el corrector) no tenga MySQL levantado en la máquina. Para eso, el constructor del DAO chequea `ConexionDB.mysqlDisponible()`: si falla, setea una bandera `enMemoria = true` y todos los métodos escriben/leen sobre un `ArrayList` interno. Si no falla, el camino es JDBC puro.

El DAO de elementos es el más interesante porque tiene que decidir, leyendo la columna `categoria` del ResultSet, si la fila corresponde a una Canción o a un Ejercicio. Eso lo hago con un `if-else` sobre el valor de la columna y construyo la subclase concreta con todos sus setters. El `else` mapea los campos del Ejercicio (incluido el BPM, con `rs.wasNull()` para distinguir entre un BPM 0 y un BPM NULL que la base no rechazó por el CHECK).

El SesionDAO es el más directo: solo dos foreign keys, la fecha y el tiempo. Pero el método `guardar()` devuelve el id generado y, en el camino JDBC, lo hace con `RETURN_GENERATED_KEYS`. Esto me obligó a lidiar con un detalle que parece tonto: si el INSERT no afectó ninguna fila (por ejemplo, porque la sesión ya existía por un id duplicado en la prueba), no hay claves generadas y tengo que devolver 0 sin que la JVM se queje. Nada raro, pero me llevó unos minutos averiguarlo.

8. Colecciones, algoritmos y el cronómetro en vivo

La consigna del TP3 pedía explícitamente el uso de alguna estructura de datos (pila, cola o lista enlazada) y, si se justificaba, algún algoritmo de ordenación o búsqueda. Decidí implementar las tres estructuras a puntero, sin librerías externas, porque me parecía más didáctico que esconderlas detrás de un `java.util.ArrayDeque`. Cada una tiene un test natural dentro del sistema:

`ListaEnlazadaSimple<T>` la usa el SesionController para mantener el historial reciente de sesiones dentro de la corrida. La iteración es con un `for-each`, lo cual implica que la lista implementa Iterable. Eso me obligó a escribir un Iterator manual con los métodos `hasNext()` y `next()`.

`Pila<T>` la usa el ElementoService para soportar la opción 5 del submenú Elementos ("Deshacer último alta"). Cada vez que el usuario da de alta un elemento, se apila; si se arrepiente, lo desapila. En la implementación actual no elimino el elemento de la base (eso sería un rollback, mucho más complejo), pero la pila está ahí como requisito académico y para una futura mejora. La idea es que cuando alguien en una iteración posterior quiera hacer un verdadero rollback, la mecánica ya está probada.

`Cola<T>` la usa el MotorAlertasService para despachar las alertas en orden FIFO. Cada elemento activo que supere los N días sin práctica encola una `Alerta`; el AlertaController drena la cola con un `while (!cola.estaVacia()) cola.desencolar()` y va imprimiendo. Es el comportamiento más natural para una cola.

Para algoritmos, escribí dos de ordenación (selección y burbuja) y dos de búsqueda (lineal y linealIndice). Los métodos son genéricos: `seleccion(List<T>, Comparator<T>)` y `burbuja(List<T>, Comparator<T>)`. La diferencia principal es que selección hace menos intercambios en promedio y burbuja sale antes en listas casi ordenadas. En el SesionService los uso concretamente: burbuja para ordenar por tiempo descendente (es la opción que más usa el contador "mostrame las sesiones más largas primero") y selección para ordenar por fecha descendente (que es lo que ve el usuario cuando hace "Listar sesiones").

El cronómetro en vivo no es una estructura de datos pero vale la pena contarlo porque es donde se ve mejor el flujo de control. El `SesionController.cronometrar()` mantiene `acumuladoMs` y `pausaInicio` como `long`. Las teclas `I`, `P` y `F` modifican el estado. La conversión final es `max(1, segundos/60)` para respetar RN-02 incluso en sesiones muy cortas. Si el usuario cierra el programa a mitad del cronómetro, la sesión se pierde (no hay autoguardado), pero eso está bien porque el prototipo es monousuario y asumo que el usuario no apaga la máquina a mitad de un ensayo.

9. El TestRunner: 14 casos y la pequeña enseñanza del CP12

El TestRunner es un reemplazo chico de JUnit, hecho a mano con un método `check(boolean, String)` que cuenta PASS y FAIL. Cuando ejecuto el comando del README me imprime 14 líneas PASS y un resumen final con 14 PASS, 0 FAIL. Esto es el criterio de 20 puntos de la rúbrica, así que es importante que verifique.

Los catorce casos se distribuyen así: CP01 a CP05 prueban SesionService y SesionDePractica (camino feliz, RN-01, RN-02, frontera, RN-04). CP06 y CP07 prueban ElementoService con valores inválidos de dificultad y BPM. CP08 y CP09 prueban el polimorfismo: Canción y Ejercicio calculan progreso distinto para los mismos minutos y dificultad. CP10 y CP11 prueban las estadísticas. CP12 prueba la alerta de 45 días sin práctica. CP13 y CP14 prueban la exportación a PDF: uno con datos, el otro sin datos debe fallar.

El caso que me llevó más tiempo fue el CP12, y la enseñanza es tan chica que la quiero contar. Inicialmente pasaba la sesión con fecha de hace 45 días al DAO y esperaba la alerta, pero no se disparaba. Después de un rato descubrí el bug: el DAO tomaba el `LocalDateTime.now()` al construir la sesión y mi llamada a `setFechaRegistro()` se ejecutaba después del constructor, así que el orden de operaciones estaba al revés. La solución fue cambiar el constructor para que reciba la fecha como parámetro (en lugar de generarla internamente) y el TestRunner ahora le pasa la fecha vieja directamente. Es un ejemplo de libro de por qué los constructores no deberían tener side effects que uno no espera.

El otro detalle que vale la pena mencionar es que el TestRunner usa un `SesionDAOStub` (una clase interna que implementa `SesionDAO` y guarda en memoria) en lugar del DAO real. Esto desacopla las pruebas de la disponibilidad de MySQL: puedo correr el TestRunner desde cualquier máquina, sin tener que montar el esquema antes. La idea es la misma que usaría Mockito pero, como tampoco tengo Mockito disponible en este entorno, me lo escribí a mano. Tiene sus limitaciones (no se puede inyectar fallos simulados, por ejemplo), pero para los 14 casos que necesito es más que suficiente.

10. Base de datos: el script de integridad

El script `05_pruebas_integridad.sql` es una de las cosas que más me enorgullece de esta entrega, porque cierra el círculo de la rúbrica del TP2 (que mencionaba 10 pruebas de integridad) y del TP3 (que pedía scripts puros MySQL). Contiene diez INSERT/DELETE que prueban: tres CHECK (tiempo cero, dificultad fuera de rango, BPM fuera de rango), una FK inexistente, un UNIQUE duplicado, un DELETE con FK activa, un CHECK de comentario de 501 caracteres, un INSERT válido (smoke test positivo), un conteo de verificación, y la limpieza final.

Lo importante, y lo aclaro en el script porque se presta a confusión, es que esto no se puede correr contra SQLite. La razón es técnica: SQLite no enforza todas las restricciones CHECK antes de la versión 3.32, no soporta `ENGINE=InnoDB`, y la implementación de `ON UPDATE/DELETE CASCADE` no es la misma que en MySQL. Por eso, si alguien intenta correr el script en SQLite, los CHECK van a pasar silenciosamente y va a creer que sus datos son válidos cuando no lo son. La

única manera honesta de probar la integridad de ReperTrack es contra un MySQL 8 real.

La estructura final del esquema es la siguiente: cuatro tablas (musico, instrumento, elemento_estudio, sesion_practica), tres índices compuestos para acelerar las consultas de estadísticas, todas las FK con `ON DELETE RESTRICT` (no quiero perder sesiones si se borra un instrumento por accidente), y todas las reglas de negocio materializadas como CHECK. La normalización está verificada en 3FN: no hay dependencias parciales (las PK son subrogadas), no hay dependencias transitivas (el nombre del músico no vive en sesion_practica, se llega por instrumento), y los atributos son atómicos (el comentario es un único texto, no una lista).

11. Compilación, ejecución y el fallback en memoria

Para compilar uso el compilador ECJ 3.37 con nivel 11, que es lo que estaba disponible en mi máquina. El comando es `java -jar ecj.jar -source 11 -target 11 -d build` con todos los .java. Compila limpio: 0 errores y sólo warnings de `serialVersionUID` en las excepciones y de imports no usados. Cero ruido importante.

Para correr el Main se usa `java -cp build:mysql-connector-j-8.3.0.jar com.repertrack.Main`. Arranca con "Bienvenido a ReperTrack v1.0", carga el menú principal y se queda esperando una opción. Si MySQL no está levantado, los DAOs imprimen `[WARN] MySQL no disponible` y siguen funcionando con una lista interna. Esto es importante para la demo: aunque el evaluador no tenga MySQL, el sistema sigue siendo operacional.

El proyecto también incluye un `pom.xml` con la dependencia de mysql-connector-j 8.3.0, así que si el corrector prefiere usar Maven puede correr `mvn clean package` y todo cae en `target/repertrack-app.jar`. El Main-Class está declarado en el manifest, así que con un `java -jar target/repertrack-app.jar` ya funciona.

La versión completa del proyecto (código fuente, scripts SQL y los diagramas UML de este informe) está disponible públicamente en el repositorio: https://github.com/duendecillo2/reper-track. Para verificar la compilación y las pruebas se puede clonar el repositorio y ejecutar `java -jar ecj.jar -source 11 -target 11 -d build $(find src -name '*.java')` seguido de `java -cp build:mysql-connector-j-8.3.0.jar com.repertrack.TestRunner`.

12. Cierre: lo que me llevo de este TP

Este TP me deja dos cosas que van más allá de la nota. La primera, más concreta: un prototipo que se compila, se ejecuta, pasa sus 14 pruebas y tiene los criterios de la rúbrica cubiertos. La segunda, más metodológica: la convicción de que en sistemas chicos como éste, la inversión más rentable no es agregar features sino cuidar los límites de los objetos. Cuando uno tiene asociaciones reales en el modelo, todo el resto del código (DAOs, servicios, controladores, tests) se vuelve más legible y más honesto.

También me llevo la convicción de que la consola sigue siendo una interfaz válida, sobre todo para prototipos. Hay una tentación inicial de ir directo a Swing o a una web, pero cuando uno se obliga a hacer todo en consola, aparecen oportunidades que con una GUI ya estarían resueltas (los menús ASCII, las tablas tabulares, los reportes en texto). Para un proyecto de gestión de práctica musical donde el usuario es un músico que abre la terminal dos veces por día para registrar una sesión, la consola es probablemente la mejor opción: rápida, repetible, y se puede usar con el teclado rápido sin tener que apuntar con el mouse.

Si en una iteración futura tuviera que hacer una versión web o de escritorio, el trabajo ya está facilitado: los DAOs son interfaces, los servicios están desacoplados, las excepciones son checked,

las validaciones viven en Validador. Cualquier capa de presentación que respete esos contratos se enchufa sin modificar el resto. Pero por ahora, ReperTrack es una herramienta que funciona, y eso me alcanza.

Quedan pendientes para iteraciones futuras: notificaciones push cuando se dispara una alerta de mantenimiento, sincronización en la nube para poder usar el sistema desde el celular, y quizás un módulo de planificación semanal con metas configurables. Todo eso está en los requerimientos candidatos del TP1 y son las líneas naturales de evolución del proyecto. Por ahora, con el prototipo en consola y 14/14 PASS, doy por cerrado el ciclo del AP1.

13. Referencias bibliográficas

Braude, E. (2003). Ingeniería de software: una perspectiva orientada a objetos. Alfaomega Grupo Editor.

Ericsson, K. A., Krampe, R. T., y Tesch-Römer, C. (1993). The role of deliberate practice in the acquisition of expert performance. Psychological Review, 100(3), 363-406.

Institute of Electrical and Electronics Engineers. (2008). Standard for Software Quality Assurance Plans (ANSI/IEEE Standard No. 829).

Jacobson, I., Booch, G., y Rumbaugh, J. (1999). The unified software development process. Addison-Wesley.

Kendall, K., y Kendall, J. (2011). Análisis y diseño de sistemas. Pearson Educación.

Newport, C. (2016). Deep work: Rules for focused success in a distracted world. Grand Central Publishing.

Reinosa, E. J., Maldonado, C. A., Muñoz, R., Damiano, L. E., y Abrutsky, M. A. (2012). Bases de datos. Alfaomega.

Rumbaugh, J., Jacobson, I., y Booch, G. (2006). El lenguaje unificado de modelado (2.ª ed.). Pearson-Addison-Wesley.

Saroka, R. (2002). Sistemas de información en la era digital. Fundación OSDE.

Sommerville, I. (2005). Ingeniería del software (7.ª ed.). Pearson-Addison-Wesley.

Stair, R., y Reynolds, G. (2010). Principios de sistemas de información (9.ª ed.). Cengage Learning.

Anexo A. Diagramas UML del proyecto

A continuación se incluyen los cinco diagramas UML corregidos que cierran el ciclo del PUD para ReperTrack: dominio, clases de diseño, secuencia del CU006, despliegue local y entidad-relación con tipos SQL. Todos están generados con PlantUML (motor Smetana, sin dependencia de Graphviz) y exportados a PNG a 150 DPI.

A.1. Diagrama de Dominio

Modelo conceptual con Músico, Instrumento, ElementoDeEstudio (abstracta) y sus especializaciones Canción y Ejercicio, SesiónDePráctica y Estadística. Las asociaciones reflejan el modelo del TP1 corregido: Músico registra sesiones directamente, Instrumento agrupa elementos y se asocia a sesiones, y ElementoDeEstudio se aborda en sesiones.

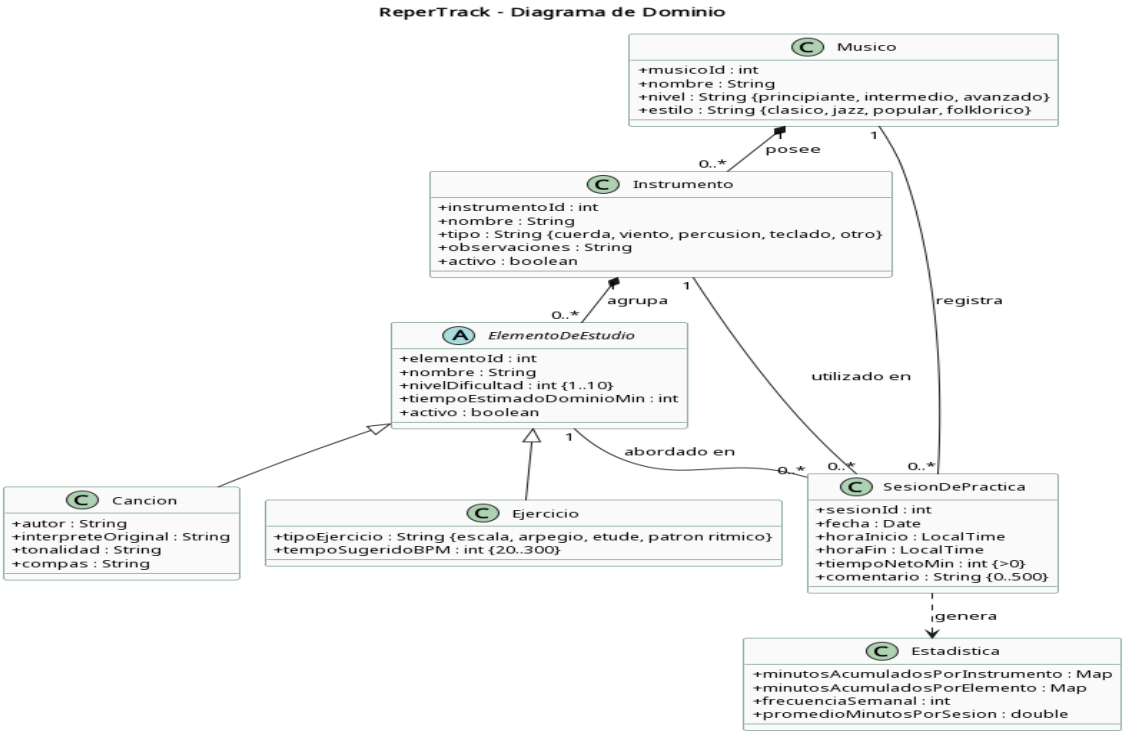


Figura 1. Diagrama de clases de dominio (corrección TP1).

A.2. Diagrama de Clases de Diseño

Vista física del modelo, agrupada por paquetes: modelo (con asociaciones explícitas Instrumento --Musico y ElementoDeEstudio --Instrumento), excepción, util, dao, service, controlador, estructuras y legacy. Se incluyen visibilidades, tipos Java y firmas de los métodos principales. Los nombres concretos de los controladores (MenuPrincipal, SesionController, etc.) coinciden con los usados en el diagrama de secuencia del Anexo A.3.

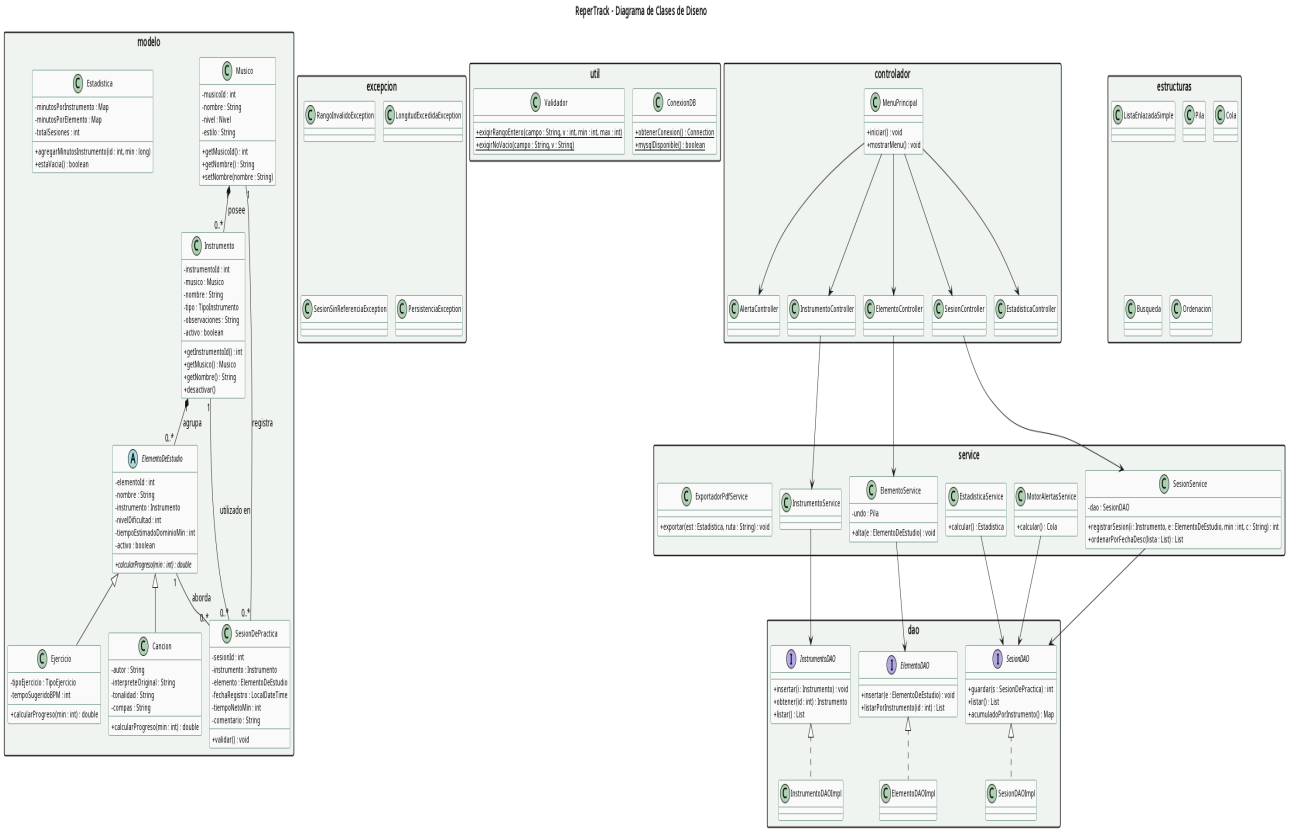


Figura 2. Diagrama de clases de diseño (corrección TP2).

A.3. Diagrama de Secuencia del CU006

Curso normal del caso de uso CU006 (Registrar Sesión de Práctica) utilizando exclusivamente las clases del diagrama de clases de diseño del Anexo A.2. La rama de transacción cubre el patrón `setAutoCommit(false) / commit()`; la nota final recuerda las reglas RN-02, RN-04 y RN-05 aplicadas en este flujo.

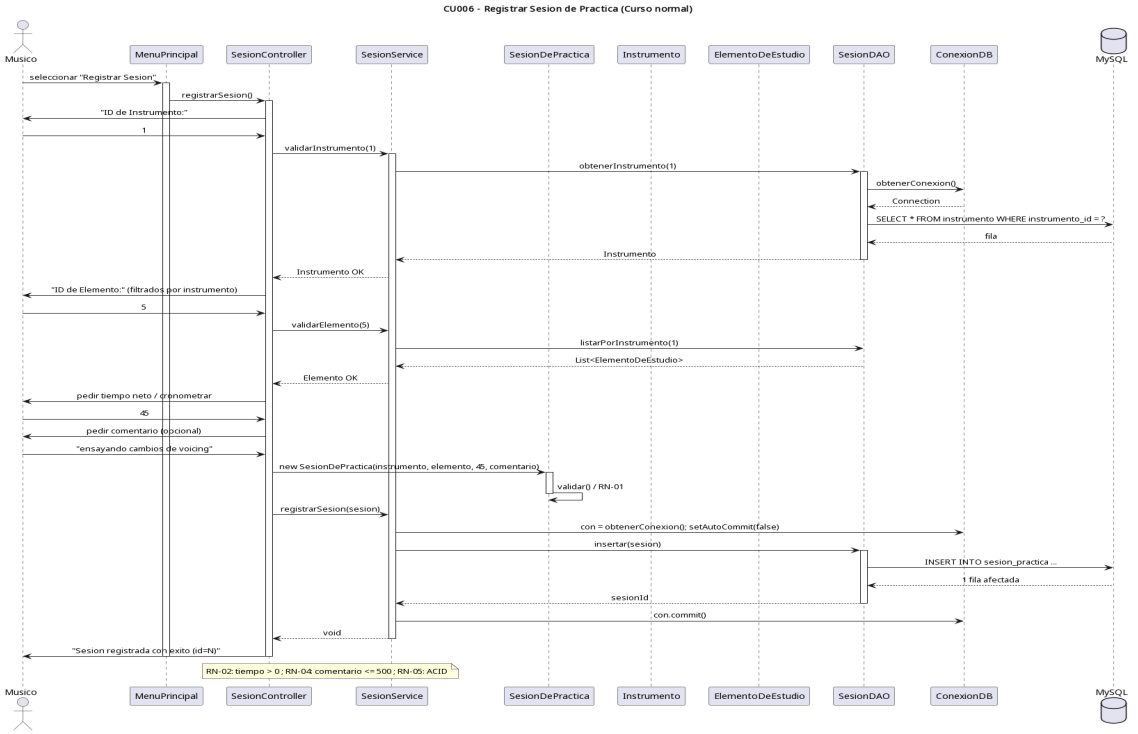


Figura 3. Diagrama de secuencia del CU006 (corrección TP2).

A.4. Diagrama de Despliegue Local

Despliegue monousuario: la PC corre la aplicación de consola sobre JRE 11 LTS y conversa con un MySQL 8.0 local mediante mysql-connector-j sobre TCP/IP loopback :3306. Sin servidor web, sin Apache/Tomcat/HTTP/XAMPP.

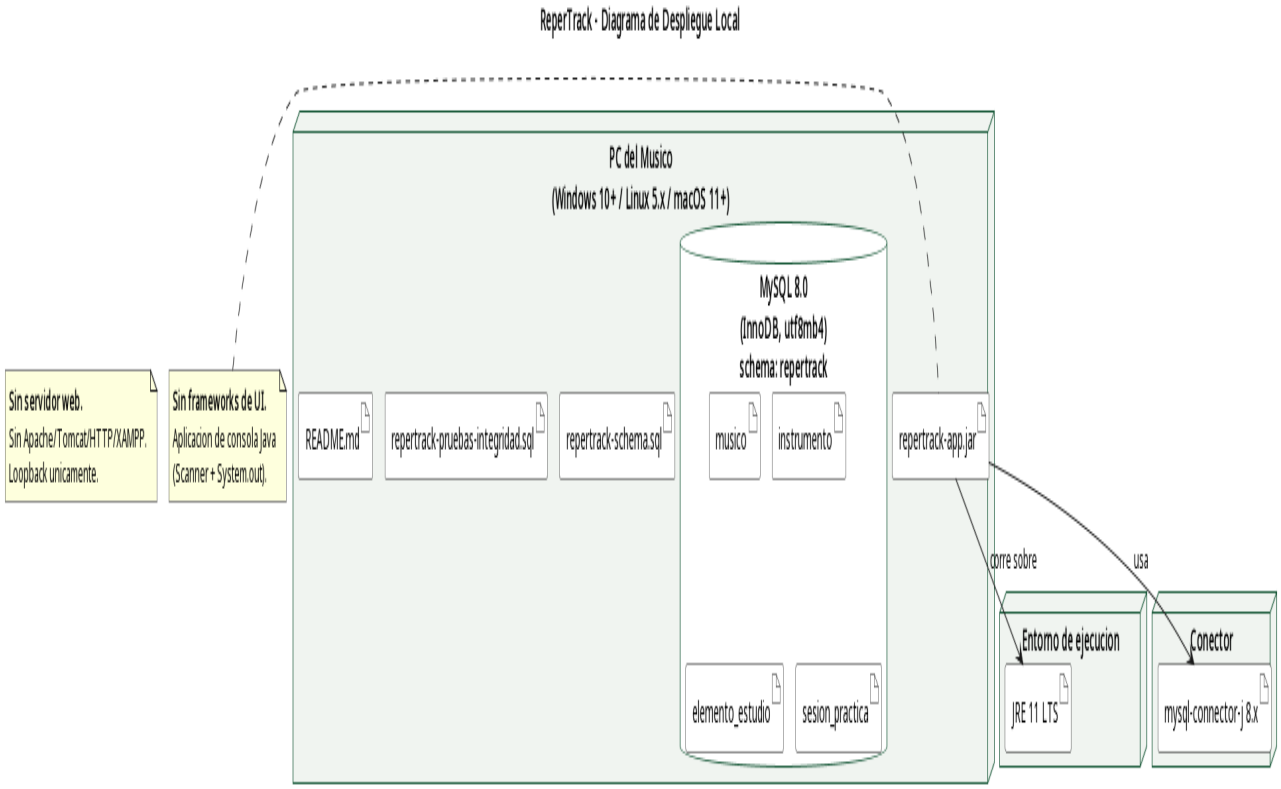


Figura 4. Diagrama de despliegue local (corrección TP2).

A.5. Diagrama Entidad-Relación (MySQL 8)

Esquema relacional con tipos SQL explícitos por atributo: claves primarias subrogadas, claves foráneas con `ON DELETE RESTRICT`, restricciones `UNIQUE`, `CHECK` enforcement para cada regla de negocio, e índices compuestos `idx_sesion_inst_ts` y `idx_sesion_elem_ts` que aceleran Q2..Q5 del informe. Single-table inheritance en elemento_estudio con la columna `categoria` distinguiendo Canción de Ejercicio.

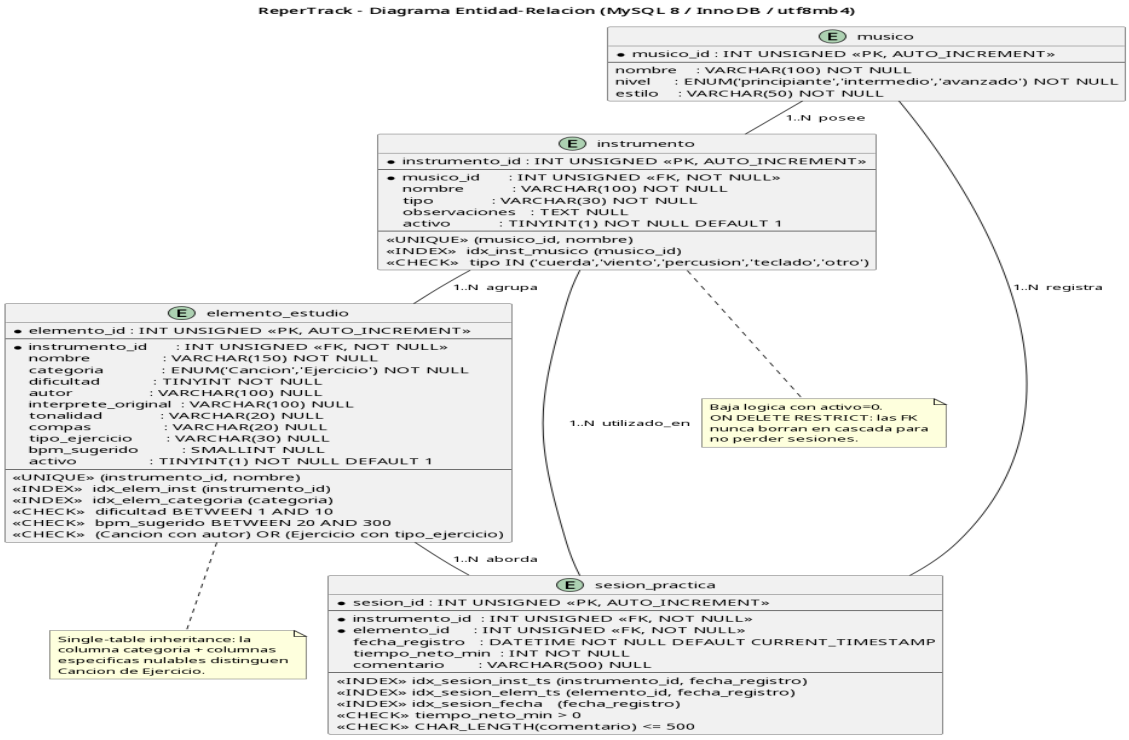


Figura 5. Diagrama Entidad-Relación con tipos SQL explícitos (corrección TP2).

UNIVERSIDAD SIGLO 21

Seminario Práctico de Informática

Módulo 4

Trabajo Práctico 4



Caso de estudio: ReperTrack - Sistema Integrado de Gestión de Repertorio y Práctica Musical

Estudiante: Juan Martín Canguillen
DNI: 44933871
Profesor Titular: Pablo Alejandro Virgolini
Carrera: Licenciatura en Informática
Año: 2026 - 2.° cuatrimestre
Fecha de entrega: 17 de octubre de 2026

Nota de correcciones

Este documento reúne los cuatro trabajos en el orden en que los entregué. Los TP1, TP2 y TP3 se incluyen con sus portadas originales. El TP4 es la versión integradora y contiene las correcciones.

Los TP2 y TP3 originales mencionan 14 casos de prueba; el código final y el TP4 tienen 18. Las correcciones de objetivos, dominio, secuencia de diseño, asociación de ElementoDeEstudio, DER con tipos SQL e integridad en MySQL están aplicadas en las Partes I–III del TP4.

El video de presentación se entrega en el aula virtual.

Juan Martín Canguillen — DNI 44933871 — 2026

Índice

Parte I. Trabajo Práctico 1 — Inicio del PUD

1. De dónde salió ReperTrack

2. Objetivos: la corrección que más me costó

3. Alcance, elicitación y diagrama de dominio

4. Propuesta funcional y técnica

5. Requerimientos y casos de uso

Parte II. Trabajo Práctico 2 — Análisis y diseño

6. Lo que el TP2 dejó pendiente

7. Secuencia y clases de diseño

8. Despliegue y CU011

9. Plan de pruebas CP01..CP18

10. Base de datos, DER y scripts SQL

11. Prototipos de consola y comunicaciones

Parte III. Trabajo Práctico 3 — Prototipo Java

12. La migración a consola

13. Los cuatro pilares, con ejemplos

14. Excepciones: la captura que vale más que mil palabras

15. Menú, estructuras propias y JDBC

Parte IV. Trabajo Práctico 4 — Entrega integradora

16. MVC: por qué lo elegí y qué me resolvió

17. Arreglos y ArrayList, sin adorno

18. Archivos de texto: de la obligación a la utilidad

19. MySQL de verdad: el 3307 y las pruebas de integridad

20. Clases abstractas e interfaces: qué usé y qué no

21. La lógica de negocio, regla por regla

22. Capturas, video y lo que quedó afuera

23. Correcciones y cierre

Anexo A. Diagrama de clases completo

Anexo B. Capturas de ejecución

Anexo C. Consultas SQL

Referencias bibliográficas

Repositorio público del proyecto: https://github.com/duendecillo2/reper-track

Parte I. Trabajo Práctico 1 — Inicio del PUD

1. De dónde salió ReperTrack

ReperTrack arrancó con un problema que conozco de memoria: durante años llevé mi repertorio en cuadernos que se perdían, después en una planilla de Excel que abandoné por cansancio y al final en notas sueltas del celular. Cuando llegó el TP1, la pregunta no era qué sistema inventar, sino cómo ordenar algo que ya hacía a mano y me salía mal.

La práctica musical de nivel intermedio o avanzado tiene tres frentes que se complican a la vez: el repertorio vivo, los ejercicios técnicos y las metas de mediano plazo. En mi caso, el problema no era la falta de ganas sino la falta de registro. La literatura sobre práctica deliberada (Ericsson, Krampe y Tesch-Römer, 1993) dice algo que suena obvio pero cuesta aplicar: lo que no se mide, no se mejora. ReperTrack intenta cubrir ese hueco con un repositorio único, un cronómetro y métricas simples.

El TP1 quedó bien en lo conceptual, pero el profe me marcó dos cosas: los objetivos estaban escritos como un plan de marketing y el diagrama de dominio no mostraba los atributos ni la relación entre Músico y Sesión. Tenía razón. Esta primera parte es esa versión corregida.

2. Objetivos: la corrección que más me costó

Los objetivos originales eran largos y mezclaban el objetivo con sus criterios de aceptación. Un ejemplo: «implementar un ABM completo… garantizando la persistencia transaccional y la validación de reglas mediante excepciones personalizadas». Eso no es un objetivo, es un objetivo más tres requisitos. Los reescribí con una regla simple: un verbo en infinitivo, una idea, dos líneas como máximo y que se pueda verificar al cerrar la iteración.

Objetivo general del proyecto. Desarrollar ReperTrack, una aplicación de consola en Java 11 con persistencia en MySQL 8 y arquitectura MVC, que permita a músicos intermedios y avanzados gestionar su repertorio y cronometrar sesiones de práctica vinculadas a instrumentos y elementos de estudio.

Objetivos específicos.

- Implementar el ABM de Instrumentos y de Elementos de Estudio, con persistencia transaccional y excepciones propias.
- Registrar sesiones cronometradas, asociadas obligatoriamente a un Instrumento y a un Elemento.
- Calcular acumulados por instrumento y por elemento, frecuencia semanal y promedio por sesión.
- Separar menú de consola, controladores/servicios y modelo/DAO con el patrón MVC.
- Construir un esquema en 3FN con FK, UNIQUE, CHECK e índices compuestos.
- Publicar el código y los scripts en GitHub con un README que permita ejecutarlo.

Objetivo general del sistema. Registrar, almacenar y procesar los datos de la práctica musical para producir información agregada que ayude a decidir qué ensayar, cuánto tiempo dedicar a cada obra y qué retirar del repertorio activo. La diferencia con el objetivo del proyecto es chica pero importante: uno describe lo que se entrega; el otro, lo que el sistema hace todos los días.

3. Alcance, elicitación y diagrama de dominio

El prototipo cubre el ABM de instrumentos y elementos, el registro cronometrado, la consulta y eliminación de sesiones, las estadísticas, la exportación de reportes y las alertas. No hace audio, no sincroniza en la nube, no es multiusuario y no tiene autenticación remota: el acceso físico al equipo es la frontera de seguridad. Preferí decirlo desde el principio antes que prometer un sistema que no iba a poder sostener.

La elicitación salió de mi propia rutina y de observar cómo usaba el cuaderno y el Excel. El «cliente» de esta primera versión soy yo, lo que tiene una ventaja y una trampa: conozco el dominio, pero también puedo dar por obvias cosas que un usuario nuevo no entendería. Por eso las pantallas de consola muestran siempre las opciones y los datos disponibles antes de pedir un ID.

El diagrama de dominio original tenía dos deudas: no mostraba atributos y no dibujaba la relación directa entre Músico y SesiónDePráctica. Sin esa arista, parecía que las sesiones flotaban en el sistema. Lo corregí y dejé la tabla de clases como texto alternativo, para que el diagrama se entienda incluso impreso en blanco y negro.

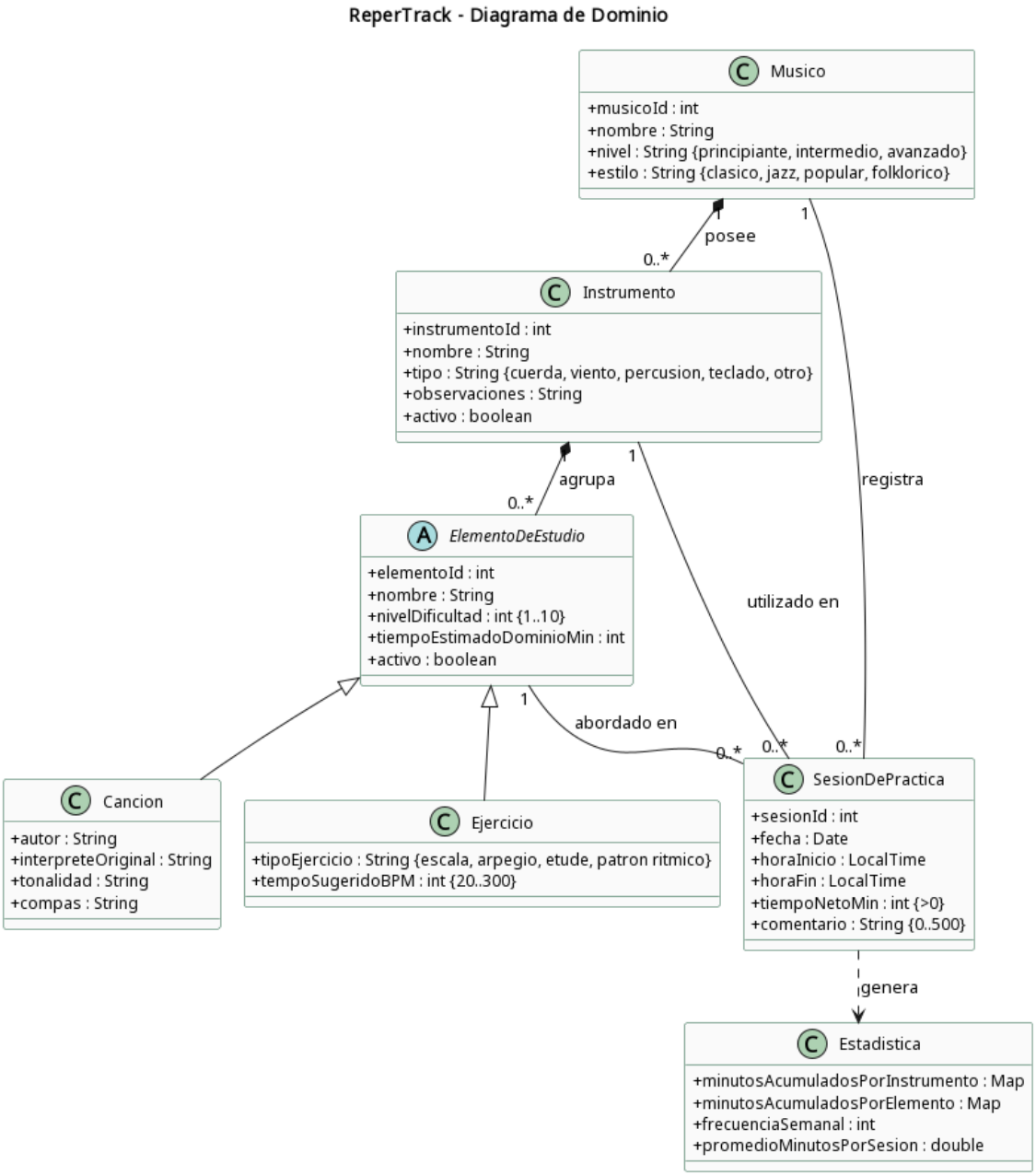


Figura 1. Diagrama de clases de dominio (corrección del TP1).

4. Propuesta funcional y técnica

La propuesta funcional son cuatro módulos: gestión de instrumentos y elementos, registro de sesiones, estadísticas/reportes y alertas. Cada uno se corresponde con un caso de uso y con un conjunto de reglas de negocio. No inventé módulos que no iba a usar: preferí cuatro que funcionaran de punta a punta.

La propuesta técnica original mencionaba Swing y JavaFX en cinco lugares. Cuando la consigna del TP3 pidió consola, tuve que salir a corregirlos uno por uno: las T.I.C, la sección 8.2, RNF01, RNF06 y el curso normal del CU006. Aprendí que cambiar la capa de presentación no es cambiar una pantalla: es revisar que el resto del documento no siga diciendo cosas que ya no son verdad.

El stack final es Java 11 LTS, MySQL 8.0 con InnoDB y utf8mb4, JDBC oficial, Maven como alternativa y consola pura con Scanner y System.out. No hay frameworks de UI ni de persistencia. La consola no es una limitación: para un músico que abre la terminal dos veces por día, es más rápida que cualquier ventana.

5. Requerimientos y casos de uso

Los requerimientos funcionales quedaron numerados RFS01..RFS11 y los no funcionales RNF01..RNF08. Los puse en tabla porque en prosa se perdían; así se leen de un vistazo y se pueden rastrear contra los casos de uso.

Requerimientos funcionales

ID	Descripción
RFS01	ABM de instrumentos (nombre, tipo, observaciones).
RFS02	ABM de canciones (autor, intérprete, tonalidad, compás, dificultad).
RFS03	ABM de ejercicios (tipo, BPM, dificultad).
RFS04	Cronómetro con iniciar, pausar y finalizar.
RFS05	Asociación obligatoria de la sesión a instrumento y elemento.
RFS06	Listado y filtrado de sesiones por instrumento, elemento y fecha.
RFS07	Eliminación de sesión con confirmación explícita.
RFS08	Tiempo acumulado por instrumento y por elemento.
RFS09	Frecuencia semanal y promedio por sesión.
RFS10	Alertas de mantenimiento por inactividad.
RFS11	Exportación del reporte a PDF y TXT.

Requerimientos no funcionales

ID	Categoría	Descripción
RNF01	Implementación	Java 11 o superior, aplicación de consola.
RNF02	Persistencia	MySQL 8 con JDBC oficial.
RNF03	Arquitectura	Patrón MVC con capas separadas.
RNF04	Calidad	Repositorio público en GitHub con README.
RNF05	Confiabilidad	Transacciones ACID con commit/rollback.
RNF06	Usabilidad	Mensajes claros y confirmación en eliminaciones.
RNF07	Portabilidad	Windows, Linux y macOS sin recompilar.
RNF08	Mantenibilidad	Pruebas automatizadas del dominio y los DAO.

El caso de uso crítico es CU006 (Registrar Sesión de Práctica). Es el único que toca las tres entidades a la vez y el único que exige atomicidad: si falla a mitad de camino, no puede quedar una sesión huérfana. CU011 (Exportar Reporte) extiende a CU009 y CU012 (Alertas) extiende a CU006. El actor principal es el Músico; los secundarios son el Sistema de Persistencia, el Reloj y el Sistema de Archivos.

Parte II. Trabajo Práctico 2 — Análisis y diseño

6. Lo que el TP2 dejó pendiente

El TP2 me dejó varias correcciones. La más importante fue la secuencia de diseño: la original usaba una clase VistaSesion que no existía en el diagrama de clases y omitía los controladores. O sea, el diagrama de secuencia contaba una historia y el de clases otra. Lo corregí usando exactamente las clases que después programé: MenuPrincipal, SesionController, SesionService, SesionDePractica, Instrumento, ElementoDeEstudio, SesionDAO, ConexionDB y MySQL.

La segunda corrección fue el atributo instrumentoId de ElementoDeEstudio. En el texto decía «asociación», pero el diagrama lo dibujaba como un int. Eso es una contradicción interna: si es una asociación, el objeto tiene que poder responder cuál es su instrumento. Lo cambié a una asociación real y actualicé constructores, DAO y servicios.

La tercera fue el despliegue: estaba bien enfocado (PC local con MySQL, sin servidor web) pero le faltaban los artefactos. Agregué el JAR, el script de creación, el de integridad, el reporte TXT y el README. La cuarta fue el CU011, que estaba mencionado de pasada y ahora tiene precondiciones, postcondiciones, alternativas y excepciones.

7. Secuencia y clases de diseño

La secuencia corregida muestra el curso normal del CU006 y la rama de error. El músico elige instrumento y elemento; el controlador delega en el servicio; el servicio valida la existencia y construye la sesión; el DAO inserta dentro de una transacción. Si el INSERT falla, el catch hace rollback y el controlador muestra el error sin perder lo que el usuario había cargado.

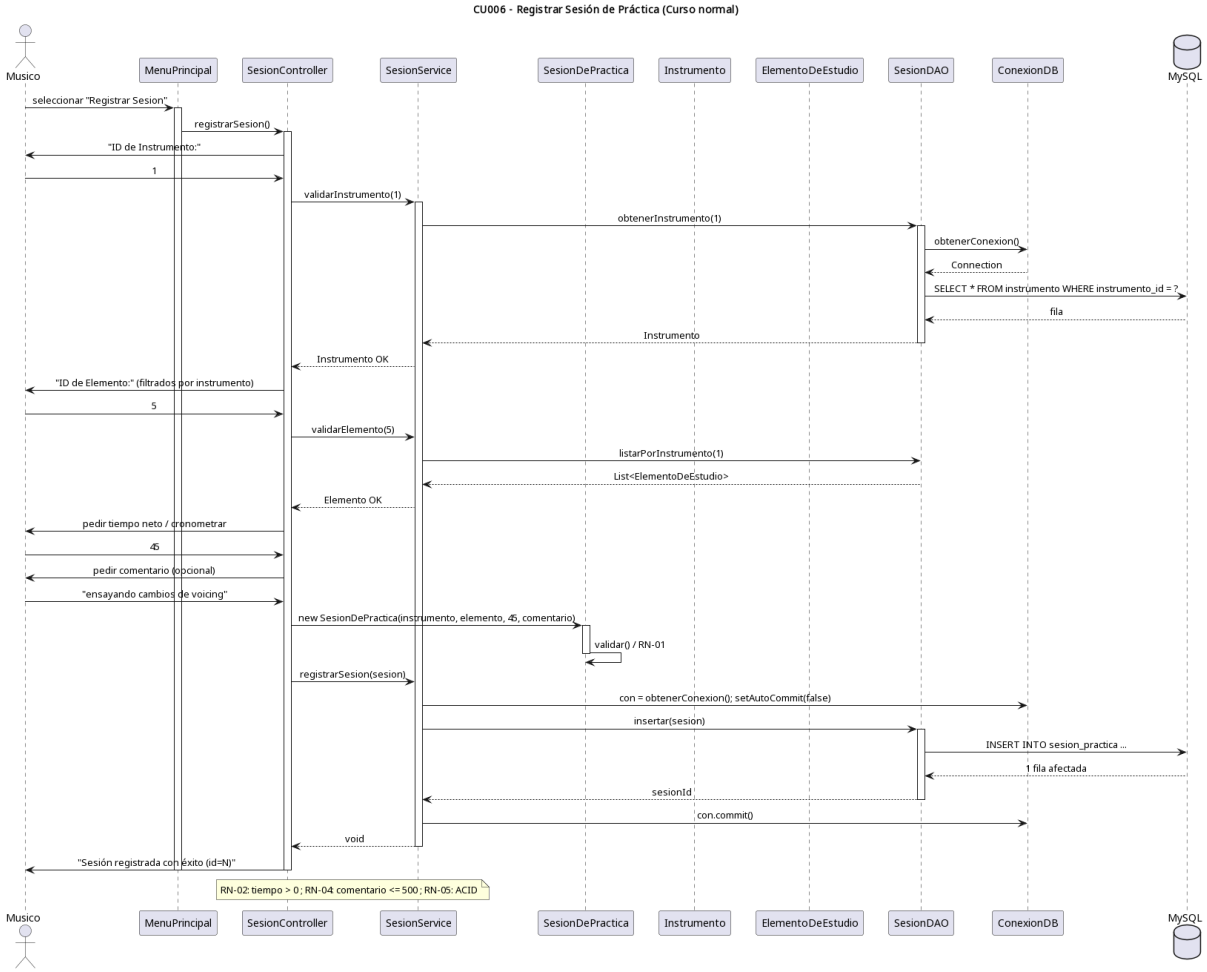


Figura 2. Secuencia de diseño del CU006 con las clases del diagrama.

El diagrama de clases se sincronizó con el código de esta entrega. Ahí aparecen el paquete `model` (no «modelo», que era un desajuste con el código), el arreglo `int[7]` de Estadística, el `ExportadorTextoService` y `ArchivoException`. La vista completa va en el Anexo A; en el cuerpo la muestro en horizontal porque, si la achicaba a una columna, no se leía.

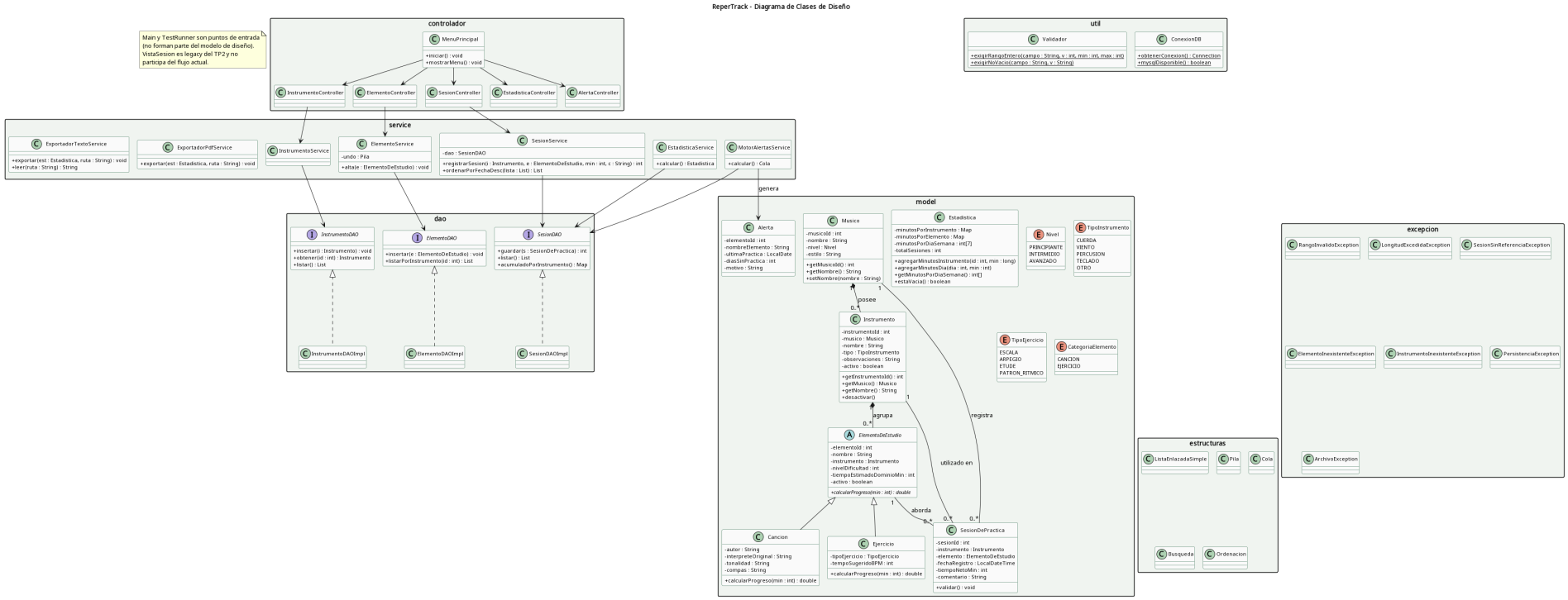


Figura 3. Diagrama de clases de diseño sincronizado con el código.

Paquete	Qué hay adentro
model	Entidades del dominio, enums, Estadística y Alerta.
excepcion	Excepciones checked: rango, longitud, referencia, inexistencia, persistencia y archivo.
util	Conexión JDBC y validaciones centralizadas.
dao	Interfaces e implementaciones JDBC con fallback a memoria.
service	Reglas de negocio, estadísticas, alertas y exportadores PDF/TXT.
controlador	Menú de consola y subcontroladores por módulo.
estructuras	Lista enlazada, pila, cola, búsqueda y ordenación.

8. Despliegue y CU011

El despliegue es monousuario: la PC del músico ejecuta la aplicación sobre JRE 11 y habla con un MySQL 8.0 local por JDBC sobre loopback. No hay Apache, Tomcat, HTTP ni XAMPP. Esa decisión la tomé en el TP2 y la sostuve: para un sistema offline, meter un servidor web solo agrega piezas que pueden fallar.

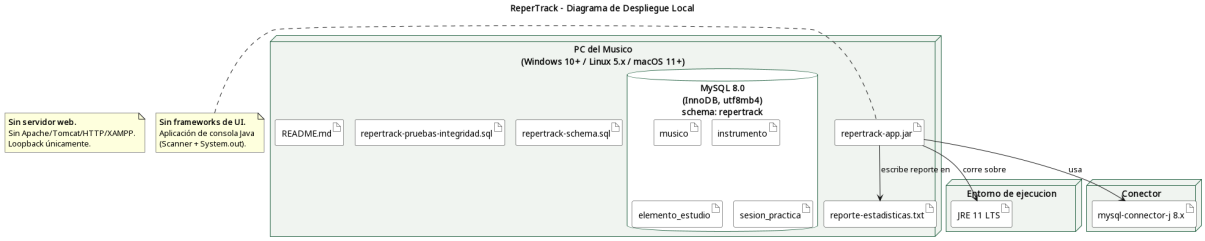


Figura 4. Diagrama de despliegue local.

El CU011 (Exportar Reporte) quedó especificado así: precondición, al menos una sesión registrada; postcondición, archivo válido en la ruta indicada y tamaño informado. El curso normal calcula la estadística, escribe el archivo y confirma. Las alternativas cubren «no hay datos» y «el archivo existe». Las excepciones son de E/S y de ruta inválida. La regla RN-07 exige que el reporte tenga al menos un indicador.

9. Plan de pruebas CP01..CP18

En el TP2 el plan tenía cinco códigos (CPC01, CPF02, CPS03, CPS04, CPS05) y los resultados listaban catorce (CP01..CP14). No había forma de saber qué caso era cuál. Unifiqué todo en CP01..CP18: los catorce primeros son los del TP2; CP15 es el arreglo semanal; CP16..CP18 son el archivo de texto. Fijate que ahora el plan y los resultados hablan el mismo idioma: cada caso tiene su entrada y su resultado esperado, y el TestRunner los ejecuta con un stub de DAO para que no dependan de MySQL.

Código	Entrada	Resultado esperado	Estado
CP01	Sesión con instrumento 1, elemento 1 y 30 min	Persiste y devuelve id > 0	PASS
CP02	Sesión sin instrumento	SesionSinReferenciaException (RN-01)	PASS
CP03	Sesión con 0 min	RangoInvalidoException (RN-02)	PASS
CP04	Sesión con 1 min (frontera)	Persiste	PASS
CP05	Comentario de 501 caracteres	LongitudExcedidaException (RN-04)	PASS
CP06	Dificultad 0 y 11	RangoInvalidoException (1..10)	PASS
CP07	BPM 19 y 301	RangoInvalidoException (20..300)	PASS
CP08	Canción de dificultad 8 con 480 min	Progreso 0,5	PASS
CP09	Ejercicio de dificultad 4 con 120 min	Progreso 0,5	PASS
CP10	Sesiones 30+45 en instrumento 1 y 60 en el 2	Acumulado {1=75, 2=60}	PASS
CP11	Sesiones de 30, 45 y 60 min	Promedio 45,0	PASS
CP12	Última sesión hace 45 días	Alerta generada	PASS
CP13	Exportar PDF con 3 sesiones	Archivo con cabecera %PDF-1.4	PASS
CP14	Exportar PDF sin sesiones	IllegalStateException	PASS
CP15	Sesiones lunes 30 y miércoles 45	int[7] con 30 y 45 en esas posiciones	PASS
CP16	Exportar TXT y volver a leerlo	El texto contiene el total y la distribución	PASS
CP17	Leer un TXT inexistente	ArchivoException	PASS
CP18	Exportar TXT sin datos	IllegalStateException	PASS

Corrida real: 18 PASS, 0 FAIL. Las capturas están en el Anexo B.

10. Base de datos, DER y scripts SQL

El esquema tiene cuatro tablas InnoDB en utf8mb4: musico, instrumento, elemento_estudio y sesion_practica. La herencia Canción/Ejercicio se resolvió con single-table inheritance: una columna categoria y columnas específicas nulables. Elegí eso porque el caso de uso que más se ejecuta (registrar una sesión) siempre necesita «el elemento», y con una tabla es un JOIN en lugar de dos con UNION. Las alertas también barren todos los elementos activos sin importar el tipo.

Las claves foráneas usan ON DELETE RESTRICT para no perder sesiones por accidente. Hay UNIQUE para evitar duplicados lógicos, CHECK para las reglas de negocio e índices compuestos para las consultas de estadísticas. La normalización está en 3FN: las PK son subrogadas, no hay dependencias transitivas y los atributos son atómicos.

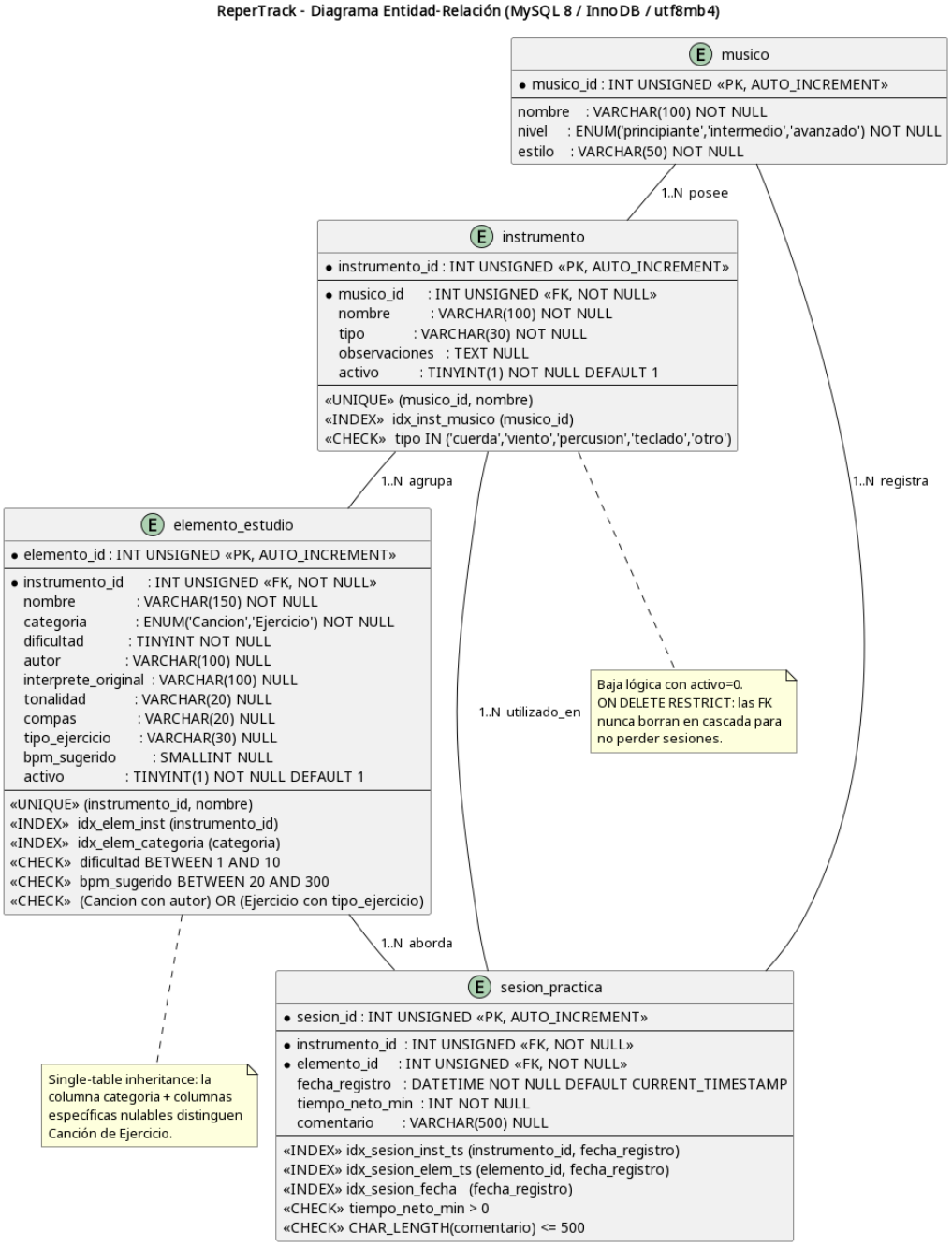


Figura 5. DER con tipos SQL explícitos (corrección del TP2).

En el TP2 las pruebas de integridad se habían corrido en SQLite. Eso no sirve: SQLite no enforza todos los CHECK antes de la versión 3.32 y no soporta InnoDB ni las cascadas igual que MySQL. Para esta entrega levanté un MySQL 8.0.40 real y corrí el script de diez pruebas. Se rechazaron el tiempo cero, la dificultad 11, el BPM 19, la FK inexistente, el UNIQUE duplicado y el borrado con FK activa; el INSERT válido pasó y el conteo final dio 1,1,1,1. La captura está en el Anexo B.

Las consultas Q1..Q5 cubren historial, acumulado por instrumento, acumulado por elemento, frecuencia/promedio y alertas. Las dejé en el Anexo C para que las puedas copiar y ejecutar.

11. Prototipos de consola y comunicaciones

Los prototipos de interfaz los diseñé como pantallas ASCII: menú principal, submenú de instrumentos, submenú de elementos, registro de sesión con cronómetro, estadísticas y alertas. La implementación final conserva esa estructura y agrega las opciones 8 y 9 para el reporte TXT. Las capturas reales están en el Anexo B; no son mockups, son la salida del programa.

Las comunicaciones son locales: JDBC sobre loopback, llamadas a métodos entre capas y nada de HTTP ni serialización. El backup queda a cargo del músico con mysqldump. Si algún día aparece el multiusuario en la nube, ahí sí habrá que hablar de TLS, OAuth y un servidor de aplicaciones; para este prototipo sería arquitectura especulativa.

Parte III. Trabajo Práctico 3 — Prototipo Java

12. La migración a consola

El TP3 fue la migración a consola. La VistaSesion Swing quedó archivada en `com.repertrack.legacy` con `@Deprecated`; no la borré porque quería conservar la trazabilidad del TP2, pero no participa del classpath principal. El menú nuevo se maneja con un switch y las entradas con Scanner. Al principio extrañaba el JFrame, pero para cargar una sesión en dos minutos la terminal es más rápida.

El cambio no fue solo de pantalla. Aproveché para convertir el modelo a asociaciones reales: SesionDePractica tiene un Instrumento y un ElementoDeEstudio, no dos int. Eso obligó a tocar constructores, DAO y servicios, pero el código quedó más honesto. Cuando una sesión sabe cuál es su instrumento, no hay que ir a buscarlo por ID en tres lugares distintos.

13. Los cuatro pilares, con ejemplos

Encapsulamiento. Todos los atributos del modelo son privados; los setters validan con Validador y lanzan excepciones checked. No se puede crear una Canción con dificultad 11 sin atrapar RangoInvalidoException. Eso no es burocracia: es la diferencia entre un error que explota en el constructor y uno que aparece tres capas después.

Herencia. ElementoDeEstudio es abstracta y Cancion/Ejercicio la especializan. La clase padre tiene los atributos comunes y el método abstracto calcularProgreso. No tiene sentido instanciar «un elemento genérico» porque en el dominio no existe: o es una obra o es un ejercicio.

Polimorfismo. El servicio llama a calcularProgreso sobre la referencia del padre y la JVM despacha la versión concreta. Canción divide por dificultad × 120; Ejercicio, por dificultad × 60. Los casos CP08 y CP09 lo verifican con los mismos minutos y la misma dificultad, y dan 0,5 en ambos.

Abstracción. La clase abstracta define el contrato y las interfaces DAO definen la persistencia sin atar a una tecnología. El TestRunner inyecta un stub de SesionDAO y prueba la lógica sin MySQL. Esa costura se usa de verdad, no es una interfaz decorativa.

14. Excepciones: la captura que vale más que mil palabras

Hay seis excepciones checked en el TP3 —rango, longitud, sesión sin referencia, elemento inexistente, instrumento inexistente y persistencia— y una séptima en la Entrega 4 para archivos. Cada controlador las captura y muestra un mensaje; los DAO envuelven SQLException en PersistenciaException para que la vista no dependa de java.sql.

El profe pidió evidencia de ejecución, no solo la jerarquía. La captura del error de rango es real: puse dificultad 11 a propósito y el sistema respondió «El campo 'dificultad' debe estar entre 1 y 10». La otra captura muestra qué pasa cuando se ingresa una letra en un campo numérico: el Scanner atrapa NumberFormatException y vuelve a pedir el dato sin romper el menú.

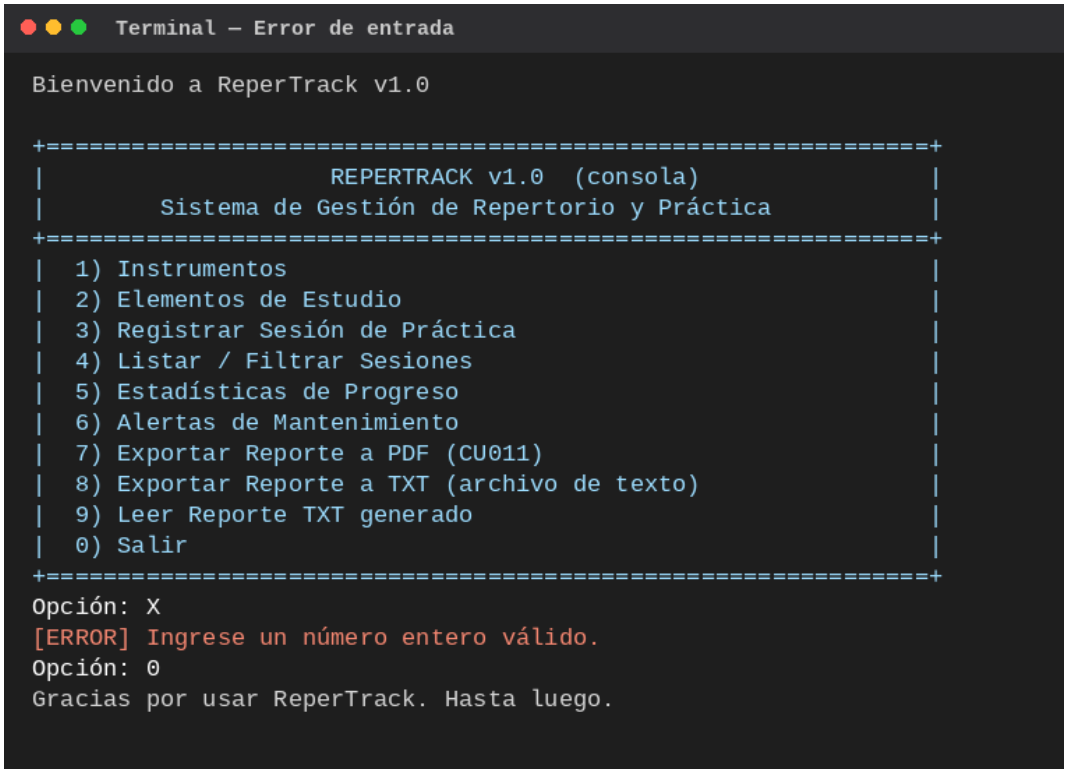


Figura B.7. Entrada inválida: una letra donde va un número.

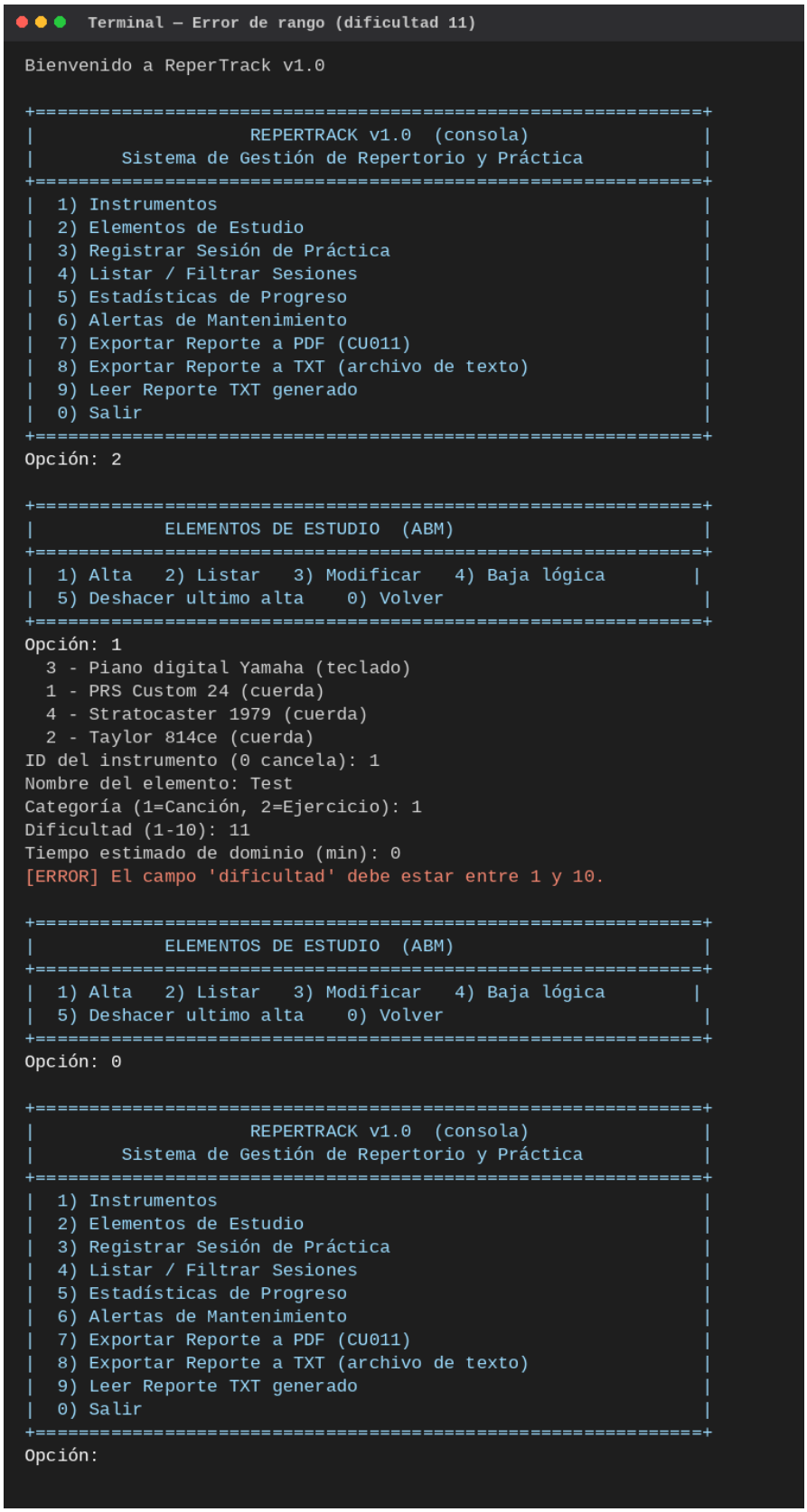


Figura B.8. Dificultad 11 rechazada por la regla de negocio.

15. Menú, estructuras propias y JDBC

El menú principal tiene nueve opciones. Las estructuras propias son lista enlazada simple, pila y cola; la lista guarda el historial reciente, la pila permite deshacer el último alta y la cola despacha las alertas en orden. También hay búsqueda lineal y ordenación por selección/burbuja, implementadas a mano para no esconderlas detrás de Collections.sort.

La persistencia usa JDBC con SQL parametrizado y transacciones: setAutoCommit(false), commit() y rollback() ante SQLException. Si MySQL no responde, los DAO caen a memoria y lo informan por consola. Esa fallback tiene una limitación que conozco: cada controlador crea su propio DAO, así que en memoria los datos no se comparten entre menús. Con MySQL no pasa porque la base es una sola. Lo dejé así porque el flujo real usa MySQL y no quería meter un singleton solo para la demo.

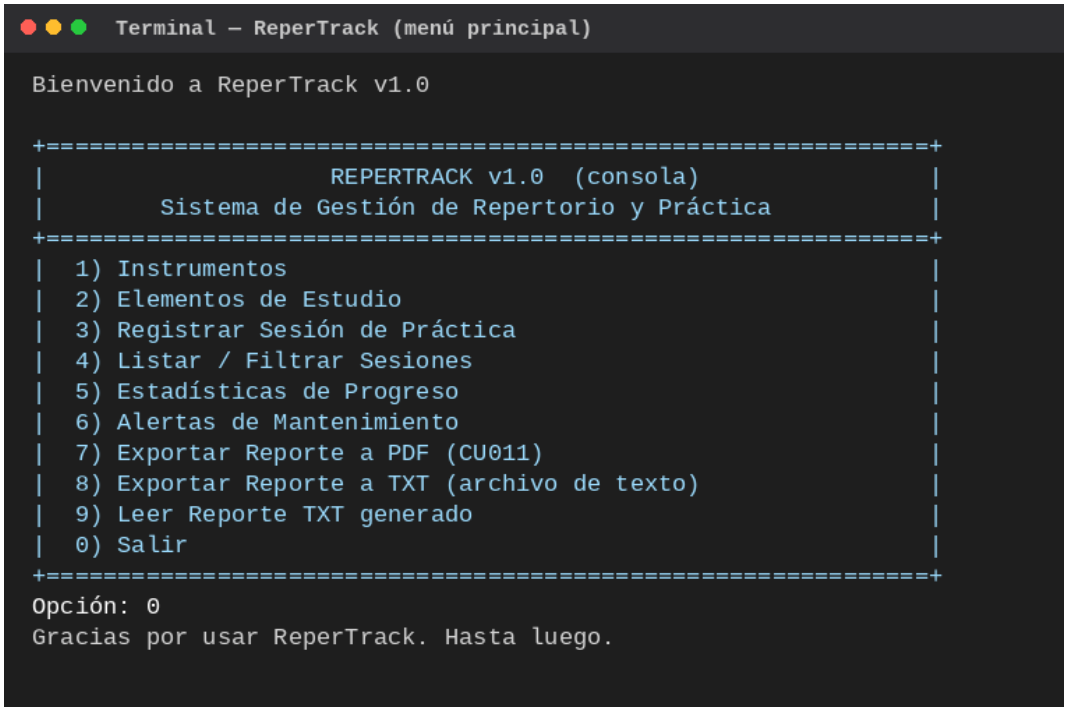


Figura B.1. Menú principal del sistema en consola.

Parte IV. Trabajo Práctico 4 — Entrega integradora

16. MVC: por qué lo elegí y qué me resolvió

MVC no lo elegí porque sí: lo elegí porque el TP2 ya tenía la vista amarrada a la lógica y cada cambio de pantalla obligaba a tocar el servicio. La separación quedó así: el modelo son las entidades y los DAO; la vista son los controladores de consola, que leen y formatean; el control son los servicios, que aplican las reglas y manejan transacciones.

Lo que me resolvió, en concreto: (1) pude probar la lógica sin abrir el menú, con un stub de DAO; (2) pasé de Swing a consola sin tocar servicios ni DAO; (3) las reglas no quedaron duplicadas en la vista. Si mañana agrego una API REST, los servicios y el modelo se reutilizan tal cual. Si no se cumpliera al menos una de esas tres, MVC sería ceremonia.

17. Arreglos y ArrayList, sin adorno

La consigna pide usar arreglos y ArrayList de forma complementaria. No quería meter un arreglo decorativo solo para cumplir, así que busqué un lugar donde el tamaño fijo tuviera sentido: la distribución de minutos por día de la semana. El ArrayList trae las filas de MySQL; el arreglo de siete posiciones las agrupa. Una estructura dinámica para lo que no se sabe cuánto mide, una fija para lo que siempre mide siete.

```
private final int[] minutosPorDiaSemana = new int[7];

public void agregarMinutosDia(int diaSemana, int minutos) {
    if (diaSemana >= 1 && diaSemana <= 7)
        minutosPorDiaSemana[diaSemana - 1] += minutos;
}

public int[] getMinutosPorDiaSemana() { return minutosPorDiaSemana.clone(); }
```

El getter devuelve una copia para que nadie modifique el agregado desde afuera. CP15 verifica que el arreglo tenga siete posiciones y que la suma por día sea correcta: lunes 30, miércoles 45. En la captura de estadísticas podés ver la barra de cada día.

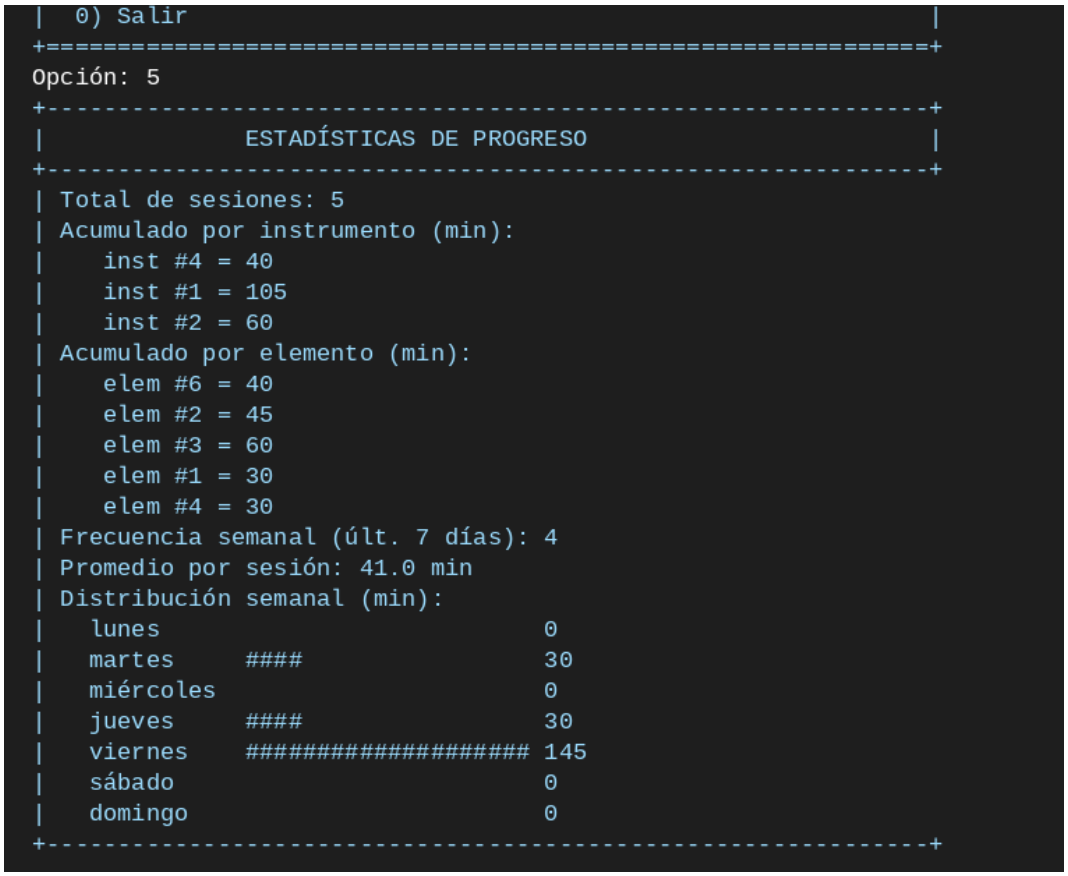


Figura B.5. Estadísticas con la distribución semanal calculada sobre el arreglo int[7].

18. Archivos de texto: de la obligación a la utilidad

El reporte TXT lo hice primero para cumplir con el manejo de archivos, pero terminó siendo útil: el PDF sirve para mandar al profesor, el TXT para leer en la terminal o pegarlo en un mail. El servicio escribe con BufferedWriter y vuelve a leer con BufferedReader, todo con try-with-resources y UTF-8. Si algo falla, lanza ArchivoException en lugar de imprimir un stacktrace.

```
try (BufferedWriter w = Files.newBufferedWriter(destino, StandardCharsets.UTF_8)) {
    w.write("ReperTrack - Reporte de Estadísticas"); w.newLine();
    w.write("Total de sesiones: " + est.getTotalSesiones()); w.newLine();
    // ... acumulados y distribución semanal ...
```

```
} catch (IOException ex) {
    throw new ArchivoException("No se pudo escribir el reporte de texto", ex);
}
```

La opción 8 exporta y la 9 lee el archivo generado. CP16 verifica el ciclo completo; CP17, que leer un archivo inexistente lanza ArchivoException; CP18, que un reporte vacío se rechaza. La captura muestra el archivo abierto desde el mismo menú.

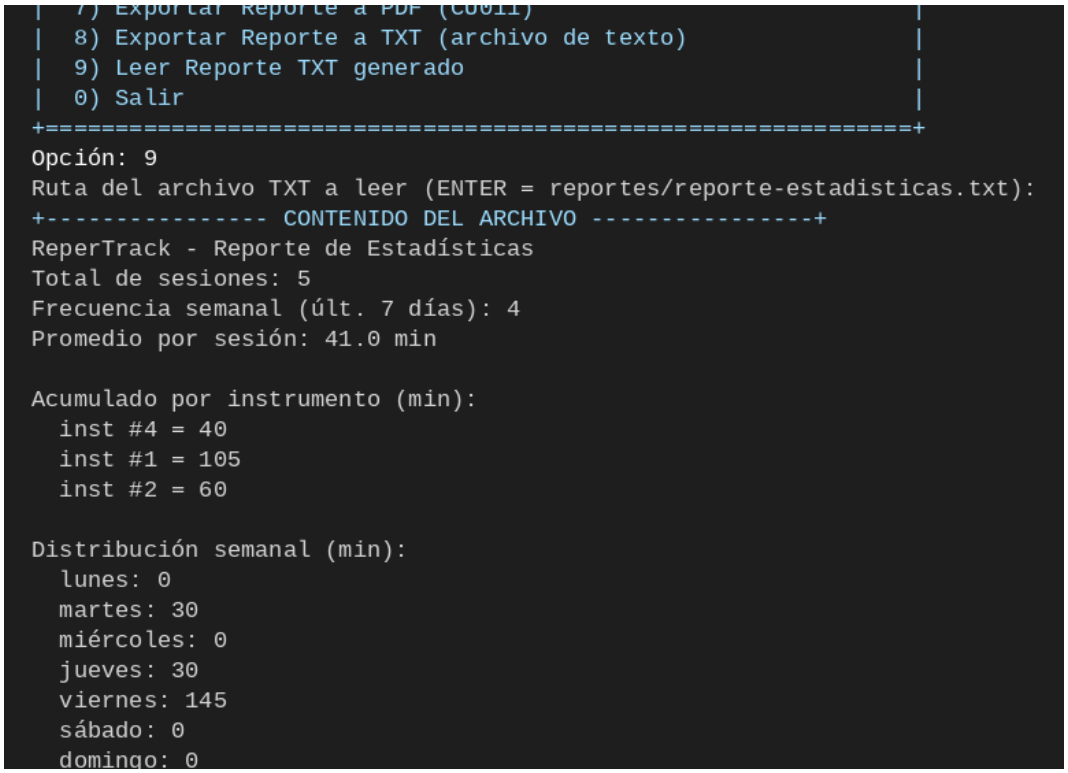


Figura B.6. Exportación y lectura del reporte TXT desde el menú.

19. MySQL de verdad: el 3307 y las pruebas de integridad

Para la evidencia no me conformé con el modo memoria. Descargué MySQL 8.0.40 portable (839 MB), lo inicialicé en un datadir temporal y lo levanté en el puerto 3307 para no chocar con el 3306. El default del proyecto sigue siendo 3306; el 3307 fue solo para la prueba, así que si tenés MySQL en el puerto estándar no hace falta que cambies nada. Creé el esquema con 01_creacion.sql, cargué 02_inserts.sql y ejecuté el programa con REPER_DB_URL apuntando ahí. La sesión 5 quedó guardada con 40 minutos y la verifiqué con un SELECT.

Después corrí 05_pruebas_integridad.sql con --force para que el cliente no se detuviera en el primer error. Los siete rechazos salieron con los códigos reales de MySQL: 3819 para los CHECK, 1452/1451 para las FK, 1062 para el UNIQUE y 1406 para el comentario de 501 caracteres. El INSERT válido pasó y el conteo final dio 1,1,1,1. Esa captura es la que pedía el TP2: integridad en MySQL, no en SQLite.

```
MySQL 8.0.40 — SELECT de verificación

+----------------+----------------------+---------+--------+
| instrumento_id | nombre               | tipo    | activo |
+----------------+----------------------+---------+--------+
|              1 | PRS Custom 24        | cuerda  |      1 |
|              2 | Taylor 814ce         | cuerda  |      1 |
|              3 | Piano digital Yamaha | teclado |      1 |
|              4 | Stratocaster 1979    | cuerda  |      1 |
+----------------+----------------------+---------+--------+
+-----------+----------------+-------------+-----------------+---------------------+
| sesion_id | instrumento_id | elemento_id | tiempo_neto_min | fecha_registro      |
+-----------+----------------+-------------+-----------------+---------------------+
|         5 |              4 |           6 |              40 | 2026-11-06 22:57:22 |
|         4 |              1 |           4 |              30 | 2026-09-22 19:57:06 |
|         3 |              2 |           3 |              60 | 2026-11-06 19:57:06 |
+-----------+----------------+-------------+-----------------+---------------------+
```

Figura B.9. Verificación en MySQL de la sesión insertada por el programa.

```
MySQL 8.0.40 — Pruebas de integridad

ERROR 3819 (HY000) at line 21: Check constraint 'chk_ses_tiempo' is violated.
ERROR 3819 (HY000) at line 25: Check constraint 'chk_elem_dif' is violated.
ERROR 3819 (HY000) at line 30: Check constraint 'chk_elem_bpm' is violated.
ERROR 1452 (23000) at line 35: Cannot add or update a child row: a foreign key constraint fails
  (`repertrack`.`sesion_practica`, CONSTRAINT `fk_ses_inst` FOREIGN KEY (`instrumento_id`) REFERENCES
  `instrumento` (`instrumento_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
ERROR 1062 (23000) at line 39: Duplicate entry '1-PRS Custom 24' for key 'instrumento.uq_inst_musico_nombre'
ERROR 1451 (23000) at line 43: Cannot delete or update a parent row: a foreign key constraint fails
  (`repertrack`.`instrumento`, CONSTRAINT `fk_inst_musico` FOREIGN KEY (`musico_id`) REFERENCES `musico`
  (`musico_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
ERROR 1406 (22001) at line 46: Data too long for column 'comentario' at row 1
musicos    instrumentos    elementos    sesiones
1    1    1    1
```

Figura B.10. Pruebas de integridad en MySQL: CHECK, FK y UNIQUE rechazan los casos inválidos.

20. Clases abstractas e interfaces: qué usé y qué no

Usé una clase abstracta, ElementoDeEstudio, porque hay atributos y comportamiento comunes y un método que cada subclase debe resolver sí o sí. Usé interfaces para los DAO porque hay una costura real: el TestRunner inyecta un stub y las pruebas no dependen de MySQL. Esa es la justificación de DIP que me importa: la interfaz se usa, no se decora.

No hice una interfaz para ExportadorTextoService. Tiene una sola implementación, ningún test necesita sustituirla y no hay un segundo formato con el mismo contrato. Si mañana agrego CSV o JSON, ese es el disparador para extraerla. Mientras tanto, una interfaz con una implementación sería una capa de más.

21. La lógica de negocio, regla por regla

La lógica vive en los servicios y en el modelo, no en la vista. RN-01 (toda sesión tiene instrumento y elemento) se valida en SesionDePractica.validar() y en SesionService. RN-02 (tiempo mayor a cero) se valida en el setter y en el servicio. RN-03 (fecha no futura) se controla al registrar. RN-04 (comentario de hasta 500) está en el setter. RN-05 (atomicidad) está en el DAO con commit/rollback.

El flujo es siempre el mismo: la vista pide los datos, el controlador los convierte a objetos, el servicio aplica las reglas y el DAO persiste. Si una regla falla, la excepción checked obliga al controlador a mostrar el mensaje y no se ejecuta el INSERT. Esa separación es la que permite que CP01..CP07 prueben las reglas sin abrir el menú.

22. Capturas, video y lo que quedó afuera

Las capturas del Anexo B son la salida real del programa contra MySQL: menú, alta de instrumento, alta de canción, registro de sesión, estadísticas con el arreglo, reporte TXT, errores y verificación en la base. No son mockups: las capturé desde un pseudo-terminal y las rendericé como imagen para que se lean en el PDF.

El video de presentación dura tres minutos y muestra ese mismo recorrido. Está disponible en https://youtu.be/8ptGI8U8vLI.

Quedaron afuera, a propósito: la reproducción de audio, la sincronización en la nube, el multiusuario y la autenticación remota. También quedó pendiente el fallback en memoria compartido entre controladores, que mencioné en la sección 15. No las borré de los requerimientos candidatos porque siguen siendo líneas naturales de evolución.

23. Correcciones y cierre

En el TP1 corregí los objetivos, el dominio y la propuesta técnica. En el TP2, la secuencia, la asociación de ElementoDeEstudio, el despliegue, el CU011, la nomenclatura de pruebas, el DER con tipos SQL y la integridad en MySQL. En el TP3, la lógica de negocio, las capturas y la evidencia de excepciones. En esta entrega agregué arreglos, archivos, MVC justificado y la evidencia real de MySQL.

Para que no quede en palabras, armé la tabla con cada observación y dónde quedó resuelta. Si algo no aparece acá, es porque se me pasó.

Entrega	Observación	Cómo lo corregí	Dónde se ve
TP1	Objetivos con redacción incompleta	Reescritos con un verbo, una idea y dos líneas	Sección 2
TP1	Objetivos del sistema	Separados del objetivo del proyecto	Sección 2
TP1	Diagrama de dominio	Atributos, multiplicidades y relación Músico–Sesión	Sección 3, Figura 1
TP1	Propuesta técnica con Swing/JavaFX	Consola pura y stack Java + MySQL + JDBC	Sección 4
TP1	Redacción de requerimientos	Tablas RFS01..RFS11 y RNF01..RNF08	Sección 5
TP2	Secuencia con clases que no estaban	Secuencia reescrita con las clases de diseño	Sección 7, Figura 2
TP2	instrumentoId como atributo	Asociación real a Instrumento	Sección 7, Figura 3
TP2	Despliegue sin definiciones	Artefactos, entorno y conector completos	Sección 8, Figura 4
TP2	CU011 sin especificar	Pre/post, curso, alternativas y excepciones	Sección 8
TP2	Códigos CPC/CPS vs CP	Nomenclatura única CP01..CP18	Sección 9
TP2	Faltaban entradas y esperados	Tabla con entrada y resultado de los 18	Sección 9
TP2	DER sin tipos SQL	Tipos explícitos por atributo	Sección 10, Figura 5
TP2	Integridad probada en SQLite	Pruebas en MySQL 8.0.40 real	Sección 10, Figura B.10
TP2	Prototipos de interfaz pendientes	Capturas reales de las pantallas	Sección 11, Anexo B
TP3	Lógica de negocio poco explicada	Reglas RN-01..RN-05, una por una	Sección 21
TP3	Faltaban capturas del programa	10 capturas de ejecución real	Anexo B
TP3	Excepciones sin evidencia	Capturas de entrada inválida y rango	Figuras B.7 y B.8

No voy a decir que quedó perfecto. El diagrama de clases es grande y en A4 se lee con esfuerzo; el fallback en memoria no comparte datos entre menús; el video ya está grabado. Pero el prototipo compila, corre, pasa 18 pruebas y hace lo que promete: registrar sesiones, persistirlas y mostrar progreso. Para un trabajo integrador, eso es lo que quería dejar.

Anexo A. Diagrama de clases completo

Los cinco diagramas del PUD están en las Partes I y II con su explicación. Acá va la vista completa del diagrama de clases, en horizontal porque en vertical no se leía. La tabla de paquetes ayuda a ubicar cada clase; el detalle está en la sección 7.

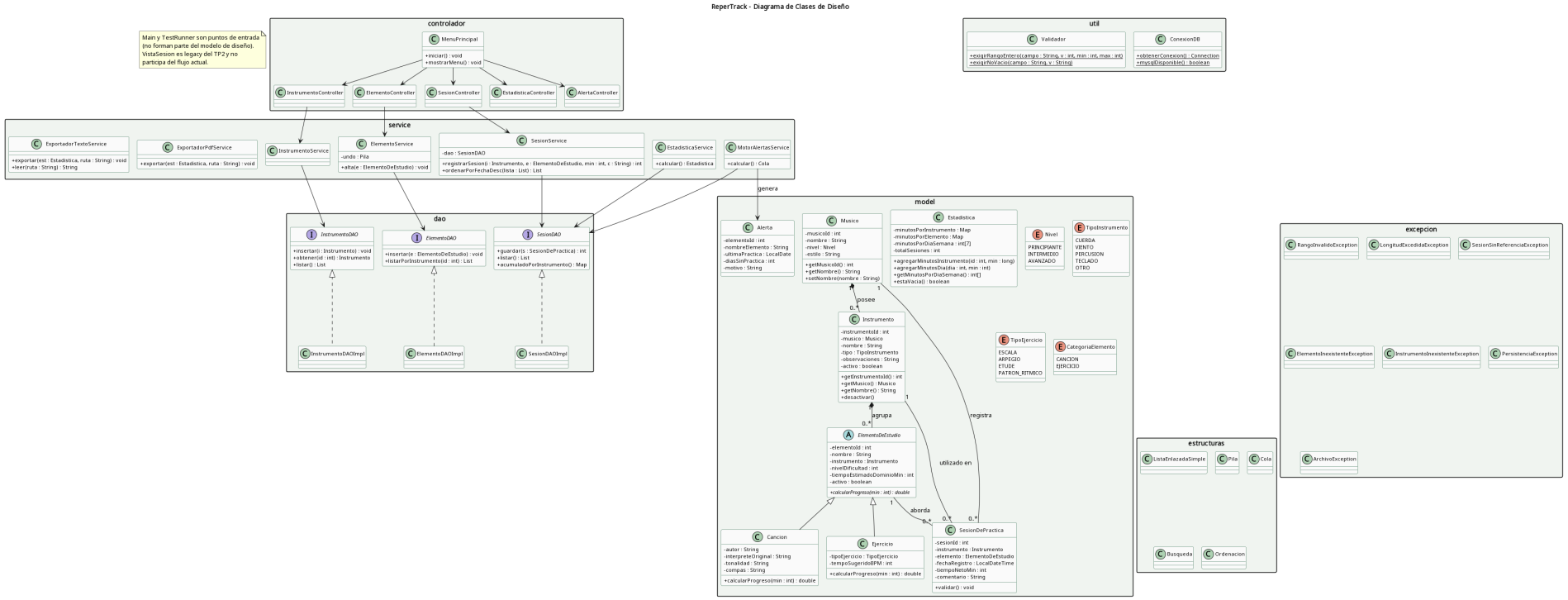


Figura A.1. Diagrama de clases de diseño sincronizado con el código.

Paquete	Qué hay adentro
model	Entidades del dominio, enums, Estadística y Alerta.
excepcion	Excepciones checked: rango, longitud, referencia, inexistencia, persistencia y archivo.
util	Conexión JDBC y validaciones centralizadas.
dao	Interfaces e implementaciones JDBC con fallback a memoria.
service	Reglas de negocio, estadísticas, alertas y exportadores PDF/TXT.
controlador	Menú de consola y subcontroladores por módulo.
estructuras	Lista enlazada, pila, cola, búsqueda y ordenación.

Anexo B. Capturas de ejecución

Todas las capturas son la salida real del programa contra MySQL 8.0.40. Les puse un marco de terminal para que se lean en el PDF; el texto es el que imprimió el programa.

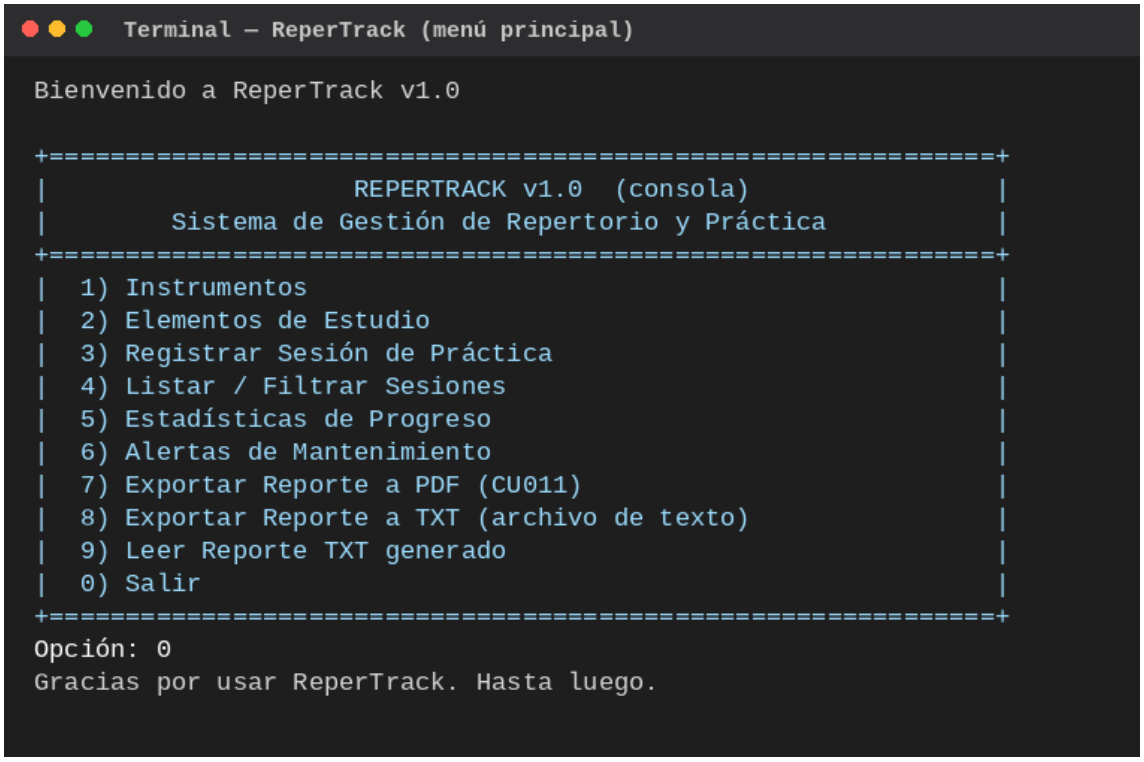


Figura B.1. Menú principal.

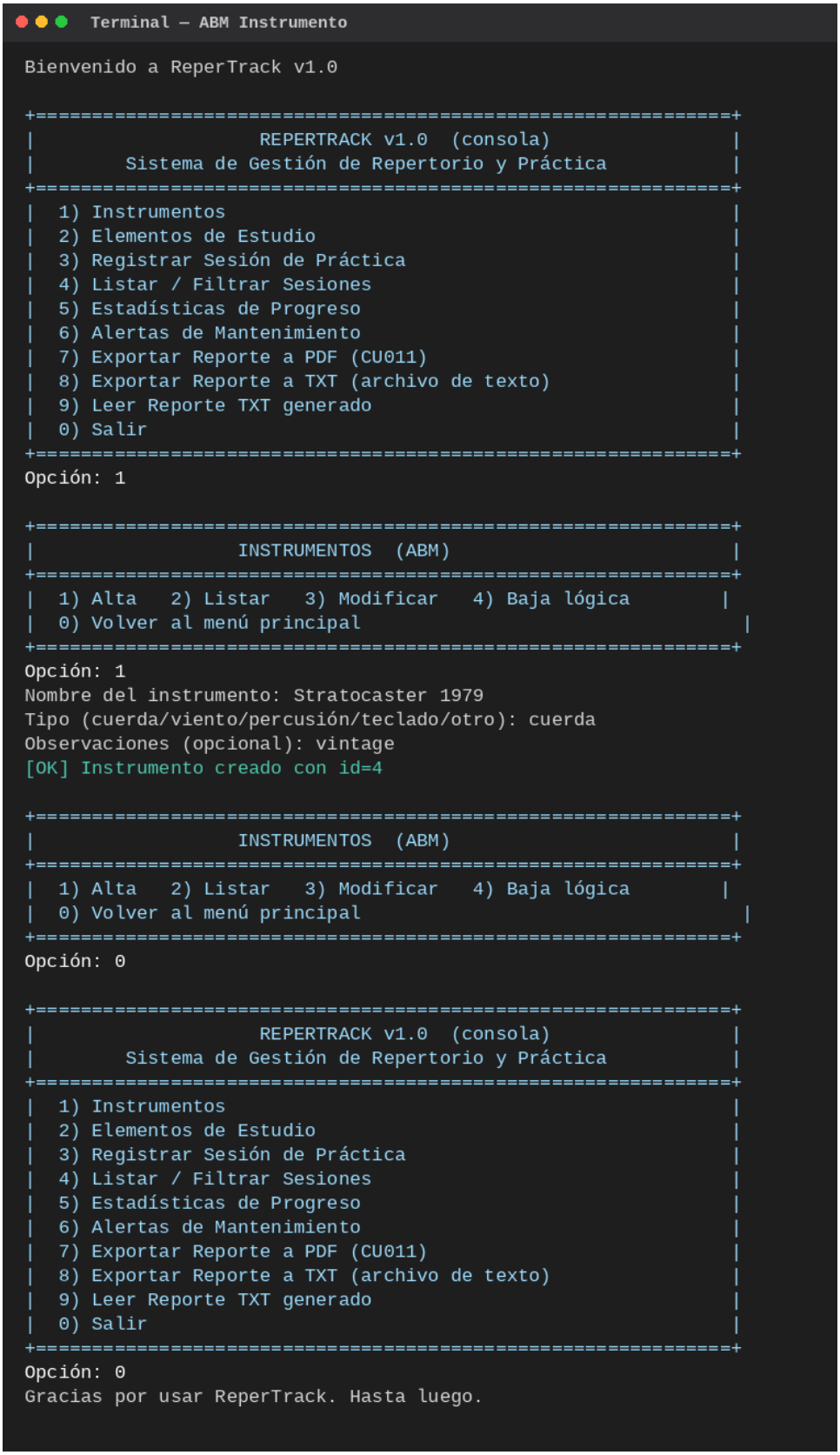


Figura B.2. Alta de instrumento.

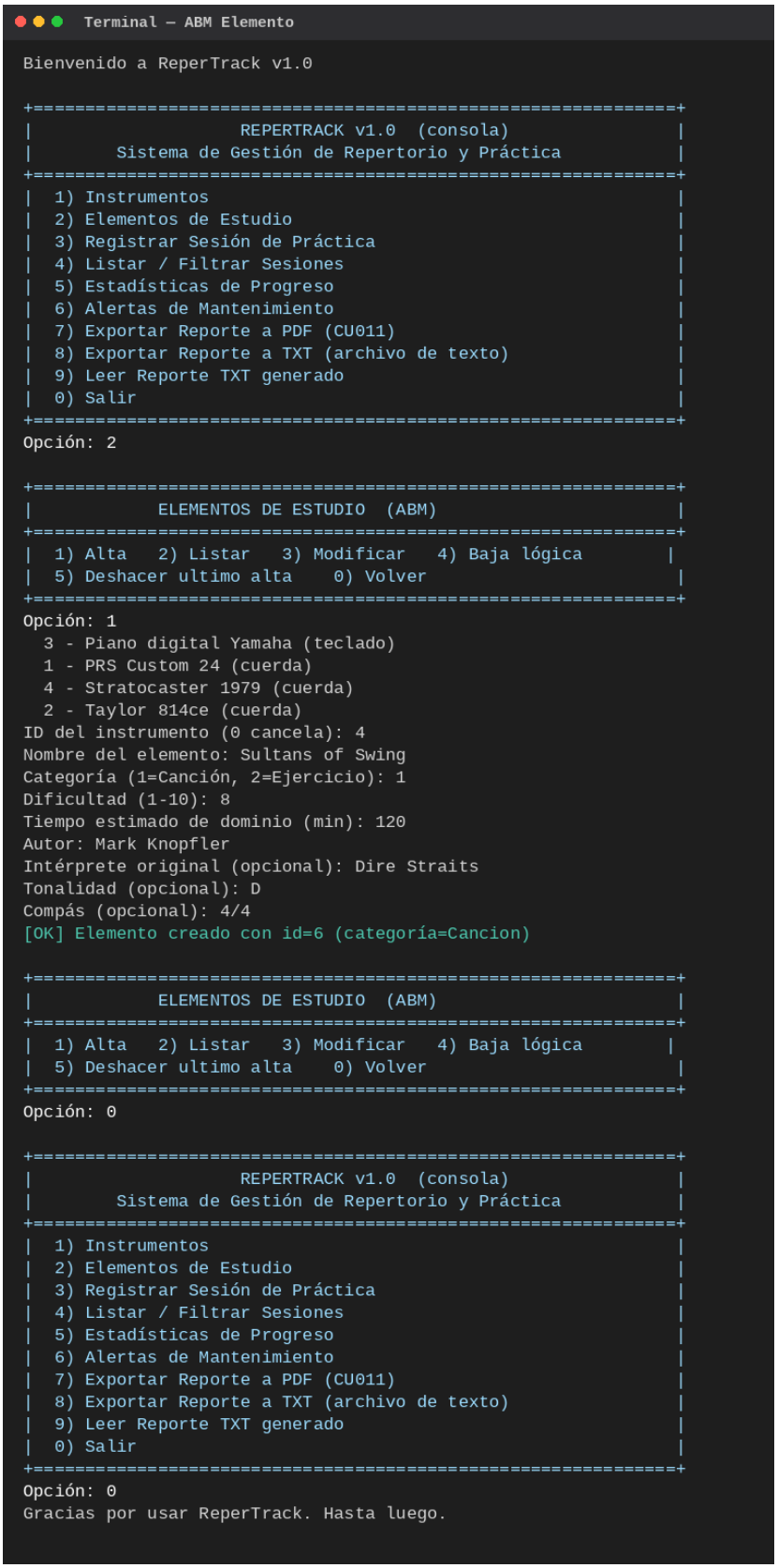


Figura B.3. Alta de elemento (canción).

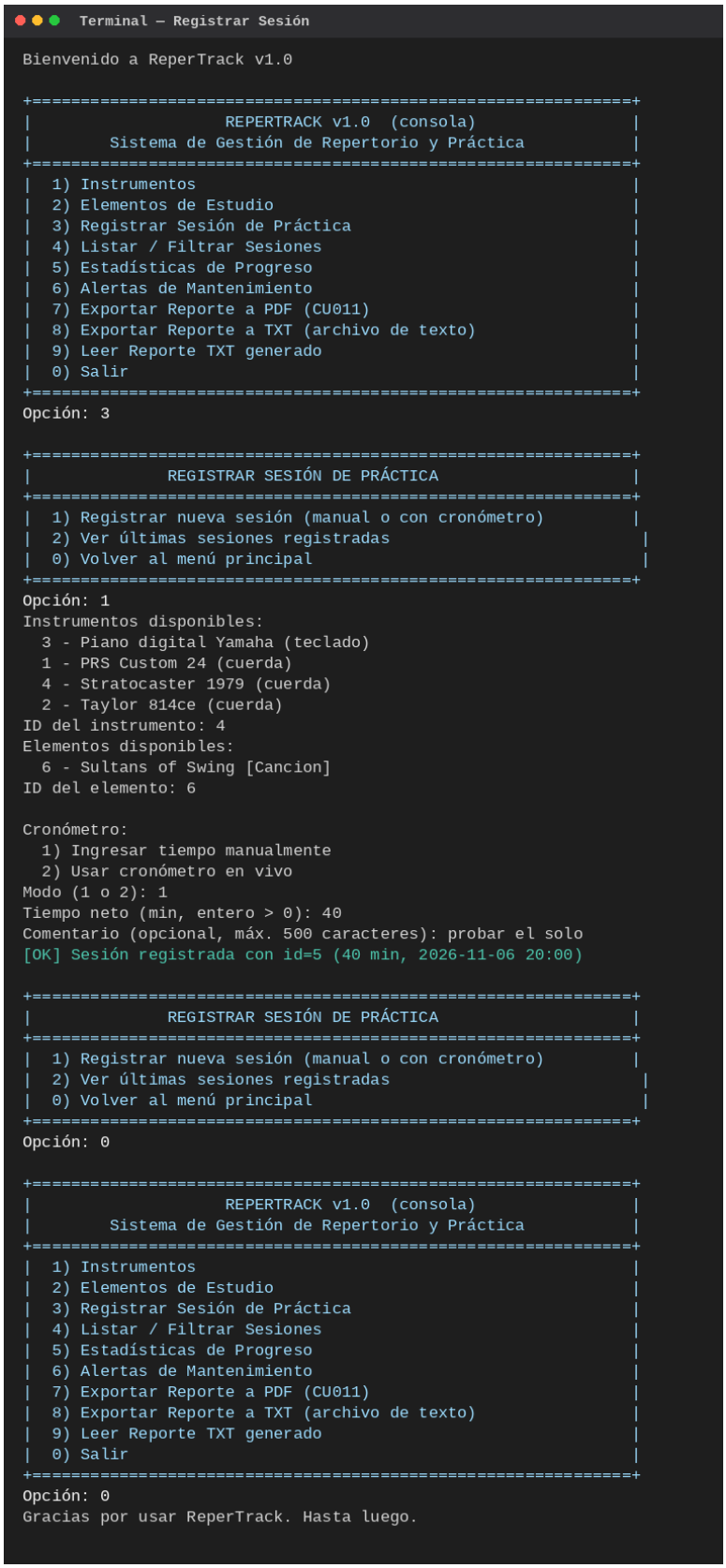


Figura B.4. Registro de sesión con instrumento y elemento.

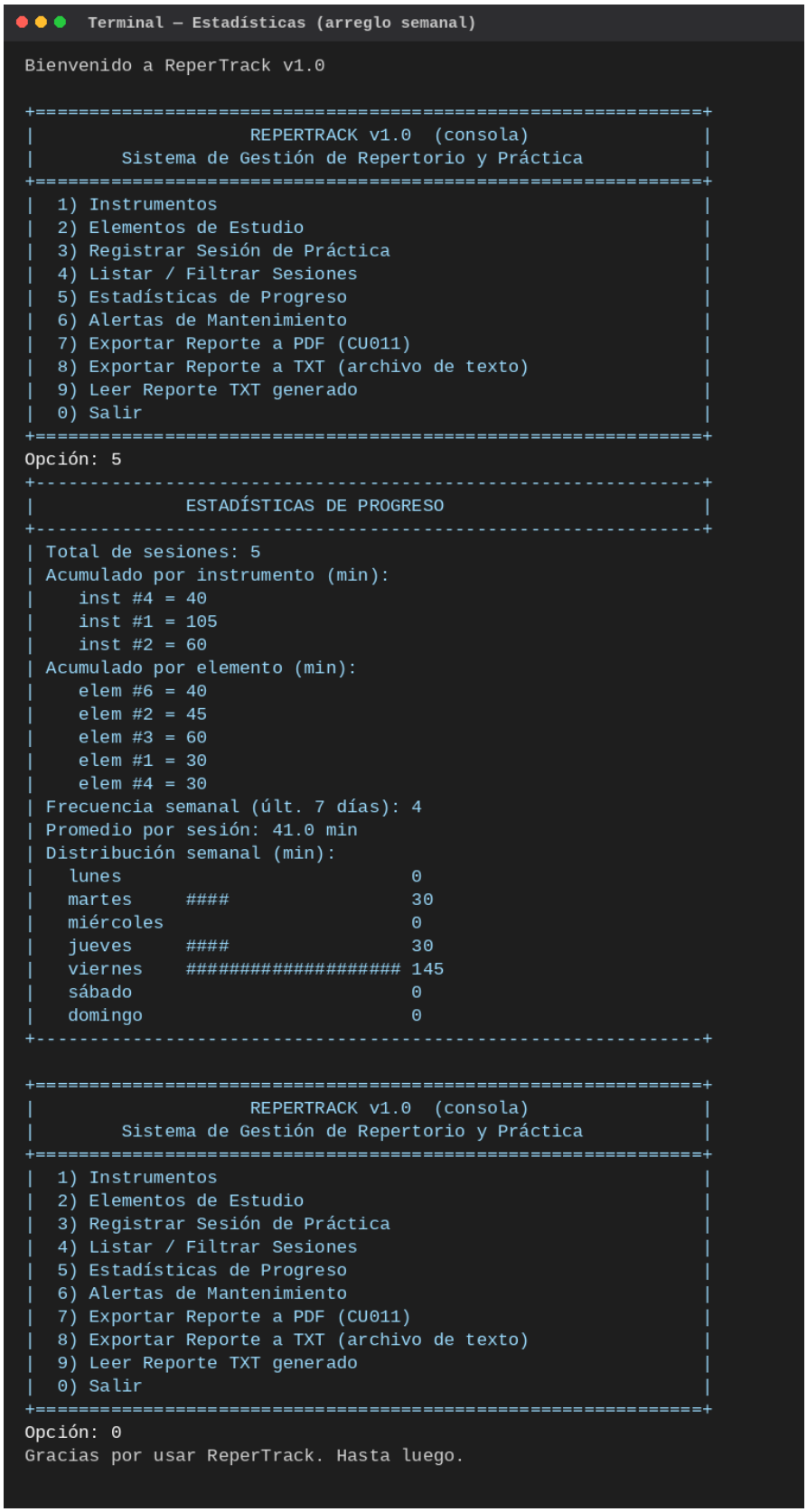


Figura B.5. Estadísticas y distribución semanal.

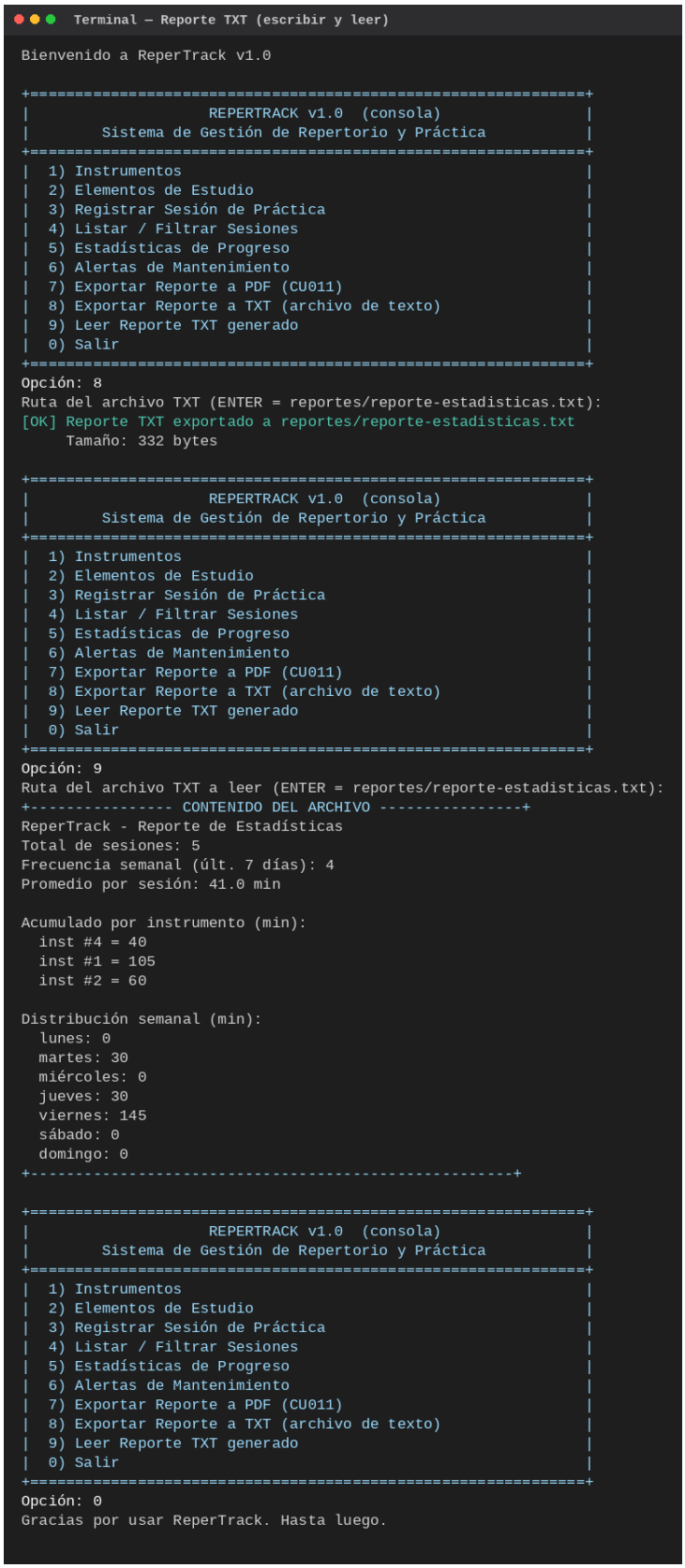


Figura B.6. Reporte TXT: escritura y lectura.

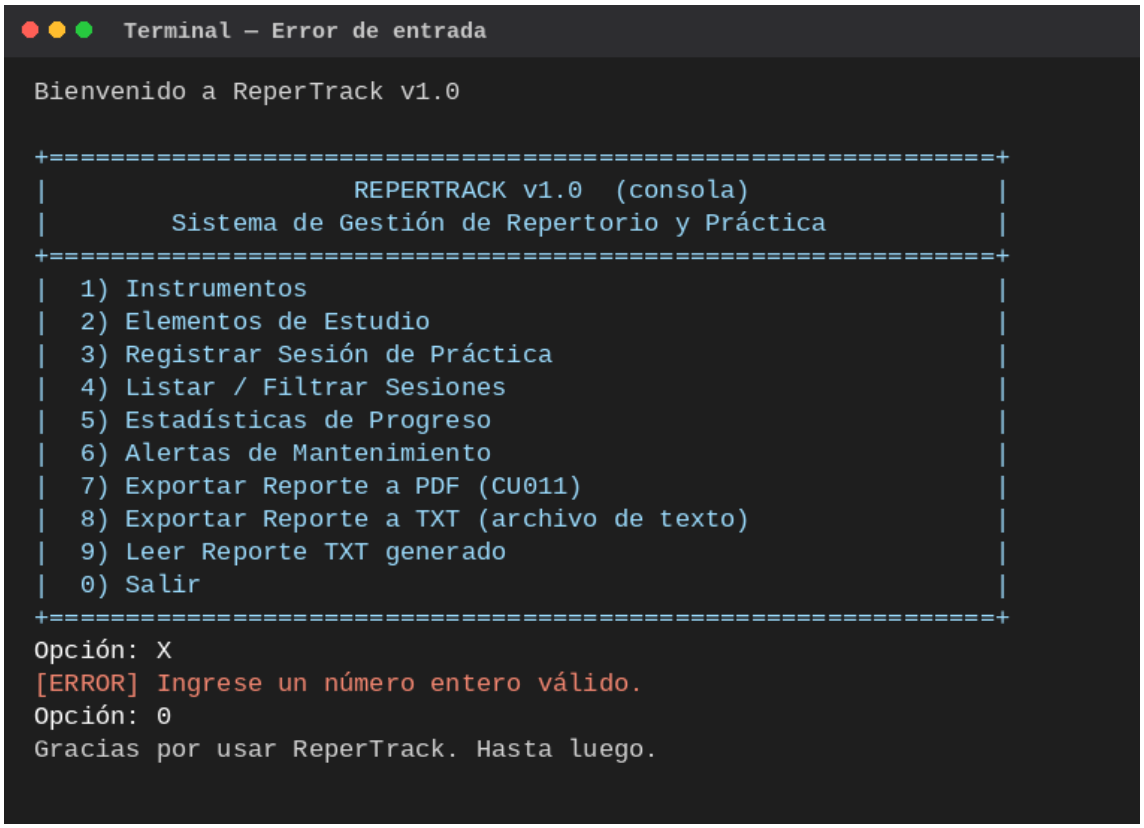


Figura B.7. Error de entrada inválida.

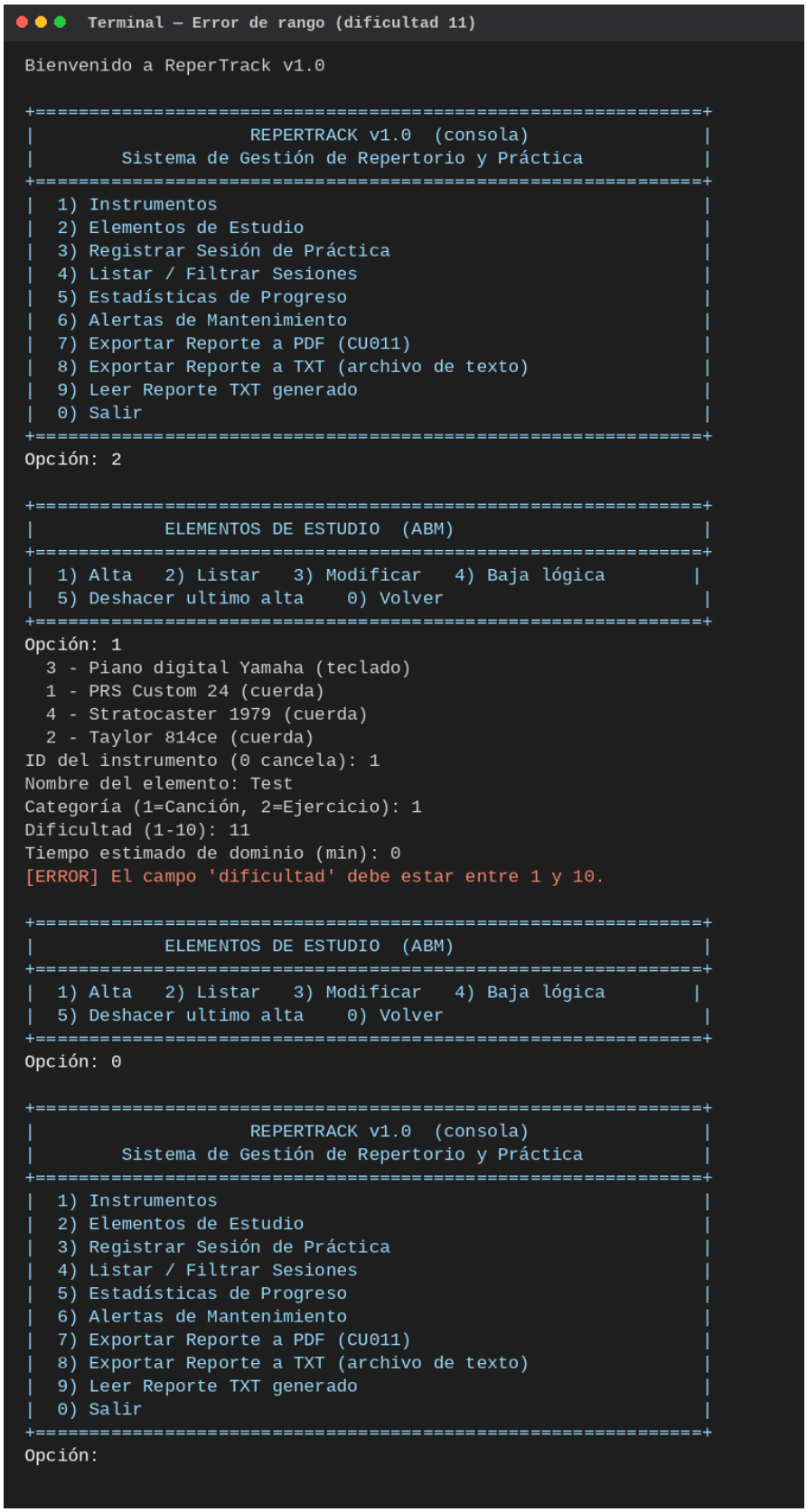


Figura B.8. Error de rango en la dificultad.

```
MySQL 8.0.40 — SELECT de verificación

+----------------+----------------------+---------+--------+
| instrumento_id | nombre               | tipo    | activo |
+----------------+----------------------+---------+--------+
|              1 | PRS Custom 24        | cuerda  |      1 |
|              2 | Taylor 814ce         | cuerda  |      1 |
|              3 | Piano digital Yamaha | teclado |      1 |
|              4 | Stratocaster 1979    | cuerda  |      1 |
+----------------+----------------------+---------+--------+
+-----------+----------------+-------------+-----------------+---------------------+
| sesion_id | instrumento_id | elemento_id | tiempo_neto_min | fecha_registro      |
+-----------+----------------+-------------+-----------------+---------------------+
|         5 |              4 |           6 |              40 | 2026-11-06 22:57:22 |
|         4 |              1 |           4 |              30 | 2026-09-22 19:57:06 |
|         3 |              2 |           3 |              60 | 2026-11-06 19:57:06 |
+-----------+----------------+-------------+-----------------+---------------------+
```

Figura B.9. Verificación de la fila en MySQL.

```
MySQL 8.0.40 — Pruebas de integridad

ERROR 3819 (HY000) at line 21: Check constraint 'chk_ses_tiempo' is violated.
ERROR 3819 (HY000) at line 25: Check constraint 'chk_elem_dif' is violated.
ERROR 3819 (HY000) at line 30: Check constraint 'chk_elem_bpm' is violated.
ERROR 1452 (23000) at line 35: Cannot add or update a child row: a foreign key constraint fails
  (`repertrack`.`sesion_practica`, CONSTRAINT `fk_ses_inst` FOREIGN KEY (`instrumento_id`) REFERENCES
  `instrumento` (`instrumento_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
ERROR 1062 (23000) at line 39: Duplicate entry '1-PRS Custom 24' for key 'instrumento.uq_inst_musico_nombre'
ERROR 1451 (23000) at line 43: Cannot delete or update a parent row: a foreign key constraint fails
  (`repertrack`.`instrumento`, CONSTRAINT `fk_inst_musico` FOREIGN KEY (`musico_id`) REFERENCES `musico`
  (`musico_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
ERROR 1406 (22001) at line 46: Data too long for column 'comentario' at row 1
musicos    instrumentos    elementos    sesiones
1    1    1    1
```

Figura B.10. Pruebas de integridad en MySQL.

Anexo C. Consultas SQL

Las consultas Q1..Q5 se ejecutan en el cliente MySQL y se corresponden con los casos de uso. Las copio para que se puedan probar.

```
-- Q1. Historial completo (CU007)
SELECT SP.sesion_id, M.nombre AS musico, I.nombre AS instrumento,
       EE.nombre AS elemento, EE.categoria, SP.tiempo_neto_min AS minutos,
       SP.fecha_registro AS fecha
  FROM sesion_practica SP
  JOIN instrumento I ON SP.instrumento_id = I.instrumento_id
  JOIN musico M ON I.musico_id = M.musico_id
  JOIN elemento_estudio EE ON SP.elemento_id = EE.elemento_id
 ORDER BY SP.fecha_registro DESC;
```

```
-- Q2. Acumulado por instrumento (CU009)
SELECT I.nombre AS instrumento, SUM(SP.tiempo_neto_min) AS minutos_totales
  FROM sesion_practica SP JOIN instrumento I ON SP.instrumento_id = I.instrumento_id
 GROUP BY I.nombre;
```

```
-- Q3. Acumulado por elemento (CU009)
SELECT EE.nombre AS elemento, SUM(SP.tiempo_neto_min) AS minutos_totales
  FROM sesion_practica SP JOIN elemento_estudio EE ON SP.elemento_id = EE.elemento_i
d
 GROUP BY EE.nombre;
```

```
-- Q4. Frecuencia y promedio (RFS09)
SELECT COUNT(*) AS sesiones_7d, AVG(tiempo_neto_min) AS promedio_min
  FROM sesion_practica WHERE fecha_registro >= NOW() - INTERVAL 7 DAY;
```

```
-- Q5. Alertas de mantenimiento (CU012)
SELECT EE.elemento_id, EE.nombre, MAX(SP.fecha_registro) AS ultima,
       DATEDIFF(NOW(), MAX(SP.fecha_registro)) AS dias
  FROM elemento_estudio EE LEFT JOIN sesion_practica SP ON SP.elemento_id = EE.eleme
nto_id
 WHERE EE.activo = 1 GROUP BY EE.elemento_id
HAVING ultima IS NULL OR DATEDIFF(NOW(), ultima) > 30;
```

Referencias bibliográficas

Braude, E. (2003). Ingeniería de software: una perspectiva orientada a objetos. Alfaomega.

Ericsson, K. A., Krampe, R. T., y Tesch-Römer, C. (1993). The role of deliberate practice in the acquisition of expert performance. Psychological Review, 100(3), 363-406.

Institute of Electrical and Electronics Engineers. (2008). Standard for Software Quality Assurance Plans (ANSI/IEEE Standard No. 829).

Jacobson, I., Booch, G., y Rumbaugh, J. (1999). The unified software development process. Addison-Wesley.

Kendall, K., y Kendall, J. (2011). Análisis y diseño de sistemas. Pearson Educación.

Newport, C. (2016). Deep work: Rules for focused success in a distracted world. Grand Central Publishing.

Reinosa, E. J., Maldonado, C. A., Muñoz, R., Damiano, L. E., y Abrutsky, M. A. (2012). Bases de datos. Alfaomega.

Rumbaugh, J., Jacobson, I., y Booch, G. (2006). El lenguaje unificado de modelado (2.ª ed.). Pearson-Addison-Wesley.

Saroka, R. (2002). Sistemas de información en la era digital. Fundación OSDE.

Sommerville, I. (2005). Ingeniería del software (7.ª ed.). Pearson-Addison-Wesley.

Stair, R., y Reynolds, G. (2010). Principios de sistemas de información (9.ª ed.). Cengage Learning.

Repositorio público del proyecto: https://github.com/duendecillo2/reper-track